United States Patent Application Publication	20250284832
Kind Code	A1
Publication Date	September 11, 2025
Inventor(s)	Durham; David M. et al.

CAPABILITY-BASED MEMORY ACCESS CONTROL FOR GRAPHICS PROCESSORS AND ACCELERATORS

Abstract

An apparatus to facilitate capability-based memory access control for graphics processors and accelerators is disclosed. The apparatus includes one or more processing cores to: determine that a set of data accesses triggers a bulk access capability check; generate the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, perform the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allow the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.

Inventors: **Durham; David M. (Beaverton, OR), LeMay; Michael (Hillsboro, OR)**

Applicant: **Intel Corporation** (Santa Clara, CA)

Family ID: **94605374**

Assignee: **Intel Corporation (Santa Clara, CA)**

Appl. No.: **18/946869**

Filed: **November 13, 2024**

Related U.S. Application Data

us-provisional-application US 63563575 20240311

Publication Classification

Int. Cl.: **G06F21/62** (20130101); **G06F21/60** (20130101)

U.S. Cl.:

CPC **G06F21/6218** (20130101); **G06F21/602** (20130101);

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a non-provisional application of U.S. Provisional Patent Application No. 63/563,575 filed Mar. 11, 2024, of which is incorporated herein by reference.

BACKGROUND OF THE DISCLOSURE
[0002] Current parallel graphics data processing includes systems and methods developed to perform specific operations on graphics data such as, for example, linear interpolation, tessellation, rasterization, texture mapping, depth testing, etc. Traditionally, graphics processors used fixed function computational units to process graphics data. However, more recently, portions of graphics processors have been made programmable, enabling such processors to support a wider variety of operations for processing vertex and fragment data.
[0003] To further increase performance, graphics processors typically implement processing techniques such as pipelining that attempt to process, in parallel, as much graphics data as possible throughout the different parts of the graphics pipeline. Parallel graphics processors with single instruction, multiple thread (SIMT) architectures are designed to maximize the amount of parallel processing in the graphics pipeline. In a SIMT architecture, groups of parallel threads attempt to execute program instructions synchronously together as often as possible to increase processing efficiency. A general overview of software and hardware for SIMT architectures can be found in Shane Cook, *CUDA Programming* Chapter 3, pages 37-51 (2013).

Description

BRIEF DESCRIPTION OF THE DRAWINGS
[0004] Embodiments described herein are illustrated by way of example and not limitation in the figures of the accompanying drawings in which like references indicate similar elements, and in which:
[0005] FIG. **1** is a block diagram illustrating a computer system configured to implement one or more aspects of the embodiments described herein;
[0006] FIG. **2**A-**2**E illustrate parallel processor components, including graphics multiprocessors;
[0007] FIG. **3** illustrates a graphics processing unit that includes dedicated sets of graphics processing resources arranged into multi-core groups;
[0008] FIG. **4**A-**4**E illustrate an example architecture in which a plurality of GPUs is communicatively coupled to a plurality of multi-core processors;
[0009] FIG. **5** illustrates a graphics processing pipeline;
[0010] FIG. **6** illustrates a machine learning software stack;
[0011] FIG. **7** illustrates a general-purpose graphics processing unit;
[0012] FIG. **8** illustrates a multi-GPU computing system;
[0013] FIG. **9**A-**9**B illustrate layers of example deep neural networks;
[0014] FIG. **10**A-**10**B illustrate example language models;
[0015] FIG. **11** illustrates training and deployment of a deep neural network;
[0016] FIG. **12**A is a block diagram illustrating distributed learning;
[0017] FIG. **12**B is a block diagram illustrating a programmable network interface and data processing unit;
[0018] FIG. **13** illustrates an example inferencing system on a chip (SOC) suitable for performing

inferencing using a trained model;
[0019] FIG. **14** is a block diagram of a processing system;
[0020] FIG. **15**A-**15**C illustrate computing systems and graphics processors;
[0021] FIG. **16** is a block diagram of a graphics processor, which may be a discrete or integrated graphics processing unit;
[0022] FIG. **17**A-**17**B illustrate block diagrams of additional graphics processor and compute accelerator architectures;
[0023] FIG. **18**A-**18**C illustrate thread execution logic including an array of processing elements employed in a graphics processor core;
[0024] FIG. **19** illustrates a tile of a multi-tile processor, according to an embodiment;
[0025] FIG. **20** is a block diagram illustrating graphics processor instruction formats;
[0026] FIG. **21** is a block diagram of an additional graphics processor architecture;
[0027] FIG. **22**A-**22**B illustrate a graphics processor command format and command sequence;
[0028] FIG. **23** illustrates example graphics software architecture for a data processing system;
[0029] FIG. **24** is a block diagram illustrating an IP core development system;
[0030] FIG. **25**A illustrates a cross-section side view of an integrated circuit package assembly that includes multiple units of hardware logic chiplets connected to a substrate (e.g., base die);
[0031] FIG. **25**B illustrates a package assembly including interchangeable chiplets;
[0032] FIG. **26** is a block diagram illustrating a system on a chip integrated circuit;
[0033] FIG. **27** is a block diagram illustrating an example computing system for providing capability-based memory access control for graphics processors and accelerators, according to implementations herein;
[0034] FIG. **28** is a block diagram illustrating an example capability management circuit for providing capability-based memory access control for graphics processors and accelerators, according to implementations herein;
[0035] FIG. **29** is a block diagram illustrating a computing system supporting default capability registers for multi-tenancy, in accordance with implementations herein;
[0036] FIG. **30** is a flow diagram illustrating an embodiment of a method for providing bulk capability checking in a capability-based memory access control for graphics processors and accelerators;
[0037] FIG. **31** is a flow diagram illustrating an embodiment of a method for providing memory-specific capability generation for capability-based memory access control for graphics processors and accelerators;
[0038] FIG. **32** is a flow diagram illustrating an embodiment of a method for providing capability location designation for capability-based memory access control for graphics processors and accelerators;
[0039] FIG. **33** is a flow diagram illustrating an embodiment of a method for providing capability decryption and configuration for capability-based memory access control for graphics processors and accelerators; and
[0040] FIG. **34** is a flow diagram illustrating an embodiment of a method for providing default capability generation for capability-based memory access control for graphics processors and accelerators.

DETAILED DESCRIPTION

[0041] Current parallel graphics data processing includes systems and methods developed to perform specific operations on graphics data such as, for example, linear interpolation, tessellation, rasterization, texture mapping, depth testing, etc. Traditionally, graphics processors used fixed function computational units to process graphics data. However, more recently, portions of graphics processors have been made programmable, enabling such processors to support a wider variety of operations for processing vertex and fragment data.
[0042] To further increase performance, graphics processors typically implement processing

techniques such as pipelining that attempt to process, in parallel, as much graphics data as possible throughout the different parts of the graphics pipeline. Parallel graphics processors with single instruction, multiple thread (SIMT) architectures are designed to maximize the amount of parallel processing in the graphics pipeline. In a SIMT architecture, groups of parallel threads attempt to execute program instructions synchronously together as often as possible to increase processing efficiency.

[0043] A graphics processing unit (GPU) is communicatively coupled to host/processor cores to accelerate, for example, graphics operations, machine-learning operations, pattern analysis operations, and/or various general-purpose GPU (GPGPU) functions. The GPU may be communicatively coupled to the host processor/cores over a bus or another interconnect (e.g., a high-speed interconnect such as PCIe or NVLink). Alternatively, the GPU may be integrated on the same package or chip as the cores and communicatively coupled to the cores over an internal processor bus/interconnect (i.e., internal to the package or chip). Regardless of the manner in which the GPU is connected, the processor cores may allocate work to the GPU in the form of sequences of commands/instructions contained in a work descriptor. The GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.

[0044] While the techniques described herein are primarily discussed in the context of a GPU, techniques may also be implemented in other types of processors, including but not limited to general-purpose processors and accelerator devices, such as artificial intelligence accelerators, vision processors, and neural processing units.

[0045] In the following description, numerous specific details are set forth to provide a more thorough understanding. However, it will be apparent to one of skill in the art that the embodiments described herein may be practiced without one or more of these specific details. In other instances, well-known features have not been described to avoid obscuring the details of the present embodiments.

System Overview

[0046] FIG. **1** is a block diagram illustrating a computing system **100** configured to implement one or more aspects of the embodiments described herein. The computing system **100** includes a processing subsystem **101** having one or more processor(s) **102** and a system memory **104** communicating via an interconnection path that may include a memory hub **105**. The memory hub **105** may be a separate component within a chipset component or may be integrated within the one or more processor(s) **102**. The memory hub **105** couples with an I/O subsystem **111** via a communication link **106**. The I/O subsystem **111** includes an I/O hub **107** that can enable the computing system **100** to receive input from one or more input device(s) **108**. Additionally, the I/O hub **107** can enable a display controller, which may be included in the one or more processor(s) **102**, to provide outputs to one or more display device(s) **110**A. In one embodiment the one or more display device(s) **110**A coupled with the I/O hub **107** can include a local, internal, or embedded display device.

[0047] The processing subsystem **101**, for example, includes one or more parallel processor(s) **112** coupled to memory hub **105** via a communication link **113**, such as a bus or fabric. The communication link **113** may be one of any number of standards-based communication link technologies or protocols, such as, but not limited to PCI Express, or may be a vendor specific communications interface or communications fabric. The one or more parallel processor(s) **112** may form a computationally focused parallel or vector processing system that can include a large number of processing cores and/or processing clusters, such as a many integrated core (MIC) processor. For example, the one or more parallel processor(s) **112** form a graphics processing subsystem that can output pixels to one of the one or more display device(s) **110**A coupled via the I/O hub **107**. The one or more parallel processor(s) **112** can also include a display controller and display interface (not shown) to enable a direct connection to one or more display device(s) **110**B.

[0048] Within the I/O subsystem **111**, a system storage unit **114** can connect to the I/O hub **107** to

provide a storage mechanism for the computing system **100**. An I/O switch **116** can be used to provide an interface mechanism to enable connections between the I/O hub **107** and other components, such as a network adapter **118** and/or wireless network adapter **119** that may be integrated into the platform, and various other devices that can be added via one or more add-in device(s) **120**. The add-in device(s) **120** may also include, for example, one or more external graphics processor devices, graphics cards, and/or compute accelerators. The network adapter **118** can be an Ethernet adapter or another wired network adapter. The wireless network adapter **119** can include one or more of a Wi-Fi, Bluetooth, near field communication (NFC), or other network device that includes one or more wireless radios.

[0049] The computing system **100** can include other components not explicitly shown, including USB or other port connections, optical storage drives, video capture devices, and the like, which may also be connected to the I/O hub **107**. Communication paths interconnecting the various components in FIG. **1** may be implemented using any suitable protocols, such as PCI (Peripheral Component Interconnect) based protocols (e.g., PCI-Express), or any other bus or point-to-point communication interfaces and/or protocol(s), such as the NVLink high-speed interconnect, Compute Express Link™ (CXL™) (e.g., CXL.mem), Infinity Fabric (IF), Ethernet (IEEE 802.3), remote direct memory access (RDMA), InfiniBand, Internet Wide Area RDMA Protocol (iWARP), Transmission Control Protocol (TCP), User Datagram Protocol (UDP), quick UDP Internet Connections (QUIC), RDMA over Converged Ethernet (ROCE), Intel QuickPath Interconnect (QPI), Intel Ultra Path Interconnect (UPI), Intel On-Chip System Fabric (IOSF), Omnipath, HyperTransport, Advanced Microcontroller Bus Architecture (AMBA) interconnect, OpenCAPI, Gen-Z, Cache Coherent Interconnect for Accelerators (CCIX), 3GPP Long Term Evolution (LTE) (4G), 3GPP 5G, and variations thereof, or wired or wireless interconnect protocols known in the art. In some examples, data can be copied or stored to virtualized storage nodes using a protocol such as non-volatile memory express (NVMe) over Fabrics (NVMe-oF) or NVMe.

[0050] The one or more parallel processor(s) **112** may incorporate circuitry optimized for graphics and video processing, including, for example, video output circuitry, and constitutes a graphics processing unit (GPU). Alternatively or additionally, the one or more parallel processor(s) **112** can incorporate circuitry optimized for general purpose processing, while preserving the underlying computational architecture, described in greater detail herein. Components of the computing system **100** may be integrated with one or more other system elements on a single integrated circuit. For example, the one or more parallel processor(s) **112**, memory hub **105**, processor(s) **102**, and I/O hub **107** can be integrated into a system on chip (SoC) integrated circuit. Alternatively, the components of the computing system **100** can be integrated into a single package to form a system in package (SIP) configuration. In one embodiment at least a portion of the components of the computing system **100** can be integrated into a multi-chip module (MCM), which can be interconnected with other multi-chip modules into a modular computing system.

In some configurations, the computing system **100** includes one or more accelerator device(s) **130** coupled with the memory hub **105**, in addition to the processor(s) **102** and the one or more parallel processor(s) **112**. The accelerator device(s) **130** are configured to perform domain specific acceleration of workloads to handle tasks that are computationally intensive or utilize high throughput. The accelerator device(s) **130** can reduce the burden placed on the processor(s) **102** and/or parallel processor(s) **112** of the computing system **100**. The accelerator device(s) **130** can include but are not limited to smart network interface cards, data processing units, cryptographic accelerators, storage accelerators, artificial intelligence (AI) accelerators, neural processing units (NPUs), storage accelerators, and/or video transcoding accelerators.

[0051] It will be appreciated that the computing system **100** shown herein is illustrative and that variations and modifications are possible. The connection topology, including the number and arrangement of bridges, the number of processor(s) **102**, and the number of parallel processor(s) **112**, may be modified as desired. For instance, system memory **104** can be connected to the

processor(s) **102** directly rather than through a bridge, while other devices communicate with system memory **104** via the memory hub **105** and the processor(s) **102**. In other alternative topologies, the parallel processor(s) **112** are connected to the I/O hub **107** or directly to one of the one or more processor(s) **102**, rather than to the memory hub **105**. In other embodiments, the I/O hub **107** and memory hub **105** may be integrated into a single chip. It is also possible that two or more sets of processor(s) **102** are attached via multiple sockets, which can couple with two or more instances of the parallel processor(s) **112**.

[0052] Some of the particular components shown herein are optional and may not be included in all implementations of the computing system **100**. For example, any number of add-in cards or peripherals may be supported, or some components may be eliminated. Furthermore, some architectures may use different terminology for components similar to those illustrated in FIG. **1**. For example, the memory hub **105** may be referred to as a Northbridge in some architectures, while the I/O hub **107** may be referred to as a Southbridge.

[0053] FIG. **2**A illustrates a parallel processor **200**. The parallel processor **200** may be a GPU, GPGPU or the like as described herein. The various components of the parallel processor **200** may be implemented using one or more integrated circuit devices, such as programmable processors, application specific integrated circuits (ASICs), or field programmable gate arrays (FPGA). The parallel processor **200** may be one or more of the parallel processor(s) **112** shown in FIG. **1**.

[0054] The parallel processor **200** includes a parallel processing unit **202**. The parallel processing unit includes an I/O unit **204** that enables communication with other devices, including other instances of the parallel processing unit **202**. The I/O unit **204** may be directly connected to other devices. For instance, the I/O unit **204** connects with other devices via the use of a hub or switch interface, such as memory hub **105**. The connections between the memory hub **105** and the I/O unit **204** form a communication link **113**. Within the parallel processing unit **202**, the I/O unit **204** connects with a host interface **206** and a memory crossbar **216**, where the host interface **206** receives commands directed to performing processing operations and the memory crossbar **216** receives commands directed to performing memory operations. In one embodiment, the I/O unit **204** is configured to enable secure I/O operations via Trusted Execution Environment (TEE)-I/O support. TEE-IO enables trusted I/O virtualization, in which a trust relationship can be established directly between a secure virtual environment, such as trusted virtual machine, and the parallel processor **200** or secure partitions of the parallel processor.

[0055] When the host interface **206** receives a command buffer via the I/O unit **204**, the host interface **206** can direct work operations to perform those commands to a front end **208**. In one embodiment the front end **208** couples with a scheduler **210**, which is configured to distribute commands or other work items to a processing cluster array **212**. The scheduler **210** ensures that the processing cluster array **212** is properly configured and in a valid state before tasks are distributed to the processing clusters of the processing cluster array **212**. The scheduler **210** may be implemented via firmware logic executing on a microcontroller. The microcontroller implemented scheduler **210** is configurable to perform complex scheduling and work distribution operations at coarse and fine granularity, enabling rapid preemption and context switching of threads executing on the processing cluster array **212**. Preferably, the host software can prove workloads for scheduling on the processing cluster array **212** via one of multiple graphics processing doorbells. In other examples, polling for new workloads or interrupts can be used to identify or indicate availability of work to perform. The workloads can then be automatically distributed across the processing cluster array **212** by the scheduler **210** logic within the scheduler microcontroller.

[0056] The processing cluster array **212** can include up to “N” processing clusters (e.g., cluster **214**A, cluster **214**B, through cluster **214**N). Each cluster **214**A-**214**N of the processing cluster array **212** can execute a large number of concurrent threads. The scheduler **210** can allocate work to the clusters **214**A-**214**N of the processing cluster array **212** using various scheduling and/or work distribution algorithms, which may vary depending on the workload arising for each type of

program or computation. The scheduling can be handled dynamically by the scheduler **210** or can be assisted in part by compiler logic during compilation of program logic configured for execution by the processing cluster array **212**. Optionally, different clusters **214**A-**214**N of the processing cluster array **212** can be allocated for processing different types of programs or for performing different types of computations.

[0057] The processing cluster array **212** can be configured to perform various types of parallel processing operations. For example, the processing cluster array **212** is configured to perform general-purpose parallel compute operations. For example, the processing cluster array **212** can include logic to execute processing tasks including filtering of video and/or audio data, performing modeling operations, including physics operations, and performing data transformations.

[0058] The processing cluster array **212** is configured to perform parallel graphics processing operations. In such embodiments in which the parallel processor **200** is configured to perform graphics processing operations, the processing cluster array **212** can include additional logic to support the execution of such graphics processing operations, including, but not limited to texture sampling logic to perform texture operations, as well as tessellation logic and other vertex processing logic. Additionally, the processing cluster array **212** can be configured to execute graphics processing related shader programs such as, but not limited to vertex shaders, tessellation shaders, geometry shaders, and pixel shaders. The parallel processing unit **202** can transfer data from system memory via the I/O unit **204** for processing. During processing the transferred data can be stored to on-chip memory (e.g., parallel processor memory **222**) during processing, then written back to system memory.

[0059] In embodiments in which the parallel processing unit **202** is used to perform graphics processing, the scheduler **210** may be configured to divide the processing workload into approximately equal sized tasks, to better enable distribution of the graphics processing operations to multiple clusters **214**A-**214**N of the processing cluster array **212**. In some of these embodiments, portions of the processing cluster array **212** can be configured to perform different types of processing. For example, a first portion may be configured to perform vertex shading and topology generation, a second portion may be configured to perform tessellation and geometry shading, and a third portion may be configured to perform pixel shading or other screen space operations, to produce a rendered image for display. Intermediate data produced by one or more of the clusters **214**A-**214**N may be stored in buffers to allow the intermediate data to be transmitted between clusters **214**A-**214**N for further processing.

[0060] During operation, the processing cluster array **212** can receive processing tasks to be executed via the scheduler **210**, which receives commands defining processing tasks from front end **208**. For graphics processing operations, processing tasks can include indices of data to be processed, e.g., surface (patch) data, primitive data, vertex data, and/or pixel data, as well as state parameters and commands defining how the data is to be processed (e.g., what program is to be executed). The scheduler **210** may be configured to fetch the indices corresponding to the tasks or may receive the indices from the front end **208**. The front end **208** can be configured to ensure the processing cluster array **212** is configured to a valid state before the workload specified by incoming command buffers (e.g., batch-buffers, push buffers, etc.) is initiated.

[0061] Each of the one or more instances of the parallel processing unit **202** can couple with parallel processor memory **222**. The parallel processor memory **222** can be accessed via the memory crossbar **216**, which can receive memory requests from the processing cluster array **212** as well as the I/O unit **204**. The memory crossbar **216** can access the parallel processor memory **222** via a memory interface **218**. The memory interface **218** can include multiple partition units (e.g., partition unit **220**A, partition unit **220**B, through partition unit **220**N) that can each couple to a portion (e.g., memory unit) of parallel processor memory **222**. The number of partition units **220**A-**220**N may be configured to be equal to the number of memory units, such that each partition unit **220**A**0**-**220**) has a corresponding memory unit **224**A-**224**N. In other embodiments, the number of

partition units **220**A-**220**N may not be equal to the number of memory units.
[0062] The memory units **224**A-**224**N can include various types of memory devices, including dynamic random-access memory (DRAM) or graphics random access memory, such as synchronous graphics random access memory (SGRAM), including graphics double data rate (GDDR) memory. Optionally, the memory units **224**A-**224**N may also include 3D stacked memory, including but not limited to high bandwidth memory (HBM). Persons skilled in the art will appreciate that the specific implementation of the memory units **224**A-**224**N can vary and can be selected from one of various conventional designs. Render targets, such as frame buffers or texture maps may be stored across the memory units **224**A-**224**N, allowing partition units **220**A-**220**N to write portions of each render target in parallel to efficiently use the available bandwidth of parallel processor memory **222**. In some embodiments, a local instance of the parallel processor memory **222** may be excluded in favor of a unified memory design that utilizes system memory in conjunction with local cache memory.
[0063] Optionally, any one of the clusters **214**A-**214**N of the processing cluster array **212** has the ability to process data that will be written to any of the memory units **224**A-**224**N within parallel processor memory **222**. The memory crossbar **216** can be configured to transfer the output of each cluster **214**A-**214**N to any partition unit **220**A-**220**N or to another cluster **214**A-**214**N, which can perform additional processing operations on the output. Each cluster **214**A-**214**N can communicate with the memory interface **218** through the memory crossbar **216** to read from or write to various external memory devices. In one of the embodiments with the memory crossbar **216** the memory crossbar **216** has a connection to the memory interface **218** to communicate with the I/O unit **204**, as well as a connection to a local instance of the parallel processor memory **222**, enabling the processing units within the different processing clusters **214**A-**214**N to communicate with system memory or other memory that is not local to the parallel processing unit **202**. Generally, the memory crossbar **216** may, for example, be able to use virtual channels to separate traffic streams between the clusters **214**A-**214**N and the partition units **220**A-**220**N.
[0064] While a single instance of the parallel processing unit **202** is illustrated within the parallel processor **200**, any number of instances of the parallel processing unit **202** can be included. For example, multiple instances of the parallel processing unit **202** can be provided on a single add-in card, or multiple add-in cards can be interconnected. For example, the parallel processor **200** can be an add-in device, included in the add-in device(s) **120** of FIG. **1**, which may be a graphics card such as a discrete graphics card that includes one or more GPUs, one or more memory devices, and device-to-device or network or fabric interfaces. The different instances of the parallel processing unit **202** can be configured to inter-operate even if the different instances have different numbers of processing cores, different amounts of local parallel processor memory, and/or other configuration differences. Optionally, some instances of the parallel processing unit **202** can include higher precision floating-point units relative to other instances. Systems incorporating one or more instances of the parallel processing unit **202** or the parallel processor **200** can be implemented in a variety of configurations and form factors, including but not limited to desktop, laptop, or handheld personal computers, servers, workstations, game consoles, and/or embedded systems. An orchestrator can form composite nodes for workload performance using one or more of: disaggregated processor resources, cache resources, memory resources, storage resources, and networking resources.
[0065] In one embodiment, the parallel processing unit **202** can be partitioned into multiple instances. Those multiple instances can be configured to execute workloads associated with different clients in an isolated manner, enabling a pre-determined quality of service to be provided for each client. For example, each cluster **214**A-**214**N can be compartmentalized and isolated from other clusters, allowing the processing cluster array **212** to be divided into multiple compute partitions or instances. In such configuration, workloads that execute on an isolated partition are protected from faults or errors associated with a different workload that executes on a different

partition. The partition units **220**A-**220**N can be configured to enable a dedicated and/or isolated path to memory for the clusters **214**A-**214**N associated with the respective compute partitions. This datapath isolation enables the compute resources within a partition can communicate with one or more assigned memory units **224**A-**224**N without being subjected to interference by the activities of other partitions. In one embodiment, datapath isolation can be enhanced by encrypting data in memory of the various partitions with encryption keys that are specific to an associated partition, such that data is secured on a per-partition basis while at rest and in transit.

[0066] FIG. **2**B is a block diagram of a partition unit **220**. The partition unit **220** may be an instance of one of the partition units **220**A-**220**N of FIG. **2**A. As illustrated, the partition unit **220** includes an L2 cache **221**, a frame buffer interface **225**, and a ROP **226** (raster operations unit). The L2 cache **221** is a read/write cache that is configured to perform load and store operations received from the memory crossbar **216** and ROP **226**. Read misses and write-back requests are output by L2 cache **221** to frame buffer interface **225** for processing. Updates can also be sent to the frame buffer via the frame buffer interface **225** for processing. In one embodiment, the frame buffer interface **225** interfaces with a memory unit **224** of the memory units **224**A-**224**N within parallel processor memory **222** of FIG. **2**A. The partition unit **220** may additionally or alternatively also interface with one of the memory units in parallel processor memory via a memory controller (not shown).

[0067] In graphics applications, the ROP **226** is a processing unit that performs raster operations such as stencil, z test, blending, and the like. The ROP **226** then outputs processed graphics data that is stored in graphics memory. In some embodiments the ROP **226** includes or couples with a CODEC **227** that includes compression logic to compress depth or color data that is written to memory or the L2 cache **221** and decompress depth or color data that is read from memory or the L2 cache **221**. The compression logic can be lossless compression logic that makes use of one or more of multiple compression algorithms. The type of compression that is performed by the CODEC **227** can vary based on the statistical characteristics of the data to be compressed. For example, in one embodiment, delta color compression is performed on depth and color data on a per-tile basis. In one embodiment the CODEC **227** includes compression and decompression logic that can compress and decompress compute data associated with machine learning operations. The CODEC **227** can, for example, compress sparse matrix data for sparse machine learning operations. The CODEC **227** can also compress sparse matrix data that is encoded in a sparse matrix format (e.g., coordinate list encoding (COO), compressed sparse row (CSR), compress sparse column (CSC), etc.) to generate compressed and encoded sparse matrix data. The compressed and encoded sparse matrix data can be decompressed and/or decoded before being processed by processing elements or the processing elements can be configured to consume compressed, encoded, or compressed and encoded data for processing. In one embodiment, the CODEC **227** can be configured as a general-purpose data compression engine for use in GPU database acceleration and large volume data analytics.

[0068] The ROP **226** may be included within each processing cluster (e.g., cluster **214**A-**214**N of FIG. **2**A) instead of within the partition unit **220**. In such embodiment, read and write requests for pixel data are transmitted over the memory crossbar **216** instead of pixel fragment data. The processed graphics data may be displayed on a display device, such as one of the one or more display device(s) **110**A-**110**B of FIG. **1**, routed for further processing by the processor(s) **102**, or routed for further processing by one of the processing entities within the parallel processor **200** of FIG. **2**A.

[0069] FIG. **2**C is a block diagram of a processing cluster **214** within a parallel processing unit. For example, the processing cluster **214** represents an instance of one of the processing clusters **214**A-**214**N of FIG. **2**A. The processing cluster **214** can be configured to execute many threads in parallel, where the term “thread” refers to an instance of a particular program executing on a particular set of input data. Optionally, single-instruction, multiple-data (SIMD) instruction issue

techniques may be used to support parallel execution of a large number of threads without providing multiple independent instruction units. Alternatively, single-instruction, multiple-thread (SIMT) techniques may be used to support parallel execution of a large number of generally synchronized threads, using a common instruction unit configured to issue instructions to a set of processing engines within each one of the processing clusters. Unlike a SIMD execution regime, where all processing engines typically execute identical instructions, SIMT execution allows different threads to follow divergent execution paths more readily through a given thread program. Persons skilled in the art will understand that a SIMD processing regime represents a functional subset of a SIMT processing regime.

[0070] Operation of the processing cluster **214** can be controlled via a pipeline manager **232** that distributes processing tasks to SIMT parallel processors. The pipeline manager **232** receives instructions from the scheduler **210** of FIG. **2**A and manages execution of those instructions via a graphics multiprocessor **234** and/or a texture unit **236**. The graphics multiprocessor **234** is an example instance of a SIMT parallel processor. However, various types of SIMT parallel processors of differing architectures may be included within the processing cluster **214**. One or more instances of the graphics multiprocessor **234** can be included within a processing cluster **214**. The graphics multiprocessor **234** may also be referred to as a streaming multiprocessors (SM) and is capable of simultaneous execution of a large number of execution threads.

[0071] The graphics multiprocessor **234** can process data and a data crossbar **240** can be used to distribute the processed data to one of multiple possible destinations, including instances of the graphics multiprocessor **234** within the processing cluster **214**. The pipeline manager **232** can facilitate the distribution of processed data by specifying destinations for processed data to be distributed via the data crossbar **240**. Each graphics multiprocessor **234** within the processing cluster **214** can include an identical set of functional execution logic (e.g., arithmetic logic units, load-store units, etc.). The functional execution logic can be configured in a pipelined manner in which new instructions can be issued before previous instructions are complete. The functional execution logic supports a variety of operations including integer and floating-point arithmetic, comparison operations, Boolean operations, bit-shifting, and computation of various algebraic functions. The same functional-unit hardware could be leveraged to perform different operations and any combination of functional units may be present.

[0072] The instructions transmitted to the processing cluster **214** constitute a thread. A set of threads executing across the set of parallel processing engines is a thread group. A thread group executes the same program on different input data. Each thread within a thread group can be assigned to a different processing engine within a graphics multiprocessor **234**. A thread group may include fewer threads than the number of processing engines within the graphics multiprocessor **234**. When a thread group includes fewer threads than the number of processing engines, one or more of the processing engines may be idle during cycles in which that thread group is being processed. A thread group may also include more threads than the number of processing engines within the graphics multiprocessor **234**. When the thread group includes more threads than the number of processing engines within the graphics multiprocessor **234**, processing can be performed over consecutive clock cycles. Optionally, multiple thread groups can be executed concurrently on the graphics multiprocessor **234**.

[0073] The graphics multiprocessor **234** may include an internal cache memory to perform load and store operations. Optionally, the graphics multiprocessor **234** can forego an internal cache and use a cache memory (e.g., level 1 (L1) cache **248**) within the processing cluster **214**. Each graphics multiprocessor **234** also has access to level 2 (L2) caches within the partition units (e.g., partition units **220**A-**220**N of FIG. **2**A) that are shared among all instances of the processing cluster **214** and may be used to transfer data between threads. The graphics multiprocessor **234** may also access off-chip global memory, which can include one or more of local parallel processor memory and/or system memory. Any memory external to the parallel processing unit **202** may be used as global

memory. Embodiments in which the processing cluster **214** includes multiple instances of the graphics multiprocessor **234** can share common instructions and data, which may be stored in the L1 cache **248**.

[0074] Each processing cluster **214** may include an MMU **245** (memory management unit) that is configured to map virtual addresses into physical addresses. In other embodiments, one or more instances of the MMU **245** may reside within the memory interface **218** of FIG. **2**A. The MMU **245** includes a set of page table entries (PTEs) used to map a virtual address to a physical address of a tile and optionally a cache line index. The MMU **245** may include address translation lookaside buffers (TLB) or caches that may reside within the graphics multiprocessor **234** or the L1 cache **248** of processing cluster **214**. The physical address is processed to distribute surface data access locality to allow efficient request interleaving among partition units. The cache line index may be used to determine whether a request for a cache line is a hit or miss.

[0075] In graphics and computing applications, a processing cluster **214** may be configured such that each graphics multiprocessor **234** is coupled to a texture unit **236** for performing texture mapping operations, e.g., determining texture sample positions, reading texture data, and filtering the texture data. Texture data is read from an internal texture L1 cache (not shown) or in some embodiments from the L1 cache within graphics multiprocessor **234** and is fetched from an L2 cache, local parallel processor memory, or system memory. Each graphics multiprocessor **234** outputs processed tasks to the data crossbar **240** to provide the processed task to another processing cluster **214** for further processing or to store the processed task in an L2 cache, local parallel processor memory, or system memory via the memory crossbar **216**. A preROP **242** (pre-raster operations unit) is configured to receive data from graphics multiprocessor **234**, direct data to ROP units, which may be located with partition units as described herein (e.g., partition units **220**A-**220**N of FIG. **2**A). The preROP **242** unit can perform optimizations for color blending, organize pixel color data, and perform address translations.

[0076] It will be appreciated that the core architecture described herein is illustrative and that variations and modifications are possible. Any number of processing units, e.g., graphics multiprocessor **234**, texture units **236**, preROPs **242**, etc., may be included within a processing cluster **214**. A parallel processing unit as described herein may include any number of instances of the processing cluster **214**. Optionally, each processing cluster **214** can be configured to operate independently of other instances of the processing cluster **214** using, for example, separate and distinct processing units, L1 caches, L2 caches to facilitate data and fault isolation.

[0077] FIG. **2**D shows an example of the graphics multiprocessor **234** in which the graphics multiprocessor **234** couples with the pipeline manager **232** of the processing cluster **214**. The graphics multiprocessor **234** has an execution pipeline including but not limited to an instruction cache **252**, an instruction unit **254**, an address mapping unit **256**, a register file **258**, one or more general purpose graphics processing unit cores (GPGPU cores **262**), and one or more load/store units **266**. The GPGPU cores **262** and load/store units **266** are coupled with cache memory **272** and shared memory **270** via a memory and cache interconnect **268**. The graphics multiprocessor **234** may additionally include ray-tracing cores **263** that include hardware logic to accelerate ray-tracing operations and tensor cores **264** that include hardware logic to accelerate tensor (e.g., matrix) operations. The instruction cache **252** may receive a stream of instructions to execute from the pipeline manager **232**. The instructions are cached in the instruction cache **252** and dispatched for execution by the instruction unit **254**. The instruction unit **254** can dispatch instructions as thread groups (e.g., warps), with each thread of the thread group assigned to a different execution unit within the GPGPU cores **262**. An instruction can access any of a local, shared, or global address space by specifying an address within a unified address space. The address mapping unit **256** can be used to translate addresses in the unified address space into a distinct memory address that can be accessed by the load/store units **266**.

[0078] The register file **258** provides a set of registers for the functional units of the graphics

multiprocessor **234**. The register file **258** provides temporary storage for operands connected to the data paths of the functional units (e.g., GPGPU cores **262**, load/store units **266**) of the graphics multiprocessor **234**. The register file **258** may be divided between each of the functional units such that each functional unit is allocated a dedicated portion of the register file **258**. For example, the register file **258** may be divided between the different warps being executed by the graphics multiprocessor **234**.

[0079] The GPGPU cores **262** can each include floating-point units (FPUs) and/or integer arithmetic logic units (ALUs) that are used to execute instructions of the graphics multiprocessor **234**. In some implementations, the GPGPU cores **262** can include hardware logic that may otherwise reside within the tensor cores **264** and/or ray-tracing cores **263**. The GPGPU cores **262** can be similar in architecture or can differ in architecture. For example and in one embodiment, a first portion of the GPGPU cores **262** include a single precision FPU and an integer ALU while a second portion of the GPGPU cores include a double precision FPU. Optionally, the FPUs can implement the IEEE 754-2008 standard for floating-point arithmetic or enable variable precision floating-point arithmetic. In one embodiment, the single precision FPUs, or a separate set of FPUs, are configurable to perform operations on 16-bit floating-point operands, such as operands in a half precision format or a bfloat16 format (e.g., Brain floating-point), which is a 16-bit floating-point format with one sign bit, eight exponent bits, and eight significand bits, of which seven are explicitly stored. The FPUs within one or more the GPGPU cores **262** may also support one or more 8-bit floating-point formats. Supported 8-bit floating-point formats include the E4M3 format having a 4-bit exponent and a 3-bit mantissa and the E5M2 format having a 5-bit exponent and 2-bit mantissa. The graphics multiprocessor **234** can additionally include one or more fixed function or special function units to perform specific functions such as copy rectangle or pixel blending operations. One or more of the GPGPU cores can also include fixed or special function logic.

[0080] The GPGPU cores **262** may include SIMD logic capable of performing a single instruction on multiple sets of data. Optionally, GPGPU cores **262** can physically execute SIMD4, SIMD8, and SIMD16 instructions and logically execute SIMD1, SIMD2, and SIMD32 instructions. The SIMD instructions for the GPGPU cores can be generated at compile time by a shader compiler or automatically generated when executing programs written and compiled for single program multiple data (SPMD) or SIMT architectures. Multiple threads of a program configured for the SIMT execution model can be executed via a single SIMD instruction. For example and in one embodiment, eight SIMT threads that perform the same or similar operations can be executed in parallel as a SIMD8 instruction. In one embodiment, a warp of 32 SIMT threads can be executed as a single SIMD32 instruction. Warp divergence can be handled via multiple SIMD instructions.

[0081] The memory and cache interconnect **268** is an interconnect network that connects each of the functional units of the graphics multiprocessor **234** to the register file **258** and to the shared memory **270**. For example, the memory and cache interconnect **268** is a crossbar interconnect that allows the load/store unit **266** to implement load and store operations between the shared memory **270** and the register file **258**. The register file **258** can operate at the same frequency as the GPGPU cores **262**, thus data transfer between the GPGPU cores **262** and the register file **258** is very low latency. The shared memory **270** can be used to enable communication between threads that execute on the functional units within the graphics multiprocessor **234**. The shared memory **270** can also be used as a program managed cached. The cache memory **272** can be used as an automatically managed data cache for example, to cache texture data communicated between the functional units and the texture unit **236**. The shared memory **270** and the cache memory **272** can couple with the data crossbar **240** to enable communication with other components of the processing cluster, facilitating the cooperative execution of cluster workgroups via multiple graphics multiprocessors within a processing cluster. Threads executing on the GPGPU cores **262** can programmatically store data within the shared memory in addition to the automatically cached data that is stored within the cache memory **272**. In one embodiment, the shared memory **270** and

the cache memory **272** may be combined into a single configurable memory unit that can be selectively configured as the cache memory **272** or the shared memory **270**.

[0082] FIG. **2**E illustrates a graphics multiprocessor **235** having an alternate configuration relative to the graphics multiprocessor **234** of FIG. **2**D. The disclosure of any features in combination with the graphics multiprocessor **235** described herein also discloses a corresponding combination with the graphics multiprocessor **234** of FIG. **2**D, but is not limited as such. The graphics multiprocessor **235** of FIG. **2**E includes multiple additional instances of execution resources **286**A-**286**D relative to the graphics multiprocessor **234** of FIG. **2**D. For example, the graphics multiprocessor **235** can include multiple instruction units **254**A-**254**D, register files **258**A-**258**D, and texture units **280**A-**280**D. The graphics multiprocessor **235** also includes multiple sets of graphics or compute execution units (e.g., GPGPU cores **262**A-**262**D, ray-tracing cores **263**A-**263**D, and tensor cores **264**A-**264**D) and multiple sets of load/store units **266**A-**266**D. The execution resources **286**A-**286**D work in concert with texture unit(s) **280**A-**280**D for texture operations, while sharing an instruction cache **252**, shared memory **270**, and cache memories **272**A-**272**B. In one embodiment, the execution resources **286**A-**286**D additionally include multi-function units (MUFU) and/or special-function units (SFU) (e.g., MFU **267**A-**267**D) that are used to perform specialized mathematical operations, such as transcendental operations including exponential, logarithmic, and trigonometric functions.

[0083] The various components can communicate via an interconnect fabric **290**. The interconnect fabric **290** may include one or more crossbar switches to enable communication between the various components of the graphics multiprocessor **235**. The GPGPU cores **262**A-**262**D, ray tracing cores **263**A-**263**B, and tensor cores **264**A-**264**D can each communicate with shared memory **270** via the interconnect fabric **290**. The interconnect fabric **290** can arbitrate communication within the graphics multiprocessor **235** to ensure a fair bandwidth allocation between components. In one embodiment, the interconnect fabric **290** may be a separate, high-speed network fabric layer upon which each component of the graphics multiprocessor **235** is stacked. The components of the graphics multiprocessor **235** can also communicate with remote components via the interconnect fabric **290**.

[0084] In one embodiment, the graphics multiprocessor **235** includes a tensor transfer engine **292**, which is a copy engine that is configurable to accelerate the movement of tensor data into and out of the graphics multiprocessor **235**. The tensor transfer engine **292** can accelerate tensor memory operations by asynchronously performing address generation and data movement operations for N-dimensional blocks of tensor data, which offloads operations that would otherwise be performed manually by program code executed by the graphics multiprocessor **235**. The tensor transfer engine **292** is configurable to copy data between, for example, the shared memory **270** and/or cache memory **272**A-**272**B of the graphics multiprocessor **235** and memory external to the graphics multiprocessor **235**, such as graphics processor global memory (e.g., parallel processor memory **222**). In one embodiment, data transfers performed by the tensor transfer engine **292** can be configured to selectively bypass various levels of intermediate data storage between the source and destination memories. For example, a transfer between global memory and shared memory **270** can bypass the register files **258**A-**258**D. In one embodiment, threads can synchronize on asynchronous tensor transfers via a non-blocking barrier synchronization mechanism.

[0085] In various embodiments, the graphics multiprocessor **235** can be tailored for specific use cases via the inclusion or exclusion of certain components, allowing various implementations of the graphics multiprocessor **235** that are tailored to target power, performance, and area characteristic. For example, compute oriented variants of the graphics multiprocessor **235** that will not perform graphics operations can exclude the ray tracing cores **263**A-**263**D. Fully graphics oriented variants can exclude the tensor transfer engine **292**, while graphics oriented variants additionally configured to accelerate neural network inference may include at least a version of the tensor transfer engine **292**.

[0086] Persons skilled in the art will understand that the architecture described in FIG. **1** and FIG. **2**A-**2**E are descriptive and not limiting as to the scope of the present embodiments. Thus, the techniques described herein may be implemented on any properly configured processing unit, including, without limitation, one or more mobile application processors, one or more desktop or server central processing units (CPUs) including multi-core CPUs, one or more parallel processing units, such as the parallel processing unit **202** of FIG. **2**A, as well as one or more graphics processors or special purpose processing units, without departure from the scope of the embodiments described herein.
[0087] The parallel processor or GPGPU as described herein may be communicatively coupled to host/processor cores to accelerate graphics operations, machine-learning operations, pattern analysis operations, and various general-purpose GPU (GPGPU) functions. The GPU may be communicatively coupled to the host processor/cores over a bus or other interconnect (e.g., a high-speed interconnect such as PCIe, NVLink, or other known protocols, standardized protocols, or proprietary protocols). In other embodiments, the GPU may be integrated on the same package or chip as the cores and communicatively coupled to the cores over an internal processor bus/interconnect (i.e., internal to the package or chip). Regardless of the manner in which the GPU is connected, the processor cores may allocate work to the GPU in the form of sequences of commands/instructions contained in a work descriptor. The GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.
[0088] FIG. **3** illustrates a graphics processing unit (GPU **380**) which includes dedicated sets of graphics processing resources arranged into multi-core groups **365**A-**365**N. The multi-core groups **365**A-**365**N correspond with the graphics multiprocessor **234** of FIG. **2**D or the graphics multiprocessor **235** of FIG. **2**E. While the details of a single example of the multi-core groups **365**A-**365**N (e.g., multi-core group **365**A) are provided, it will be appreciated that the other multi-core groups **365**B-**365**N may be equipped with the same or similar sets of graphics processing resources. Details described with respect to the multi-core groups **365**A-**365**N may also apply to graphics multiprocessor **234** or graphics multiprocessor **235**, as described herein.
[0089] As illustrated, a multi-core group **365**A may include graphics cores **370**, tensor cores **371**, and ray tracing cores **372**. The graphics cores **370** are analogous to the GPGPU cores **262**A-**262**D and are configurable to execute instructions to perform graphics and/or general-purpose compute operations. A scheduler/dispatcher **368** schedules and dispatches the graphics threads for execution on the various cores within the multi-core group **365**A. Register files **369** are included that store operand values used by the cores when performing graphics or general-purpose compute operations for executed threads. These register files **369** may include, for example, registers configurable to store integer values or floating-point values, including vector registers for storing packed integer and/or floating-point data elements and tile registers for storing tensor/matrix values. The tile registers may be implemented as multi-dimensional registers that include combined sets of vector registers.
[0090] One or more combined level 1 (L1) caches and shared memory units **373** store graphics data such as texture data, vertex data, pixel data, ray data, bounding volume data, etc., locally within each multi-core group **365**A. One or more texture units **374** can also be used to perform texturing operations, such as texture mapping and sampling. A Level 2 (L2) cache **375** shared by all or a subset of the multi-core groups **365**A-**365**N stores graphics data and/or instructions for multiple concurrent graphics threads. As illustrated, the L2 cache **375** may be shared across a plurality of multi-core groups **365**A-**365**N. One or more memory controllers **367** couple the GPU **380** to a memory **366** which may be a system memory (e.g., DRAM) and/or a dedicated graphics memory (e.g., GDDR6 memory).
[0091] Input/output circuitry (I/O circuitry **363**) couples the GPU **380** to one or more I/O devices **362** such as digital signal processors (DSPs), network controllers, or user input devices. An on-chip interconnect may be used to couple the I/O devices **362** to the GPU **380** and memory **366**. At least

one I/O memory management units (IOMMU **364**) of the I/O circuitry **363** couples the I/O devices **362** directly to the memory **366**. Optionally, the IOMMU **364** manages multiple sets of page tables to map virtual addresses to physical addresses in memory **366**. The I/O devices **362**, CPU(s) **361**, and GPU **380** may then share the same virtual address space.
[0092] In one implementation of the IOMMU **364**, the IOMMU **364** supports virtualization. In this case, it may manage a first set of page tables to map guest/graphics virtual addresses to guest/graphics physical addresses and a second set of page tables to map the guest/graphics physical addresses to system/host physical addresses (e.g., within memory **366**). The base addresses of each of the first and second sets of page tables may be stored in control registers and swapped out on a context switch (e.g., so that the new context is provided with access to the relevant set of page tables). While not illustrated in FIG. **3**, each of the cores within the multi-core groups **365**A-**365**N may include translation lookaside buffers (TLBs) to cache guest virtual to guest physical translations, guest physical to host physical translations, and guest virtual to host physical translations.
[0093] The CPU(s) **361**, GPU **380**, and I/O devices **362** may be integrated on a single semiconductor chip and/or chip package. The memory **366** may be integrated on the same chip or may be coupled to the one or more memory controllers **367** via an off-chip interface. In one implementation, the memory **366** comprises GDDR6 memory which shares the same virtual address space as other physical system-level memories, although the underlying principles described herein are not limited to this specific implementation.
[0094] The tensor cores **371** may include a plurality of execution units specifically designed to perform matrix operations, which are the compute operations used to perform deep learning operations. For example, simultaneous matrix multiplication operations may be used for neural network training and inferencing. The tensor cores **371** may perform matrix processing using a variety of operand precisions including single precision floating-point (e.g., 32 bits), half-precision floating-point (e.g., 16 bits), integer words (16 bits), bytes (8 bits), and half-bytes (4 bits). For example, a neural network implementation extracts features of each rendered scene, potentially combining details from multiple frames, to construct a high-quality final image.
[0095] In deep learning implementations, parallel matrix multiplication work may be scheduled for execution on the tensor cores **371**. The training of neural networks, in particular, utilizes a significant number of matrix dot product operations. In order to process an inner-product formulation of an N×N×N matrix multiply, the tensor cores **371** may include at least N dot-product processing elements. Before the matrix multiply begins, one entire matrix is loaded into tile registers and at least one column of a second matrix is loaded each cycle for N cycles. Each cycle, there are N dot products that are processed.
[0096] Matrix elements may be stored at different precisions depending on the particular implementation, including 16-bit words, 8-bit bytes (e.g., INT8) and 4-bit half-bytes (e.g., INT4). Different precision modes may be specified for the tensor cores **371** to ensure that an efficient precision is used for different workloads (e.g., such as inferencing workloads which can tolerate quantization to bytes and half-bytes). Supported formats additionally include 64-bit floating-point (FP64) and non-IEEE floating-point formats such as the bfloat16 format. One embodiment includes support for a reduced precision tensor-float (TF32) mode, which performs computations using the range of FP32 (8-bits) and the precision of FP16 (10-bits). Reduced precision TF32 operations can be performed on FP32 inputs and produce FP32 outputs at higher performance relative to FP32 and increased precision relative to FP16. In one embodiment, one or more 8-bit floating-point (FP8), 6-bit floating-point (FP6), and 4-bit floating-point (FP4) formats are supported, including floating-point formats represented as microscaling (MX) formats.
[0097] In one embodiment the tensor cores **371** support a sparse mode of operation for matrices in which the vast majority of values are zero. The tensor cores **371** include support for sparse input matrices that are encoded in a sparse matrix representation (e.g., coordinate list encoding (COO),

compressed sparse row (CSR), compress sparse column (CSC), etc.). The tensor cores **371** also include support for compressed sparse matrix representations in the event that the sparse matrix representation may be further compressed. Compressed, encoded, and/or compressed and encoded matrix data, along with associated compression and/or encoding metadata, can be read by the tensor cores **371** and the non-zero values can be extracted. For example, for a given input matrix A, a non-zero value can be loaded from the compressed and/or encoded representation of at least a portion of matrix A. Based on the location in matrix A for the non-zero value, which may be determined from index or coordinate metadata associated with the non-zero value, a corresponding value in input matrix B may be loaded. Depending on the operation to be performed (e.g., multiply), the load of the value from input matrix B may be bypassed if the corresponding value is a zero value. In one embodiment, the pairings of values for certain operations, such as multiply operations, may be pre-scanned by scheduler logic and only operations between non-zero inputs are scheduled. Depending on the dimensions of matrix A and matrix B and the operation to be performed, output matrix C may be dense or sparse. Where output matrix C is sparse and depending on the configuration of the tensor cores **371**, output matrix C may be output in a compressed format, a sparse encoding, or a compressed sparse encoding.

[0098] The ray tracing cores **372** may accelerate ray tracing operations for both real-time ray tracing and non-real-time ray tracing implementations. In particular, the ray tracing cores **372** may include ray traversal/intersection circuitry for performing ray traversal using bounding volume hierarchies (BVHs) and identifying intersections between rays and primitives enclosed within the BVH volumes. The ray tracing cores **372** may also include circuitry for performing depth testing and culling (e.g., using a Z buffer or similar arrangement). In one implementation, the ray tracing cores **372** perform traversal and intersection operations in concert with the image denoising techniques described herein, at least a portion of which may be executed on the tensor cores **371**. For example, the tensor cores **371** may implement a deep learning neural network to perform denoising of frames generated by the ray tracing cores **372**. However, the CPU(s) **361**, graphics cores **370**, and/or ray tracing cores **372** may also implement all or a portion of the denoising and/or deep learning algorithms.

[0099] In addition, as described above, a distributed approach to denoising may be employed in which the GPU **380** is in a computing device coupled to other computing devices over a network or high-speed interconnect. In this distributed approach, the interconnected computing devices may share neural network learning/training data to improve the speed with which the overall system learns to perform denoising for different types of image frames and/or different graphics applications.

[0100] The ray tracing cores **372** may process all BVH traversal and/or ray-primitive intersections, saving the graphics cores **370** from being overloaded with thousands of instructions per ray. For example, each ray tracing core **372** includes a first set of specialized circuitry for performing bounding box tests (e.g., for traversal operations) and/or a second set of specialized circuitry for performing the ray-triangle intersection tests (e.g., intersecting rays which have been traversed). Thus, for example, the multi-core group **365**A can simply launch a ray probe, and the ray tracing cores **372** independently perform ray traversal and intersection and return hit data (e.g., a hit, no hit, multiple hits, etc.) to the thread context. The graphics cores **370** and tensor cores **371** are then freed to perform other graphics or compute work while the ray tracing cores **372** perform the traversal and intersection operations. Optionally, each ray tracing core **372** may include a traversal unit to perform BVH testing operations and/or an intersection unit which performs ray-primitive intersection tests. The intersection unit generates a “hit”, “no hit”, or “multiple hit” response, which it provides to the appropriate thread. During the traversal and intersection operations, the execution resources of the other cores (e.g., graphics cores **370** and tensor cores **371**) are freed to perform other forms of graphics work. In one optional embodiment described below, a hybrid rasterization/ray tracing approach is used in which rendering operations are distributed between the

graphics cores **370** and ray tracing cores **372**.

[0101] The ray tracing cores **372** may include hardware support for a ray tracing instruction set such as Microsoft's DirectX Ray Tracing (DXR) which includes a DispatchRays command, as well as ray-generation, closest-hit, any-hit, and miss shaders, which enable the assignment of sets of shaders and textures for each object. Another ray tracing platform which may be supported by the ray tracing cores **372**, graphics cores **370** and tensor cores **371** is Vulkan API (e.g., Vulkan version 1.1.85 and later). Note, however, that the underlying principles described herein are not limited to any particular ray tracing ISA. In general, the ray tracing cores **372**, tensor cores **371**, and graphics cores **370** may support a ray tracing instruction set that includes instructions/functions for one or more of ray generation, closest hit, any hit, ray-primitive intersection, per-primitive and hierarchical bounding box construction, miss, visit, and exceptions. More specifically, an embodiment includes ray tracing instructions to perform one or more of the following functions:

[0102] Ray Generation—Ray generation instructions may be executed for each pixel, sample, or other user-defined work assignment.

[0103] Closest Hit—A closest hit instruction may be executed to locate the closest intersection point of a ray with primitives within a scene.

[0104] Any Hit—An any hit instruction identifies multiple intersections between a ray and primitives within a scene, potentially to identify a new closest intersection point.

[0105] Intersection—An intersection instruction performs a ray-primitive intersection test and outputs a result.

[0106] Per-primitive Bounding box Construction—This instruction builds a bounding box around a given primitive or group of primitives (e.g., when building a new BVH or other acceleration data structure).

[0107] Miss—Indicates that a ray misses all geometry within a scene, or specified region of a scene.

[0108] Visit—Indicates the child volumes a ray will traverse.

[0109] Exceptions—Includes various types of exception handlers (e.g., invoked for various error conditions).

[0110] In one embodiment the ray tracing cores **372** may be adapted to accelerate general-purpose compute operations using computational techniques that are analogous to ray intersection tests. A compute framework can be provided that enables shader programs to be compiled into low level instructions and/or primitives that perform general-purpose compute operations via the ray tracing cores. Example computational problems that can benefit from compute operations performed on the ray tracing cores **372** include computations involving beam, wave, ray, or particle propagation within a coordinate space. Interactions associated with that propagation can be computed relative to a geometry or mesh within the coordinate space. For example, computations associated with electromagnetic signal propagation through an environment can be accelerated via the use of instructions or primitives that are executed via the ray tracing cores. Diffraction and reflection of the signals by objects in the environment can be computed as direct ray-tracing analogies.

[0111] Ray tracing cores **372** can also be used to perform computations that are not directly analogous to ray tracing. For example, mesh projection, mesh refinement, and volume sampling computations can be accelerated using the ray tracing cores **372**. Generic coordinate space calculations, such as nearest neighbor calculations can also be performed. For example, the set of points near a given point can be discovered by defining a bounding box in the coordinate space around the point. BVH and ray probe logic within the ray tracing cores **372** can then be used to determine the set of point intersections within the bounding box. The intersections constitute the origin point and the nearest neighbors to that origin point. Computations that are performed using the ray tracing cores **372** can be performed in parallel with computations performed on the graphics cores **370** and tensor cores **371**. A shader compiler can be configured to compile a compute shader or other general-purpose graphics processing program into low level primitives that can be

parallelized across the graphics cores **370**, tensor cores **371**, and ray tracing cores **372**.

Techniques for GPU to Host Processor Interconnection

[0112] FIG. **4**A illustrates an example architecture in which a plurality of GPUs **410-413**, e.g., such as the parallel processor **200** shown in FIG. **2**A, are communicatively coupled to a plurality of multi-core processors **405-406** over high-speed links **440**A-**440**D (e.g., buses, point-to-point interconnects, etc.). The high-speed links **440**A-**440**D may support a communication throughput of 4 GB/s, 30 GB/s, 80 GB/s or higher, depending on the implementation. Various interconnect protocols may be used including, but not limited to, PCIe 4.0, PCIe 5.0, PCIe 6.0, and various NVLink and NVlink-C2C (Chip-to-Chip) interconnect protocols (e.g., NVLink v5). However, the underlying principles described herein are not limited to any particular communication protocol or throughput.

[0113] Two or more of the GPUs **410-413** may be interconnected over high-speed links **442**A-**442**B, which may be implemented using the same or different protocols/links than those used for high-speed links **440**A-**440**D. Similarly, two or more of the multi-core processors **405-406** may be connected over high-speed link **443** which may be symmetric multi-processor (SMP) buses operating at 20 GB/s, 30 GB/s, 120 GB/s or lower or higher speeds. Alternatively, all communication between the various system components shown in FIG. **4**A may be accomplished using the same protocols/links (e.g., over a common interconnection fabric). As mentioned, however, the underlying principles described herein are not limited to any particular type of interconnect technology.

[0114] Each of multi-core processor **405** and multi-core processor **406** may be communicatively coupled to a processor memory **401-402**, via memory interconnects **430**A-**430**B, respectively, and each GPU **410-413** is communicatively coupled to GPU memory **420-423** over GPU memory interconnects **450**A-**450**D, respectively. The memory interconnects **430**A-**430**B and **450**A-**450**D may utilize the same or different memory access technologies. By way of example, and not limitation, the processor memories **401-402** and GPU memories **420-423** may be volatile memories such as dynamic random-access memories (DRAMs) (including stacked DRAMs), Graphics DDR SDRAM (GDDR) (e.g., GDDR5, GDDR6), or High Bandwidth Memory (HBM) and/or may be non-volatile memories such as 3D XPoint/Optane or Nano-Ram. For example, some portion of the memories may be volatile memory and another portion may be non-volatile memory (e.g., using a two-level memory (2LM) hierarchy). A memory subsystem as described herein may be compatible with a number of memory technologies, such as Double Data Rate versions released by JEDEC (Joint Electronic Device Engineering Council).

[0115] As described below, although the various processors **405-406** and GPUs **410-413** may be physically coupled to particular processor memory **401-402** and GPU memory **420-423**, respectively, a unified memory architecture may be implemented in which the same virtual system address space (also referred to as the “effective address” space) is distributed among all of the various physical memories. For example, processor memories **401-402** may each comprise 64 GB of the system memory address space and GPU memories **420-423** may each comprise 32 GB of the system memory address space (resulting in a total of 256 GB addressable memory in this example).

[0116] FIG. **4**B illustrates additional optional details for an interconnection between a processor **407** and a graphics accelerator **446**. The graphics accelerator **446** may include one or more GPU chips integrated on a line card which is coupled to the processor **407** via the high-speed link **440**. Alternatively, the graphics accelerator **446** may be integrated on the same package or chip as the processor **407**. The processor **407** includes a plurality of cores **460**A-**460**D, each with a translation lookaside buffer **461**A-**461**D and one or more caches **462**A-**462**D. The cores may include various other components for executing instructions and processing data which are not illustrated to avoid obscuring the underlying principles of the components described herein (e.g., instruction fetch units, branch prediction units, decoders, execution units, reorder buffers, etc.). The caches **462**A-**462**D may comprise level 1 (L1) and level 2 (L2) caches. In addition, one or more shared cache(s)

456 may be included in the caching hierarchy and shared by sets of the cores **460**A-**460**D. For example, one embodiment of the processor **407** includes 24 cores, each with its own L1 cache, twelve shared L2 caches, and twelve shared L3 caches. In this embodiment, one of the L2 and L3 caches are shared by two adjacent cores. The processor **407** connects with system memory **441**, which may include processor memories **401-402**.

[0117] Coherency is maintained for data and instructions stored in the various caches **462**A-**462**D, shared cache(s) **456** and system memory **441** via inter-core communication over a coherence bus **464**. For example, each cache may have cache coherency logic/circuitry associated therewith to communicate to over the coherence bus **464** in response to detected reads or writes to particular cache lines. In one implementation, a cache snooping protocol is implemented over the coherence bus **464** to snoop cache accesses. Cache snooping/coherency techniques are well understood by those of skill in the art and will not be described in detail here to avoid obscuring the underlying principles described herein. A proxy circuit **425** may be provided that communicatively couples the graphics accelerator **446** to the coherence bus **464**, allowing the graphics accelerator **446** to participate in the cache coherence protocol as a peer of the cores. In particular, an interface **435** provides connectivity to the proxy circuit **425** over high-speed link **440** (e.g., a PCIe bus, NVLink, etc.) and an interface **437** connects the graphics accelerator **446** to the high-speed link **440**.

[0118] In one implementation, the interface **437** couples with an accelerator integration circuit **436** that provides cache management, memory access, context management, and interrupt management services on behalf of graphics processing engines **431**, **432**, . . . , N of the graphics accelerator **446**. The graphics processing engines **431**, **432**, . . . , N may each comprise a separate graphics processing unit (GPU). Alternatively, the graphics processing engines **431**, **432**, . . . , N may comprise different types of graphics processing engines within a GPU such as graphics execution units, media processing engines (e.g., video encoders/decoders), samplers, and block image transfer (BLIT) engines. In other words, the graphics accelerator may include graphics processing engines **431-432**, . . . , N of a single GPU or the graphics processing engines **431-432**, . . . , N may be associated with multiple GPUs integrated on a common package, line card, or chip. The graphics processing engines **431-432**, . . . , N may be configured with any graphics processor or compute accelerator architecture described herein. Work to be performed by the graphics processing engines **431**, **432** can be specified via work descriptors that provide an indication of the work to be done by the graphics accelerator **446**.

[0119] The accelerator integration circuit **436** may include a memory management unit (MMU **439**) for performing various memory management functions such as virtual-to-physical memory translations (also referred to as effective-to-real memory translations) and memory access protocols for accessing the system memory **441**. The MMU **439** may also include a translation lookaside buffer (TLB) (not shown) for caching the virtual/effective to physical/real address translations. In one implementation, a cache **438** stores commands and data for efficient access by the graphics processing engines **431**, **432**, . . . , N. The data stored in cache **438** and graphics memories **433-434**, . . . , M may be kept coherent with the core caches **462**A-**462**D, shared cache(s) **456** and system memory **441**. As mentioned, this may be accomplished via a proxy circuit **425** which takes part in the cache coherency mechanism on behalf of the cache **438** and graphics memories **433-434**, . . . , M (e.g., sending updates to the cache **438** related to modifications/accesses of cache lines on processor caches **462**A-**462**D, shared cache(s) **456** and receiving updates from the cache **438**).

[0120] Registers **445** store context data for threads executed by the graphics processing engines **431-432**, . . . , N and a context management circuit **448** manages the thread contexts. For example, the context management circuit **448** may perform save and restore operations to save and restore contexts of the various threads during contexts switches (e.g., where a first thread is saved and a second thread is restored so that the second thread can be execute by a graphics processing engine). For example, on a context switch, the context management circuit **448** may store current register values to a designated region in memory (e.g., identified by a context pointer). It may then restore

the register values when returning to the context. An interrupt management circuit **447**, for example, may receive and processes interrupts received from system devices.

[0121] In one implementation, virtual/effective addresses from a graphics processing engine are translated to real/physical addresses in system memory **441** by the MMU **439**. Optionally, the accelerator integration circuit **436** supports multiple (e.g., 4, 8, 16) graphics accelerators **446** and/or other accelerator devices. The graphics accelerator **446** may be dedicated to a single application executed on the processor **407** or may be shared between multiple applications. Optionally, a virtualized graphics execution environment is provided in which the resources of the graphics processing engines **431**-**432**, . . . , N are shared with multiple applications, virtual machines (VMs), or containers. The resources may be subdivided into "slices" which are allocated to different VMs and/or applications based on the processing requirements and priorities associated with the VMs and/or applications, or based on a pre-determined partitioning profile for a graphics accelerator **446**. VMs and containers can be used interchangeably herein.

[0122] A virtual machine (VM) can be software that runs an operating system and one or more applications. A VM can be defined by specification, configuration files, virtual disk file, non-volatile random-access memory (NVRAM) setting file, and the log file and is backed by the physical resources of a host computing platform. A VM can include an operating system (OS) or application environment that is installed on software, which imitates dedicated hardware. The end user has the same experience on a virtual machine as they would have on dedicated hardware. Specialized software, called a hypervisor, emulates the PC client or server's CPU, memory, hard disk, network, and other hardware resources completely, enabling virtual machines to share the resources. The hypervisor can emulate multiple virtual hardware platforms that are isolated from each other, allowing virtual machines to run Linux®, Windows® Server, VMware ESXi, and other operating systems on the same underlying physical host.

[0123] A container can be a software package of applications, configurations, and dependencies so the applications run reliably on one computing environment to another. Containers can share an operating system installed on the server platform and run as isolated processes. A container can be a software package that contains components that the software utilizes to run, such as system tools, libraries, and settings. Containers are not installed like traditional software programs, which allows them to be isolated from the other software and the operating system itself. The isolated nature of containers provides several benefits. First, the software in a container will run the same in different environments. For example, a container that includes PHP and MySQL can run identically on both a Linux® computer and a Windows® machine. Second, containers provide added security since the software will not affect the host operating system. While an installed application may alter system settings and modify resources, such as the Windows registry, a container can only modify settings within the container.

[0124] Thus, the accelerator integration circuit **436** acts as a bridge to the system for the graphics accelerator **446** and provides address translation and system memory cache services. In one embodiment, to facilitate the bridging functionality, the accelerator integration circuit **436** may also include shared I/O **497** (e.g., PCIe, USB, or others) and hardware to enable system control of voltage, clocking, performance, thermals, and security. The shared I/O **497** may utilize separate physical connections or may traverse the high-speed link **440**. In addition, the accelerator integration circuit **436** may provide virtualization facilities for the host processor to manage virtualization of the graphics processing engines, interrupts, and memory management.

[0125] Because hardware resources of the graphics processing engines **431**-**432**, . . . , N are mapped explicitly to the real address space seen by the processor **407**, any host processor can address these resources directly using an effective address value. One optional function of the accelerator integration circuit **436** is the physical separation of the graphics processing engines **431**-**432**, . . . , N so that they appear to the system as independent units. In one embodiment, the accelerator integration circuit **436** includes security circuitry **444** that enables configurable

cryptographic isolation of data associated with each slice of resources. Different slices can be associated with different security domains, such that data associated with the various security domains are encrypted using different cryptographic keys. In one embodiment, the security domains of the graphics accelerator **446** can be integrated into trusted execution environments supported by the processor **407**. In one embodiment, secure I/O capabilities can be enabled that allow each security domain to be presented as a separate trusted I/O device, which enables support for trusted DMA and MMIO operations.

[0126] One or more graphics memories **433-434**, . . . , M may be coupled to each of the graphics processing engines **431-432**, . . . , N, respectively. The graphics memories **433-434**, ... M store instructions and data being processed by each of the graphics processing engines **431-432**, . . . , N. The graphics memories **433-434**, . . . , M may be volatile memories such as DRAMs (including stacked DRAMs), GDDR memory (e.g., GDDR5, GDDR6), or HBM, and/or may be non-volatile memories such as 3D XPoint/Optane, Samsung Z-NAND, or Nano-Ram.

[0127] To reduce data traffic over the high-speed link **440**, biasing techniques may be used to ensure that the data stored in graphics memories **433-434**, . . . , M is data which will be used frequently by the graphics processing engines **431-432**, . . . , N and preferably not used by the cores **460**A-**460**D (at least not frequently). Similarly, the biasing mechanism attempts to keep data utilized by the cores (and preferably not the graphics processing engines **431-432**, . . . , N) within the caches **462**A-**462**D, shared cache(s) **456** and the system memory **441**.

[0128] In an alternative variant, the accelerator integration circuit **436** is integrated within the processor **407** and the graphics processing engines **431-432**, . . . , N communicate over the high-speed link **440** to the accelerator integration circuit **436** via interface **437** and interface **435** (which, again, may be utilize any form of bus, fabric, or interface protocol). In this variant, the accelerator integration circuit **436** performs the same operations as those described above.

[0129] The embodiments described may support different programming models including a dedicated-process programming model (no graphics accelerator virtualization) and shared programming models (with virtualization). The latter may include programming models which are controlled by the accelerator integration circuit **436** and programming models which are controlled by the graphics accelerator **446**. In the embodiments of the dedicated process model, graphics processing engines **431**, **432**, . . . N may be dedicated to a single application or process under a single operating system. The single application can funnel other application requests to the graphics processing engines **431**, **432**, ... N, providing virtualization within a VM/partition. In the dedicated-process programming models, the graphics processing engines **431**,**432**, . . . , N, may be shared by multiple VM/application partitions. The shared models utilize a system hypervisor to virtualize the graphics processing engines **431-432**, . . . , N to allow access by each operating system. For single-partition systems without a hypervisor, the graphics processing engines **431-432**, . . . , N are owned by the operating system. In both cases, the operating system can virtualize the graphics processing engines **431-432**, . . . , N to provide access to each process or application. For the shared programming model, the graphics accelerator **446** or individual graphics processing engines **431-432**, . . . , N selects a process element using a process handle. The process elements may be stored in system memory **441** and be addressable using the effective address to real address translation techniques described herein. The process handle may be an implementation-specific value provided to the host process when registering its context with the graphics processing engines **431-432**, . . . , N, which can be performed by the host process by calling system software to add the process element to the process element linked list. The lower 16-bits of the process handle may be the offset of the process element within the process element linked list.

[0130] FIG. **4**C illustrates an accelerator integration slice **490**. As used herein, a “slice” comprises a specified portion of the processing resources of the accelerator integration circuit **436**. The application's address space **482** within system memory **441** stores process elements **483**. The process elements **483** may be stored in response to GPU invocations **481** from an application **480**

executed on the processor **407**. A process element **483** contains the process state for the application **480**. A work descriptor (WD **484**) contained in the process element **483** can be a single job requested by an application or may contain a pointer to a queue of jobs. In the latter case, the WD **484** is a pointer to the job request queue in the application's address space **482**.

[0131] The graphics accelerator **446** and/or the individual graphics processing engines **431-432**, . . . , N can be shared by all or a subset of the processes in the system. For example, the technologies described herein may include an infrastructure for setting up the process state and sending a WD **484** to a graphics accelerator **446** to start a job in a virtualized environment.

[0132] In one implementation, the dedicated-process programming model is implementation-specific. In this model, a single process owns the graphics accelerator **446** or an individual graphics processing engine (e.g., graphics processing engine **431**). Because the graphics accelerator **446** is owned by a single process, the hypervisor initializes the accelerator integration circuit **436** for the owning partition and the operating system initializes the accelerator integration circuit **436** for the owning process at the time when the graphics accelerator **446** is assigned.

[0133] In operation, the fetch unit **491** in the accelerator integration slice **490** fetches a WD **484** to be processed. The WD **484** includes the indication of work to be done by one or more graphics processing engines of the graphics accelerator **446**. Data from the WD **484** may be stored in registers **445** and used by the MMU **439**, interrupt management circuit **447** and/or context management circuit **448** as illustrated. For example, the MMU **439** may include segment/page walk circuitry for accessing segment/page tables **486** within the OS virtual address space **485**. The interrupt management circuit **447** may process interrupt events **492** received from the graphics accelerator **446**. When performing graphics operations, an effective address **493** generated by a graphics processing engine **431-432**, . . . , Nis translated to a real address by the MMU **439**.

[0134] The registers **445** may be duplicated for each graphics processing engine **431-432**, . . . , N and/or graphics accelerator **446** and may be initialized by the hypervisor or operating system. Each of these duplicated registers may be included in an accelerator integration slice **490**. In one embodiment, each graphics processing engine **431-432**, . . . , N may be presented to the hypervisor **496** as a distinct graphics processor device. Quality of Service (QOS) settings can be configured for clients of a specific graphics processing engine **431-432**, . . . , N. Cryptographic and physical data isolation between the clients of each engine can be enabled via isolated memory access paths and automatic data encryption for each client. Example registers that may be initialized by the hypervisor are shown in Table 1.

TABLE-US-00001 TABLE 1 Hypervisor Initialized Registers 1 Slice Control Register 2 Real Address (RA) Scheduled Processes Area Pointer 3 Authority Mask Override Register 4 Interrupt Vector Table Entry Offset 5 Interrupt Vector Table Entry Limit 6 State Register 7 Logical Partition ID 8 Real address (RA) Hypervisor Accelerator Utilization Record Pointer 9 Storage Description Register

[0135] Example registers that may be initialized by the operating system are shown in Table 2.

TABLE-US-00002 TABLE 2 Operating System Initialized Registers 1 Process and Thread Identification 2 Effective Address (EA) Context Save/Restore Pointer 3 Virtual Address (VA) Accelerator Utilization Record Pointer 4 Virtual Address (VA) Storage Segment Table Pointer 5 Authority Mask 6 Work descriptor

[0136] Each WD **484** may be specific to a particular graphics accelerator **446** and/or graphics processing engine **431-432**, . . . , N. It contains all the information a graphics processing engine utilizes to do its work, or it can be a pointer to a memory location where the application has set up a command queue of work to be completed.

[0137] FIG. **4**D illustrates additional optional details of a shared model. It includes a hypervisor real address space **498** in which a process element list **499** is stored. The hypervisor real address space **498** is accessible via a hypervisor **496** which virtualizes the graphics processing engines for the operating system **495**.

[0138] The shared programming models allow for all or a subset of processes from all or a subset of partitions in the system to use a graphics accelerator **446**. There are two programming models where the graphics accelerator **446** is shared by multiple processes and partitions: time-sliced shared and graphics directed shared.
[0139] In this model, the hypervisor **496** owns the graphics accelerator **446** and makes its function available to all operating systems **495**. For a graphics accelerator **446** to support virtualization by the hypervisor **496**, the graphics accelerator **446** may adhere to the following requirements: 1) An application's job request should be autonomous (that is, the state does not have to be maintained between jobs), or the graphics accelerator **446** should provide a context save and restore mechanism. 2) An application's job request is guaranteed by the graphics accelerator **446** to complete in a specified amount of time, including any translation faults, or the graphics accelerator **446** provides the ability to preempt the processing of the job. 3) The graphics accelerator **446** should be guaranteed fairness between processes when operating in the directed shared programming model.
[0140] For the directed shared model, the application **480** may have to make an operating system **495** system call with a graphics accelerator **446** type, a work descriptor (WD), an authority mask register (AMR) value, and a context save/restore area pointer (CSRP). The graphics accelerator **446** type describes the targeted acceleration function for the system call. The graphics accelerator **446** type may be a system-specific value. The WD is formatted specifically for the graphics accelerator **446** and can be in the form of a graphics accelerator **446** command, an effective address pointer to a user-defined structure, an effective address pointer to a queue of commands, or any other data structure to describe the work to be done by the graphics accelerator **446**. In one embodiment, the AMR value is the AMR state to use for the current process. The value passed to the operating system is similar to an application setting the AMR. If the accelerator integration circuit **436** and graphics accelerator **446** implementations do not support a User Authority Mask Override Register (UAMOR), the operating system may apply the current UAMOR value to the AMR value before passing the AMR in the hypervisor call. The hypervisor **496** may optionally apply the current Authority Mask Override Register (AMOR) value before placing the AMR into the process element **483**. The CSRP may be one of the registers **445** containing the effective address of an area in the application's address space **482** for the graphics accelerator **446** to save and restore the context state. This pointer is optional if no state is to be saved between jobs or when a job is preempted. The context save/restore area may be pinned system memory.
[0141] Upon receiving the system call, the operating system **495** may verify that the application **480** has registered and been given the authority to use the graphics accelerator **446**. The operating system **495** then calls the hypervisor **496** with the information shown in Table 3.
TABLE-US-00003 TABLE 3 OS to Hypervisor Call Parameters 1 A work descriptor (WD) 2 An Authority Mask Register (AMR) value (potentially masked). 3 An effective address (EA) Context Save/Restore Area Pointer (CSRP) 4 A process ID (PID) and optional thread ID (TID) 5 A virtual address (VA) accelerator utilization record pointer (AURP) 6 The virtual address of the storage segment table pointer (SSTP) 7 A logical interrupt service number (LISN)
[0142] Upon receiving the hypervisor call, the hypervisor **496** verifies that the operating system **495** has registered and been given the authority to use the graphics accelerator **446**. The hypervisor **496** then puts the process element **483** into the process element linked list for the corresponding graphics accelerator **446** type. The process element may include the information shown in Table 4.
TABLE-US-00004 TABLE 4 Process Element Information 1 A work descriptor (WD) 2 An Authority Mask Register (AMR) value (potentially masked). 3 An effective address (EA) Context Save/Restore Area Pointer (CSRP) 4 A process ID (PID) and optional thread ID (TID) 5 A virtual address (VA) accelerator utilization record pointer (AURP) 6 The virtual address of the storage segment table pointer (SSTP) 7 A logical interrupt service number (LISN) 8 Interrupt vector table, derived from the hypervisor call parameters. 9 A state register (SR) value 10 A logical partition ID

(LPID) 11 A real address (RA) hypervisor accelerator utilization record pointer 12 The Storage Descriptor Register (SDR)

[0143] The hypervisor may initialize the registers **445** of accelerator integration slice **490**.

[0144] As illustrated in FIG. **4**E, in one optional implementation a unified memory addressable via a common virtual memory address space used to access the physical processor memories **401**-**402** and GPU memories **420**-**423** is employed. In this implementation, operations executed on the GPUs **410**-**413** utilize the same virtual/effective memory address space to access the processors memories **401**-**402** and vice versa, thereby simplifying programmability. A first portion of the virtual/effective address space may be allocated to the processor memory **401**, a second portion to the second processor memory **402**, a third portion to the GPU memory **420**, and so on. The entire virtual/effective memory space (sometimes referred to as the effective address space) may thereby be distributed across each of the processor memories **401**-**402** and GPU memories **420**-**423**, allowing any processor or GPU to access any physical memory with a virtual address mapped to that memory.

[0145] Bias/coherence management circuitry **494**A-**494**E within one or more of the MMUs **439**A-**439**E may be provided that ensures cache coherence between the caches of the host processors (e.g., multi-core processor **405**) and the GPUs **410**-**413** and implements biasing techniques indicating the physical memories in which certain types of data should be stored. While multiple instances of bias/coherence management circuitry **494**A-**494**E are illustrated in FIG. **4**E, the bias/coherence circuitry may be implemented within the MMU of one or more host processors and/or within the accelerator integration circuit **436**.

[0146] The GPU-attached memory **420**-**423** may be mapped as part of system memory and accessed using shared virtual memory (SVM) technology, but without suffering the typical performance drawbacks associated with full system cache coherence. The ability to GPU-attached memory **420**-**423** to be accessed as system memory without onerous cache coherence overhead provides a beneficial operating environment for GPU offload. This arrangement allows the host processor software to setup operands and access computation results, without the overhead of traditional I/O DMA data copies. Such traditional copies involve driver calls, interrupts and memory mapped I/O (MMIO) accesses that are all inefficient relative to simple memory accesses. At the same time, the ability to access GPU attached memory **420**-**423** without cache coherence overheads can be relevant to the execution time of an offloaded computation. In cases with substantial streaming write memory traffic, for example, cache coherence overhead can significantly reduce the effective write bandwidth seen by a GPU **410**-**413**. The efficiency of operand setup, the efficiency of results access, and the efficiency of GPU computation all play a role in determining the effectiveness of GPU offload.

[0147] A selection between GPU bias and host processor bias may be driven by a bias tracker data structure. A bias table may be used, for example, which may be a page-granular structure (i.e., controlled at the granularity of a memory page) that includes 1 or 2 bits per GPU-attached memory page. The bias table may be implemented in a stolen memory range of one or more GPU-attached memories **420**-**423**, with or without a bias cache in the GPU **410**-**413** (e.g., to cache frequently/recently used entries of the bias table). Alternatively, the entire bias table may be maintained within the GPU.

[0148] In one implementation, the bias table entry associated with each access to the GPU-attached memory **420**-**423** is accessed prior the actual access to the GPU memory, causing the following operations. First, local requests from the GPU **410**-**413** that find their page in GPU bias are forwarded directly to a corresponding GPU memory **420**-**423**. Local requests from the GPU that find their page in host bias are forwarded to the processor (e.g., over a high-speed link as discussed above). Optionally, requests from the host processor that find the requested page in host processor bias complete the request like a normal memory read. Alternatively, requests directed to a GPU-biased page may be forwarded to the GPU **410**-**413**. The GPU may then transition the page to a

host processor bias if it is not currently using the page. The bias state of a page can be changed either by a software-based mechanism, a hardware-assisted software-based mechanism, or, for a limited set of cases, a purely hardware-based mechanism. One mechanism for changing the bias state employs an API call (e.g., OpenCL), which, in turn, calls the GPU's device driver which, in turn, sends a message (or enqueues a command descriptor) to the GPU directing it to change the bias state and, for some transitions, perform a cache flushing operation in the host. The cache flushing operation is utilized for a transition from host processor bias to GPU bias but is not utilized for the opposite transition.

[0149] Cache coherency may be maintained by temporarily rendering GPU-biased pages uncacheable by the host processor. To access these pages, the host processor may request access from the GPU **410** which may or may not grant access right away, depending on the implementation. Thus, to reduce communication between the host processor and GPU **410** it is beneficial to ensure that GPU-biased pages are those which are utilized by the GPU but not the host processor and vice versa.

Graphics Processing Pipeline

[0150] FIG. **5** illustrates a graphics processing pipeline **500**. A graphics multiprocessor, such as graphics multiprocessor **234** as in FIG. **2**D or graphics multiprocessor **235** of FIG. **2**E can implement the graphics processing pipeline **500**. The graphics multiprocessor can be included within the parallel processing subsystems as described herein, such as the parallel processor **200** of FIG. **2**A, which may be related to the parallel processor(s) **112** of FIG. **1** and may be used in place of one of those. The various parallel processing systems can implement the graphics processing pipeline **500** via one or more instances of the parallel processing unit (e.g., parallel processing unit **202** of FIG. **2**A) as described herein. For example, a shader unit (e.g., graphics multiprocessor **234** of FIG. **2**C) may be configured to perform the functions of one or more of a vertex processing unit **504**, a tessellation control processing unit **508**, a tessellation evaluation processing unit **512**, a geometry processing unit **516**, and a fragment/pixel processing unit **524**. The functions of data assembler **502**, primitive assemblers **506**, **514**, **518**, tessellation unit **510**, rasterizer **522**, and raster operations unit **526** may also be performed by other processing engines within a processing cluster (e.g., processing cluster **214** of FIG. **2**A) and a corresponding partition unit (e.g., partition unit **220**A-**220**N of FIG. **2**A). The graphics processing pipeline **500** may also be implemented using dedicated processing units for one or more functions. It is also possible that one or more portions of the graphics processing pipeline **500** are performed by parallel processing logic within a general-purpose processor (e.g., CPU). Optionally, one or more portions of the graphics processing pipeline **500** can access on-chip memory (e.g., parallel processor memory **222** as in FIG. **2**A) via a memory interface **528**, which may be an instance of the memory interface **218** of FIG. **2**A. The graphics processing pipeline **500** may also be implemented via a multi-core group **365**A as in FIG. **3**.

[0151] The data assembler **502** is a processing unit that may collect vertex data for surfaces and primitives. The data assembler **502** then outputs the vertex data, including the vertex attributes, to the vertex processing unit **504**. The vertex processing unit **504** is a programmable execution unit that executes vertex shader programs, lighting and transforming vertex data as specified by the vertex shader programs. The vertex processing unit **504** reads data that is stored in cache, local or system memory for use in processing the vertex data and may be programmed to transform the vertex data from an object-based coordinate representation to a world space coordinate space or a normalized device coordinate space.

[0152] A first instance of a primitive assembler **506** receives vertex attributes from the vertex processing unit **504**. The primitive assembler **506** readings stored vertex attributes and constructs graphics primitives for processing by tessellation control processing unit **508**. The graphics primitives include triangles, line segments, points, patches, and so forth, as supported by various graphics processing application programming interfaces (APIs).

[0153] The tessellation control processing unit **508** treats the input vertices as control points for a

geometric patch. The control points are transformed from an input representation from the patch (e.g., the patch's bases) to a representation that is suitable for use in surface evaluation by the tessellation evaluation processing unit **512**. The tessellation control processing unit **508** can also compute tessellation factors for edges of geometric patches. A tessellation factor applies to a single edge and quantifies a view-dependent level of detail associated with the edge. A tessellation unit **510** is configured to receive the tessellation factors for edges of a patch and to tessellate the patch into multiple geometric primitives such as line, triangle, or quadrilateral primitives, which are transmitted to a tessellation evaluation processing unit **512**. The tessellation evaluation processing unit **512** operates on parameterized coordinates of the subdivided patch to generate a surface representation and vertex attributes for each vertex associated with the geometric primitives.

[0154] A second instance of a primitive assembler **514** receives vertex attributes from the tessellation evaluation processing unit **512**, reading stored vertex attributes, and constructs graphics primitives for processing by the geometry processing unit **516**. The geometry processing unit **516** is a programmable execution unit that executes geometry shader programs to transform graphics primitives received from primitive assembler **514** as specified by the geometry shader programs. The geometry processing unit **516** may be programmed to subdivide the graphics primitives into one or more new graphics primitives and calculate parameters used to rasterize the new graphics primitives.

[0155] The geometry processing unit **516** may be able to add or delete elements in the geometry stream. The geometry processing unit **516** outputs the parameters and vertices specifying new graphics primitives to primitive assembler **518**. The primitive assembler **518** receives the parameters and vertices from the geometry processing unit **516** and constructs graphics primitives for processing by a viewport scale, cull, and clip unit **520**. The geometry processing unit **516** reads data that is stored in parallel processor memory or system memory for use in processing the geometry data. The viewport scale, cull, and clip unit **520** performs clipping, culling, and viewport scaling and outputs processed graphics primitives to a rasterizer **522**.

[0156] The rasterizer **522** can perform depth culling and other depth-based optimizations. The rasterizer **522** also performs scan conversion on the new graphics primitives to generate fragments and output those fragments and associated coverage data to the fragment/pixel processing unit **524**. The fragment/pixel processing unit **524** is a programmable execution unit that is configured to execute fragment shader programs or pixel shader programs. The fragment/pixel processing unit **524** transforming fragments or pixels received from rasterizer **522**, as specified by the fragment or pixel shader programs. For example, the fragment/pixel processing unit **524** may be programmed to perform operations included but not limited to texture mapping, shading, blending, texture correction and perspective correction to produce shaded fragments or pixels that are output to a raster operations unit **526**. The fragment/pixel processing unit **524** can read data that is stored in either the parallel processor memory or the system memory for use when processing the fragment data. Fragment or pixel shader programs may be configured to shade at sample, pixel, tile, or other granularities depending on the sampling rate configured for the processing units.

[0157] The raster operations unit **526** is a processing unit that performs raster operations including, but not limited to stencil, z-test, blending, and the like, and outputs pixel data as processed graphics data to be stored in graphics memory (e.g., parallel processor memory **222** as in FIG. **2**A, and/or system memory **104** as in FIG. **1**), to be displayed on the one or more display device(s) **110**A-**110**B or for further processing by one of the one or more processor(s) **102** or parallel processor(s) **112**. The raster operations unit **526** may be configured to compress z or color data that is written to memory and decompress z or color data that is read from memory.

Machine Learning Overview

[0158] The architecture described above can be applied to perform training and inference operations using machine learning models. Machine learning has been successful at solving many kinds of tasks. The computations that arise when training and using machine learning algorithms

(e.g., neural networks) lend themselves naturally to efficient parallel implementations. Accordingly, parallel processors such as general-purpose graphics processing units (GPGPUs) have played a significant role in the practical implementation of deep neural networks. Parallel graphics processors with single instruction, multiple thread (SIMT) architectures are designed to maximize the amount of parallel processing in the graphics pipeline. In an SIMT architecture, groups of parallel threads attempt to execute program instructions synchronously together as often as possible to increase processing efficiency. The efficiency provided by parallel machine learning algorithm implementations allows the use of high-capacity networks and enables those networks to be trained on larger datasets.

[0159] A machine learning algorithm is an algorithm that can learn based on a set of data. For example, machine learning algorithms can be designed to model high-level abstractions within a data set. For example, image recognition algorithms can be used to determine which of several categories to which a given input belong; regression algorithms can output a numerical value given an input; and pattern recognition algorithms can be used to generate translated text or perform text to speech and/or speech recognition.

[0160] An example type of machine learning algorithm is a neural network. There are many types of neural networks; a simple type of neural network is a feedforward network. A feedforward network may be implemented as an acyclic graph in which the nodes are arranged in layers. Typically, a feedforward network topology includes an input layer and an output layer that are separated by at least one hidden layer. The hidden layer transforms input received by the input layer into a representation that is useful for generating output in the output layer. The network nodes are fully connected via edges to the nodes in adjacent layers, but there are no edges between nodes within each layer. Data received at the nodes of an input layer of a feedforward network are propagated (i.e., “fed forward”) to the nodes of the output layer via an activation function that calculates the states of the nodes of each successive layer in the network based on coefficients (“weights”) respectively associated with each of the edges connecting the layers. Depending on the specific model being represented by the algorithm being executed, the output from the neural network algorithm can take various forms.

[0161] Before a machine learning algorithm can be used to model a particular problem, the algorithm is trained using a training data set. Training a neural network involves selecting a network topology, using a set of training data representing a problem being modeled by the network, and adjusting the weights until the network model performs with a minimal error for all instances of the training data set. For example, during a supervised learning training process for a neural network, the output produced by the network in response to the input representing an instance in a training data set is compared to the “correct” labeled output for that instance, an error signal representing the difference between the output and the labeled output is calculated, and the weights associated with the connections are adjusted to minimize that error as the error signal is backward propagated through the layers of the network. The network is considered “trained” when the errors for each of the outputs generated from the instances of the training data set are minimized.

[0162] The accuracy of a machine learning algorithm can be affected significantly by the quality of the data set used to train the algorithm. The training process can be computationally intensive and may utilize a significant amount of time on a conventional general-purpose processor. Accordingly, parallel processing hardware is used to train many types of machine learning algorithms. This is particularly useful for optimizing the training of neural networks, as the computations performed in adjusting the coefficients in neural networks lend themselves naturally to parallel implementations. Specifically, many machine learning algorithms and software applications have been adapted to make use of the parallel processing hardware within general-purpose graphics processing devices.

[0163] FIG. **6** is a generalized diagram of a machine learning software stack **600**. A machine learning application **602** is any logic that can be configured to train a neural network using a

training dataset or to use a trained deep neural network to implement machine intelligence. The machine learning application **602** can include training and inference functionality for a neural network and/or specialized software that can be used to train a neural network before deployment. The machine learning application **602** can implement any type of machine intelligence including but not limited to image recognition, mapping and localization, autonomous navigation, speech synthesis, medical imaging, or language translation. Example machine learning applications **602** include, but are not limited to, voice-based virtual assistants, image or facial recognition algorithms, autonomous navigation, and the software tools that are used to train the machine learning models used by the machine learning applications **602**.

[0164] Hardware acceleration for the machine learning application **602** can be enabled via a machine learning framework **604**. The machine learning framework **604** can provide a library of machine learning primitives. Machine learning primitives are basic operations that are commonly performed by machine learning algorithms. Without the machine learning framework **604**, developers of machine learning algorithms would have to create and optimize the main computational logic associated with the machine learning algorithm, then re-optimize the computational logic as new parallel processors are developed. Instead, the machine learning application can be configured to perform the computations using the primitives provided by the machine learning framework **604**. Example primitives include tensor convolutions, activation functions, and pooling, which are computational operations that are performed while training a convolutional neural network (CNN). The machine learning framework **604** can also provide primitives to implement basic linear algebra subprograms performed by many machine-learning algorithms, such as matrix and vector operations. Examples of a machine learning framework **604** include, but are not limited to, TensorFlow, TensorRT, PyTorch, MXNet, Caffe, and other high-level machine learning frameworks.

[0165] The machine learning framework **604** can process input data received from the machine learning application **602** and generate the appropriate input to a compute framework **606**. The compute framework **606** can abstract the underlying instructions provided to the GPGPU driver **608** to enable the machine learning framework **604** to take advantage of hardware acceleration via the GPGPU hardware **610** without having the machine learning framework **604** to have intimate knowledge of the architecture of the GPGPU hardware **610**. Additionally, the compute framework **606** can enable hardware acceleration for the machine learning framework **604** across a variety of types and generations of the GPGPU hardware **610**. In one example, compute frameworks **606** can include the CUDA compute framework and associated machine learning libraries, such as the CUDA Deep Neural Network (cuDNN) library. The machine learning software stack **600** can also include communication libraries or frameworks to facilitate multi-GPU and multi-node compute.

GPGPU Machine Learning Acceleration

[0166] FIG. **7** illustrates a general-purpose graphics processing unit (GPGPU **700**), which may be the parallel processor **200** of FIG. **2**A or the parallel processor(s) **112** of FIG. **1**. The general-purpose processing unit may be configured to provide support for hardware acceleration of primitives provided by a machine learning framework to accelerate the processing the type of computational workloads associated with training deep neural networks. Additionally, the GPGPU **700** can be linked directly to other instances of the GPGPU to create a multi-GPU cluster to improve training speed for particularly deep neural networks. Primitives are also supported to accelerate inference operations for deployed neural networks.

[0167] The GPGPU **700** includes a host interface **702** to enable a connection with a host processor. The host interface **702** may be a PCI Express interface. However, the host interface can also be a vendor specific communications interface or communications fabric. The GPGPU **700** receives commands from the host processor and uses a global scheduler **704** to distribute execution threads associated with those commands to a set of processing clusters **706**A-**706**H. The processing clusters **706**A-**706**H share a cache memory **708**. The cache memory **708** can serve as a higher-level

cache for cache memories within the processing clusters **706**A-**706**H. The illustrated processing clusters **706**A-**706**H may correspond with processing clusters **214**A-**214**N as in FIG. **2**A.
[0168] The GPGPU **700** includes memory **714**A-**714**B coupled with the processing clusters **706**A-**706**H via a set of memory controllers **712**A-**712**B. The memory **714**A-**714**B can include various types of memory devices including dynamic random-access memory (DRAM) or graphics random access memory, such as synchronous graphics random access memory (SGRAM), including graphics double data rate (GDDR) memory. The memory **714**A-**714**B may also include 3D stacked memory, including but not limited to high bandwidth memory (HBM).
[0169] Each of the processing clusters **706**A-**706**H may include a set of graphics multiprocessors, such as the graphics multiprocessor **234** of FIG. **2**D, graphics multiprocessor **235** of FIG. **2**E, or may include a multi-core group **365**A-**365**N as in FIG. **3**. The graphics multiprocessors of the compute cluster include multiple types of integer and floating-point logic units that can perform computational operations at a range of precisions including suited for machine learning computations. For example, at least a subset of the floating-point units in each of the processing clusters **706**A-**706**H can be configured to perform 16-bit or 32-bit floating-point operations, while a different subset of the floating-point units can be configured to perform 64-bit floating-point operations.
[0170] Multiple instances of the GPGPU **700** can be configured to operate as a compute cluster. The communication mechanism used by the compute cluster for synchronization and data exchange varies across embodiments. For example, the multiple instances of the GPGPU **700** communicate over the host interface **702**. In one embodiment the GPGPU **700** includes an I/O hub **709** that couples the GPGPU **700** with a GPU link **710** that enables a direct connection to other instances of the GPGPU. The GPU link **710** may be coupled to a dedicated GPU-to-GPU bridge that enables communication and synchronization between multiple instances of the GPGPU **700**. Optionally, the GPU link **710** couples with a high-speed interconnect to transmit and receive data to other GPGPUs or parallel processors. The multiple instances of the GPGPU **700** may be located in separate data processing systems and communicate via a network device that is accessible via the host interface **702**. The GPU link **710** may be configured to enable a connection to a host processor in addition to or as an alternative to the host interface **702**.
[0171] While the illustrated configuration of the GPGPU **700** can be configured to train neural networks, an alternate configuration of the GPGPU **700** can be configured for deployment within a high performance or low power inferencing platform. In an inferencing configuration, the GPGPU **700** includes fewer of the processing clusters **706**A-**706**H relative to the training configuration. Additionally, memory technology associated with the memory **714**A-**714**B may differ between inferencing and training configurations. In one embodiment, the inferencing configuration of the GPGPU **700** can support inferencing specific instructions. For example, an inferencing configuration can provide support for one or more 8-bit integer or floating-point dot product instructions, which are commonly used during inferencing operations for deployed neural networks.
[0172] FIG. **8** illustrates a multi-GPU computing system **800**. The multi-GPU computing system **800** can include a processor **802** coupled to multiple GPGPUs **806**A-**806**D via a host interface switch **804**. The host interface switch **804** may be a PCI express switch device that couples the processor **802** to a PCI express bus over which the processor **802** can communicate with the set of GPGPUs **806**A-**806**D. Each of the multiple GPGPUs **806**A-**806**D can be an instance of the GPGPU **700** of FIG. **7**. The GPGPUs **806**A-**806**D can interconnect via a set of high-speed point to point GPU to GPU links (P2P GPU links **816**). The high-speed GPU to GPU links can connect to each of the GPGPUs **806**A-**806**D via a dedicated GPU link, such as the GPU link **710** as in FIG. **7**. The P2P GPU links **816** enable direct communication between each of the GPGPUs **806**A-**806**D without having communication over the host interface bus to which the processor **802** is connected. With GPU-to-GPU traffic directed to the P2P GPU links, the host interface bus remains available

for system memory access or to communicate with other instances of the multi-GPU computing system **800**, for example, via one or more network devices. While in FIG. **8** the GPGPUs **806**A-**806**D connect to the processor **802** via the host interface switch **804**, the processor **802** may alternatively include direct support for the P2P GPU links **816** and connect directly to the GPGPUs **806**A-**806**D. In one embodiment the P2P GPU link **816** enable the multi-GPU computing system **800** to operate as a single logical GPU.

Machine Learning Neural Network Implementations

[0173] The computing architecture described herein can be configured to perform the types of parallel processing that is particularly suited for training and deploying neural networks for machine learning. A neural network can be generalized as a network of functions having a graph relationship. As is well-known in the art, there are a variety of types of neural network implementations used in machine learning. One example type of neural network is the feedforward network, as previously described. A second example type of neural network is the Convolutional Neural Network (CNN), while a third example type of neural network are recurrent neural networks (RNNs).

[0174] A CNN is a specialized feedforward neural network for processing data having a known, grid-like topology, such as image data. Accordingly, CNNs are commonly used for compute vision and image recognition applications, but they also may be used for other types of pattern recognition such as speech and language processing. The nodes in the CNN input layer are organized into a set of "filters" (feature detectors inspired by the receptive fields found in the retina), and the output of each set of filters is propagated to nodes in successive layers of the network. The computations for a CNN include applying the convolution mathematical operation to each filter to produce the output of that filter. Convolution is a specialized kind of mathematical operation performed by two functions to produce a third function that is a modified version of one of the two original functions. In convolutional network terminology, the first function to the convolution can be referred to as the input, while the second function can be referred to as the convolution kernel. The output may be referred to as the feature map. The input to a convolution layer can be a multidimensional array of data that defines the various color components of an input image. The convolution kernel can be a multidimensional array of parameters, where the parameters are adapted by the training process for the neural network.

[0175] RNNs are a family of feedforward neural networks that include feedback connections between layers. RNNs enable modeling of sequential data by sharing parameter data across different parts of the neural network. The architecture for an RNN includes cycles that represent the influence of a present value of a variable on its own value at a future time, as at least a portion of the output data from the RNN is used as feedback for processing subsequent input in a sequence. This feature makes RNNs particularly useful for language processing due to the variable nature in which language data can be composed.

[0176] The figures described below present example feedforward, CNN, and RNN networks, as well as describe a general process for respectively training and deploying each of those types of networks. It will be understood that these descriptions are example and non-limiting as to any specific embodiment described herein and the concepts illustrated can be applied generally to deep neural networks and machine learning techniques in general.

[0177] Deep neural networks used in deep learning typically include a front-end network to perform feature recognition coupled to a back-end network which represents a mathematical model that can perform operations (e.g., object classification, speech recognition, etc.) based on the feature representation provided to the model. Deep learning enables machine learning to be performed without having hand crafted feature engineering to be performed for the model. Instead, deep neural networks can learn features based on statistical structure or correlation within the input data. The learned features can be provided to a mathematical model that can map detected features to an output. The mathematical model used by the network is generally specialized for the specific

task to be performed, and different models will be used to perform different task.
[0178] Once the neural network is structured, a learning model can be applied to the network to train the network to perform specific tasks. The learning model describes how to adjust the weights within the model to reduce the output error of the network. Backpropagation of errors is a common method used to train neural networks. An input vector is presented to the network for processing. The output of the network is compared to the desired output using a loss function and an error value is calculated for each of the neurons in the output layer. The error values are then propagated backwards until each neuron has an associated error value which roughly represents its contribution to the original output. The network can then learn from those errors using an algorithm, such as the stochastic gradient descent algorithm, to update the weights of the of the neural network.
[0179] FIG. **9**A-**9**B illustrate an example convolutional neural network. FIG. **9**A illustrates various layers within a CNN. As shown in FIG. **9**A, an example CNN used to model image processing can receive input **902** describing the red, green, and blue (RGB) components of an input image. The input **902** can be processed by multiple convolutional layers (e.g., first convolutional layer **904**, second convolutional layer **906**). The output from the multiple convolutional layers may optionally be processed by fully connected layers **908**. Neurons in a fully connected layer have full connections to all activations in the previous layer, as previously described for a feedforward network. The output from the fully connected layers **908** can be used to generate an output result from the network. The activations within the fully connected layers **908** can be computed using matrix multiplication instead of convolution. Not all CNN implementations make use of fully connected layers **908**. For example, in some implementations the second convolutional layer **906** can generate output for the CNN.
[0180] The convolutional layers are sparsely connected, which differs from traditional neural network configuration found in the fully connected layers **908**. Traditional neural network layers are fully connected, such that every output unit interacts with every input unit. However, the convolutional layers are sparsely connected because the output of the convolution of a field is input (instead of the respective state value of each of the nodes in the field) to the nodes of the subsequent layer, as illustrated. The kernels associated with the convolutional layers perform convolution operations, the output of which is sent to the next layer. The dimensionality reduction performed within the convolutional layers is one aspect that enables the CNN to scale to process large images.
[0181] FIG. **9**B illustrates example computation stages within a convolutional layer of a CNN. Input to a convolutional layer **912** of a CNN can be processed in three stages of a convolutional layer **914**. The three stages can include a convolution stage **916**, a detector stage **918**, and a pooling stage **920**. The convolutional layer **914** can then output data to a successive convolutional layer. The final convolutional layer of the network can generate output feature map data or provide input to a fully connected layer, for example, to generate a classification value for the input to the CNN.
[0182] In the convolution stage **916** performs several convolutions in parallel to produce a set of linear activations. The convolution stage **916** can include an affine transformation, which is any transformation that can be specified as a linear transformation plus a translation. Affine transformations include rotations, translations, scaling, and combinations of these transformations. The convolution stage computes the output of functions (e.g., neurons) that are connected to specific regions in the input, which can be determined as the local region associated with the neuron. The neurons compute a dot product between the weights of the neurons and the region in the local input to which the neurons are connected. The output from the convolution stage **916** defines a set of linear activations that are processed by successive stages of the convolutional layer **914**.
[0183] The linear activations can be processed by a detector stage **918**. In the detector stage **918**, each linear activation is processed by a non-linear activation function. The non-linear activation function increases the nonlinear properties of the overall network without affecting the receptive

fields of the convolution layer. Several types of non-linear activation functions may be used. One particular type is the rectified linear unit (ReLU), which uses an activation function defined as $f(x)=\max(0, x)$, such that the activation is thresholded at zero.

[0184] The pooling stage **920** uses a pooling function that replaces the output of the second convolutional layer **906** with a summary statistic of the nearby outputs. The pooling function can be used to introduce translation invariance into the neural network, such that small translations to the input do not change the pooled outputs. Invariance to local translation can be useful in scenarios where the presence of a feature in the input data is more important than the precise location of the feature. Various types of pooling functions can be used during the pooling stage **920**, including max pooling, average pooling, and l2-norm pooling. Additionally, some CNN implementations do not include a pooling stage. Instead, such implementations substitute and additional convolution stage having an increased stride relative to previous convolution stages.

[0185] The output from the convolutional layer **914** can then be processed by the next layer **922**. The next layer **922** can be an additional convolutional layer or one of the fully connected layers **908**. For example, the first convolutional layer **904** of FIG. **9**A can output to the second convolutional layer **906**, while the second convolutional layer can output to a first layer of the fully connected layers **908**.

[0186] A variant on the CNN is a convolutional deep belief network, which has a structure similar to a CNN and is trained in a manner similar to a deep belief network. A deep belief network (DBN) is a generative neural network that is composed of multiple layers of stochastic (random) variables. DBNs can be trained layer-by-layer using greedy unsupervised learning. The learned weights of the DBN can then be used to provide pre-train neural networks by determining an initial set of weights for the neural network.

[0187] FIG. **10**A-**10**B illustrate an example language models. FIG. **10**A illustrates a recurrent neural network (RNN **1000**). In a recurrent neural network (RNN), the previous state of the network influences the output of the current state of the network. RNNs can be built in a variety of ways using a variety of functions. The use of RNNs generally revolves around using mathematical models to predict the future based on a prior sequence of inputs. For example, an RNN may be used to perform statistical language modeling to predict an upcoming word given a previous sequence of words. The RNN **1000** can be described has having an input layer **1002** that receives an input vector, hidden layers **1004** to implement a recurrent function, a feedback mechanism **1005** to enable a 'memory' of previous states, and an output layer **1006** to output a result. The RNN **1000** operates based on time-steps. The state of the RNN at a given time step is influenced based on the previous time step via the feedback mechanism **1005**. For a given time step, the state of the hidden layers **1004** is defined by the previous state and the input at the current time step. An initial input (x.sub.1) at a first-time step can be processed by the hidden layer **1004**. A second input (x.sub.2) can be processed by the hidden layer **1004** using state information that is determined during the processing of the initial input (x.sub.1). A given state can be computed as s.sub.t=f(Ux.sub.t+Ws.sub.t-1), where U and W are parameter matrices. The function f is generally a nonlinearity, such as the hyperbolic tangent function (Tanh) or a variant of the rectifier function $f(x)=\max(0, x)$. However, the specific mathematical function used in the hidden layers **1004** can vary depending on the specific implementation details of the RNN **1000**. Acceleration for variations on RNN networks may also be enabled. One example RNN variant is the long short term memory (LSTM) RNN. LSTM RNNs are capable of learning long-term dependencies that may be utilized for processing longer sequences of language.

[0188] FIG. **10**B illustrates baseline components of a transformer model **1010**. The transformer model **1010** address issues found in RNN models with long input sequences as well as enabling enable increased parallelization. The transformer model **1010**, and variants thereof, are used to build Language Models (LLMs) that perform various tasks such as machine translation, automatic summarization, and dialogue management. A transformer model **1010** may also be configured to

perform image generation tasks, such as text-to-image generation.
[0189] The transformer model **1010** includes multiple instances of an encoder **1016** and a decoder **1026**. The encoders are stacked end-to-end, with the output of the final encoder being routed as input to a multiheaded attention layer of each decoder **1026**. Input that enters the bottommost instance of encoder **1016** is processed by input embeddings **1012** that convert input tokens into vectors that can be processed by the encoder **1016**. The output dictionary is vectorized by output embeddings **1022** before entry into the bottommost instance of decoder **1026**. Positional encodings **1014**, **1024** are added to input vectors and output vectors at the bottoms of the encoder **1016** and decoder **1026** stacks. The positional encodings **1014**, **1024**, inject information about the relative or absolute position of tokens in a sequence of tokens to be processed as the transformer model **1010** does not naturally encode the order of the tokens.
[0190] The encoder **1016** of the transformer model **1010** analyses the input text and creates a number of hidden states that protect the context and meaning of the text data. Encoder **1016** layers make up part of the core of the transformer architecture, although decoder **1026** only variants of the transformer model **1010** are also possible. The encoder **1016** includes two sublayers, a multi-head attention (MHA) sublayer and a feedforward network (FFN) sublayer. The MHA sublayer performs multiple concurrent self-attention operations to compute attention scores, which enable the transformer model **1010** weigh the importance and relative relationships of different tokens in the input sequence in a context-aware manner. The FFN sublayer is a position-wise fully connected feedforward neural network. The output of each sublayer is processed by an add and normalize (A&N) operation, defined as LayerNorm(x+Sublayer(x)), where the output of the sublayer is added to the input of the sublayer and normalized. In some implementations, the normalization operation may be performed before the sublayer instead of afterwards.
[0191] The decoder **1026** includes three sublayers, a masked MHA sublayer, an MHA sublayer, and an FFN sublayer. The masked MHA sublayer is similar to the MHA layer, excepting that masking is applied to prevent query position from attending to the keys of future positions. The MHA sublayer of the decoder **1026** is similar to the MHA sublayer of the encoder, with an additional input from the encoder **1016**. The FFN of the decoder **1026** is the same as the FFN of the encoder **1016**. The Linear and Softmax block takes the output of the last instance of decoder **1026** of the decoder stack and generates a probability distribution that represent output probabilities.
[0192] GPU acceleration is also available for variants of the transformer model **1010** that replace some or all FFN sublayers with sparse mixture of experts (MoE) layers. MoE layers each include a number of experts, where each expert is a neural network. The MoE layers may themselves be FFNs, or may also be MoEs, allowing hierarchical MoE layers.
[0193] The training of a transformer model **1010** can be optimized via the use of adaptive precision logic that adjusts the computation precision that is applied during training. The adaptive precision logic can attempt to use the smallest datatype possible during training without significantly reducing training accuracy. For example, to minimize data loss due to the use of 16-bit, 8-bit, and 4-bit floating-point format, dynamic scaling and casting can be applied during training based on the statistical analysis of the tensor data that is generated during training. Tensor data that is generated during training can be statistically analyzed using a variety of analysis techniques to determine a set of scaling factors to apply to blocks of data, such as the input to each layer of the transformer model **1010**. For example, absolute minimum and/or maximum values can be used to determine scaling factors that can prevent underflow or overflow of low-precision floating-point data types.
[0194] FIG. **11** illustrates training and deployment of a deep neural network. Once a given network has been structured for a task the neural network is trained using a training dataset **1102**. Training frameworks **1104** have been developed to enable hardware acceleration of the training process. For example, the machine learning framework **604** of FIG. **6** may be configured as a training framework **1104**. The training framework **1104** can hook into an untrained neural network **1106** and enable the untrained neural net to be trained using the parallel processing resources described

herein to generate a trained neural network **1108**. To start the training process the initial weights may be chosen randomly or by pre-training using a deep belief network. The training cycle then be performed in either a supervised or unsupervised manner.

[0195] Supervised learning is a learning method in which training is performed as a mediated operation, such as when the training dataset **1102** includes input paired with the desired output for the input, or where the training dataset includes input having known output and the output of the neural network is manually graded. The network processes the inputs and compares the resulting outputs against a set of expected or desired outputs. Errors are then propagated back through the system. The training framework **1104** can adjust to adjust the weights that control the untrained neural network **1106**. The training framework **1104** can provide tools to monitor how well the untrained neural network **1106** is converging towards a model suitable to generating correct answers based on known input data. The training process occurs repeatedly as the weights of the network are adjusted to refine the output generated by the neural network. The training process can continue until the neural network reaches a statistically desired accuracy associated with a trained neural network **1108**. The trained neural network **1108** can then be deployed to implement any number of machine learning operations to generate an inference result **1114** based on input of new data **1112**.

[0196] Unsupervised learning is a learning method in which the network attempts to train itself using unlabeled data. Thus, for unsupervised learning the training dataset **1102** will include input data without any associated output data. The untrained neural network **1106** can learn groupings within the unlabeled input and can determine how individual inputs are related to the overall dataset. Unsupervised training can be used to generate a self-organizing map, which is a type of trained neural network **1108** capable of performing operations useful in reducing the dimensionality of data. Unsupervised training can also be used to perform anomaly detection, which allows the identification of data points in an input dataset that deviate from the normal patterns of the data.

[0197] Variations on supervised and unsupervised training may also be employed. Semi-supervised learning is a technique in which in the training dataset **1102** includes a mix of labeled and unlabeled data of the same distribution. Incremental learning is a variant of supervised learning in which input data is continuously used to further train the model. Incremental learning enables the trained neural network **1108** to adapt to the new data **1112** without forgetting the knowledge instilled within the network during initial training. Whether supervised or unsupervised, the training process for particularly deep neural networks may be too computationally intensive for a single compute node. Instead of using a single compute node, a distributed network of computational nodes can be used to accelerate the training process.

[0198] FIG. **12**A is a block diagram illustrating distributed learning. Distributed learning is a training model that uses multiple distributed computing nodes to perform supervised or unsupervised training of a neural network. The distributed computational nodes can each include one or more host processors and one or more of the general-purpose processing nodes, such as the GPGPU **700** as in FIG. **7**. As illustrated, distributed learning can be performed with model parallelism **1202**, data parallelism **1204**, or a combination of model and data parallelism **1206**.

[0199] In model parallelism **1202**, different computational nodes in a distributed system can perform training computations for different parts of a single network. For example, each layer of a neural network can be trained by a different processing node of the distributed system. The benefits of model parallelism include the ability to scale to particularly large models. Splitting the computations associated with different layers of the neural network enables the training of very large neural networks in which the weights of all layers would not fit into the memory of a single computational node. In some instances, model parallelism can be particularly useful in performing unsupervised training of large neural networks. Some implementations of model parallelism **1202** may also be referred to as tensor parallelism.

[0200] In data parallelism **1204**, the different nodes of the distributed network have a complete

instance of the model and each node receives a different portion of the data. The results from the different nodes are then combined. While different approaches to data parallelism are possible, data parallel training approaches all utilize a technique of combining results and synchronizing the model parameters between each node. Example approaches to combining data include parameter averaging and update-based data parallelism. Parameter averaging trains each node on a subset of the training data and sets the global parameters (e.g., weights, biases) to the average of the parameters from each node. Parameter averaging uses a central parameter server that maintains the parameter data. Update based data parallelism is similar to parameter averaging except that instead of transferring parameters from the nodes to the parameter server, the updates to the model are transferred. Additionally, update-based data parallelism can be performed in a decentralized manner, where the updates are compressed and transferred between nodes.

[0201] Combined model and data parallelism **1206** can be implemented, for example, in a distributed system in which each computational node includes multiple GPUs. Combined model and data parallelism **1206** may also be referred to as hybrid parallelism. Each node can have a complete instance of the model, with separate GPUs within each node being used to train different portions of the model. Distributed training has increased overhead relative to training on a single machine. However, the parallel processors and GPGPUs described herein can each implement various techniques to reduce the overhead of distributed training, including techniques to enable high bandwidth GPU-to-GPU data transfer and accelerated remote data synchronization. Pipeline parallelism is a variant of combined model and data parallelism **1206**, in which different nodes contain less than the entire model, but more than a single layer of the model. In pipeline parallelism, different groups of layers or sub-models are distributed across the different processing nodes. Another variant is expert parallelism, which routes requests to specific experts within the model to different GPUs. Expert parallelism can be used, for example, with MoE transformer models.

[0202] FIG. **12**B is a block diagram illustrating a programmable network interface **1210** and data processing unit. The programmable network interface **1210** is a programmable network engine that can be used to accelerate network-based compute tasks within a distributed environment. The programmable network interface **1210** can couple with a host system via host interface **1270**. The programmable network interface **1210** can be used to accelerate network or storage operations for CPUs or GPUs of the host system. The host system can be, for example, a node of a distributed learning system used to perform distributed training, for example, as shown in FIG. **12**A. The host system can also be a data center node within a data center.

[0203] In one embodiment, access to remote storage containing model data can be accelerated by the programmable network interface **1210**. For example, the programmable network interface **1210** can be configured to present remote storage devices as local storage devices to the host system. The programmable network interface **1210** can also accelerate remote direct memory access (RDMA) operations performed between GPUs of the host system with GPUs of remote systems. In one embodiment, the programmable network interface **1210** can enable storage functionality such as, but not limited to NVME-oF. The programmable network interface **1210** can also accelerate encryption, data integrity, compression, and other operations for remote storage on behalf of the host system, allowing remote storage to approach the latencies of storage devices that are directly attached to the host system.

[0204] The programmable network interface **1210** can also perform resource allocation and management on behalf of the host system. Storage security operations can be offloaded to the programmable network interface **1210** and performed in concert with the allocation and management of remote storage resources. Network-based operations to manage access to the remote storage that would otherwise by performed by a processor of the host system can instead be performed by the programmable network interface **1210**.

[0205] In one embodiment, network and/or data security operations can be offloaded from the host

system to the programmable network interface **1210**. Data center security policies for a data center node can be handled by the programmable network interface **1210** instead of the processors of the host system. For example, the programmable network interface **1210** can detect and mitigate against an attempted network-based attack (e.g., DDoS) on the host system, preventing the attack from compromising the availability of the host system.

[0206] The programmable network interface **1210** can include a system on a chip (SoC **1220**) that executes an operating system via multiple processor cores **1222**. The processor cores **1222** can include general-purpose processor (e.g., CPU) cores. In one embodiment the processor cores **1222** can also include one or more GPU cores. The SoC **1220** can execute instructions stored in a memory device **1240**. A storage device **1250** can store local operating system data. The storage device **1250** and memory device **1240** can also be used to cache remote data for the host system. Network ports **1260**A-**1260**B enable a connection to a network or fabric and facilitate network access for the SoC **1220** and, via the host interface **1270**, for the host system. The programmable network interface **1210** can also include an I/O interface **1275**, such as a USB interface. The I/O interface **1275** can be used to couple external devices to the programmable network interface **1210** or as a debug interface. The programmable network interface **1210** also includes a management interface **1230** that enables software on the host device to manage and configure the programmable network interface **1210** and/or SoC **1220**. In one embodiment the programmable network interface **1210** may also include one or more accelerators or GPUs **1245** to accept offload of parallel compute tasks from the SoC **1220**, host system, or remote systems coupled via the network ports **1260**A-**1260**B.

Example Machine Learning Applications

[0207] Machine learning can be applied to solve a variety of technological problems, including but not limited to computer vision, autonomous driving and navigation, speech recognition, and language processing. Computer vision has traditionally been an active research areas for machine learning applications. Applications of computer vision range from reproducing human visual abilities, such as recognizing faces, to creating new categories of visual abilities. For example, computer vision applications can be configured to recognize sound waves from the vibrations induced in objects visible in a video. Parallel processor accelerated machine learning enables computer vision applications to be trained using significantly larger training dataset than previously feasible and enables inferencing systems to be deployed using low power parallel processors.

[0208] Parallel processor accelerated machine learning has autonomous driving applications including lane and road sign recognition, obstacle avoidance, navigation, and driving control. Accelerated machine learning techniques can be used to train driving models based on datasets that define the appropriate responses to specific training input. The parallel processors described herein can enable rapid training of the increasingly complex neural networks used for autonomous driving solutions and enables the deployment of low power inferencing processors in a mobile platform suitable for integration into autonomous vehicles.

[0209] Parallel processor accelerated deep neural networks have enabled machine learning approaches to automatic speech recognition (ASR). ASR includes the creation of a function that computes a probable linguistic sequence given an input acoustic sequence. Accelerated machine learning using deep neural networks have enabled the replacement of the hidden Markov models (HMMs) and Gaussian mixture models (GMMs) previously used for ASR.

[0210] Parallel processor accelerated machine learning can also be used to accelerate natural language processing. Automatic learning procedures can make use of statistical inference algorithms to produce models that are robust to erroneous or unfamiliar input. Example natural language processor applications include automatic machine translation between human languages.

[0211] The parallel processing platforms used for machine learning can be divided into training platforms and deployment platforms. Training platforms are generally highly parallel and include optimizations to accelerate multi-GPU single node training and multi-node, multi-GPU training.

Example parallel processors suited for training include the GPGPU **700** of FIG. **7** and the multi-GPU computing system **800** of FIG. **8**. On the contrary, deployed machine learning platforms generally include lower power parallel processors suitable for use in products such as cameras, autonomous robots, and autonomous vehicles.

[0212] Additionally, machine learning techniques can be applied to accelerate or enhance graphics processing activities. For example, a machine learning model can be trained to recognize output generated by a GPU accelerated application and generate an upscaled version of that output. Such techniques can be applied to accelerate the generation of high-resolution images for a gaming application. Various other graphics pipeline activities can benefit from the use of machine learning. For example, machine learning models can be trained to perform tessellation operations on geometry data to increase the complexity of geometric models, allowing fine-detailed geometry to be automatically generated from geometry of relatively lower detail.

[0213] FIG. **13** illustrates an example inferencing system on a chip (SOC **1300**) suitable for performing inferencing using a trained model. The SOC **1300** can integrate processing components including a media processor **1302**, a vision processor **1304**, a GPGPU **1306** and a multi-core processor **1308**. The GPGPU **1306** may be a GPGPU as described herein, such as the GPGPU **700**, and the multi-core processor **1308** may be a multi-core processor described herein, such as the multi-core processors **405**-**406**. The SOC **1300** can additionally include on-chip memory **1305** that can enable a shared on-chip data pool that is accessible by each of the processing components. The processing components can be optimized for low power operation to enable deployment to a variety of machine learning platforms, including autonomous vehicles and autonomous robots. For example, one implementation of the SOC **1300** can be used as a portion of the main control system for an autonomous vehicle. Where the SOC **1300** is configured for use in autonomous vehicles the SOC is designed and configured for compliance with the relevant functional safety standards of the deployment jurisdiction.

[0214] During operation, the media processor **1302** and vision processor **1304** can work in concert to accelerate computer vision operations. The media processor **1302** can enable low latency decode of multiple high-resolution (e.g., **4**K, **8**K) video streams. The decoded video streams can be written to a buffer in the on-chip memory **1305**. The vision processor **1304** can then parse the decoded video and perform preliminary processing operations on the frames of the decoded video in preparation of processing the frames using a trained image recognition model. For example, the vision processor **1304** can accelerate convolution operations for a CNN that is used to perform image recognition on the high-resolution video data, while back-end model computations are performed by the GPGPU **1306**.

[0215] The multi-core processor **1308** can include control logic to assist with sequencing and synchronization of data transfers and shared memory operations performed by the media processor **1302** and the vision processor **1304**. The multi-core processor **1308** can also function as an application processor to execute software applications that can make use of the inferencing compute capability of the GPGPU **1306**. For example, at least a portion of the navigation and driving logic can be implemented in software executing on the multi-core processor **1308**. Such software can directly issue computational workloads to the GPGPU **1306** or the computational workloads can be issued to the multi-core processor **1308**, which can offload at least a portion of those operations to the GPGPU **1306**.

[0216] The GPGPU **1306** can include compute clusters such as a low power configuration of the processing clusters **706**A-**706**H within the GPGPU **700**. The compute clusters within the GPGPU **1306** can support instruction that are specifically optimized to perform inferencing computations on a trained neural network. For example, the GPGPU **1306** can support instructions to perform low precision computations such as 8-bit and 4-bit integer vector operations.

Additional System Overview

[0217] FIG. **14** is a block diagram of a processing system **1400**. The elements of FIG. **14** having

the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. The processing system **1400** may be used in a single processor desktop system, a multiprocessor workstation system, or a server system having processor(s) **1402** or processor cores **1407**. The processing system **1400** may be a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices such as within Internet-of-things (IoT) devices with wired or wireless connectivity to a local or wide area network.
[0218] The processing system **1400** may be a processing system having components that correspond with those of FIG. **1**. For example, in different configurations, processor(s) **1402** or processor cores **1407** may correspond with processor(s) **102** of FIG. **1**. The graphics processor(s) **1408** may correspond with parallel processor(s) **112** of FIG. **1**. The external graphics processor **1418** may be one of the add-in device(s) **120** of FIG. **1**.
[0219] The processing system **1400** can include, couple with, or be integrated within: a server-based gaming platform; a game console, including a game and media console; a mobile gaming console, a handheld game console, or an online game console. The processing system **1400** may be part of a mobile phone, smart phone, tablet computing device or mobile Internet-connected device such as a laptop with low internal storage capacity. The processing system **1400** can also include, couple with, or be integrated within: a wearable device, such as a smart watch wearable device; smart eyewear or clothing enhanced with augmented reality (AR) or virtual reality (VR) features to provide visual, audio or tactile outputs to supplement real world visual, audio or tactile experiences or otherwise provide text, audio, graphics, video, holographic images or video, or tactile feedback. The processing system **1400** may include or be part of a television or set top box device. The processing system **1400** can include, couple with, or be integrated within a self-driving vehicle such as a bus, tractor trailer, car, motor or electric power cycle, plane or glider (or any combination thereof). The self-driving vehicle may use the processing system **1400** to process the environment sensed around the vehicle.
[0220] The processor(s) **1402** may include one or more instances of processor cores **1407** to process instructions which, when executed, perform operations for system or user software. The least one of the processor cores **1407** may be configured to process a specific instruction set **1409**. The instruction set **1409** may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). In one embodiment, one of the processor cores **1407** may process a different instruction set **1409**, which may include instructions to facilitate the emulation of other instruction sets. Processor core **1407** may also include other processing devices, such as a Digital Signal Processor (DSP).
[0221] The processor(s) **1402** may include cache memory **1404**. Depending on the architecture, the processor(s) **1402** can have a single internal cache or multiple levels of internal cache. In some embodiments, the cache memory is shared among various components of the processor(s) **1402**. In some embodiments, the processor(s) **1402** also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor cores **1407** using known cache coherency techniques. A register file **1406** can be additionally included in the processor(s) **1402** and may include different types of registers for storing different types of data (e.g., integer registers, floating-point registers, status registers, and an instruction pointer register). Some registers may be general-purpose registers, while other registers may be specific to the design of the processor(s) **1402**.
[0222] The processor(s) **1402** may be coupled with one or more interface bus(es) **1410** to transmit communication signals such as address, data, or control signals between the processor(s) **1402** and other components in the processing system **1400**. The interface bus(es) **1410**, in one of these embodiments, can be a processor bus, such as a version of the Direct Media Interface (DMI) bus. However, processor busses are not limited to the DMI bus, and may include one or more Peripheral

Component Interconnect buses (e.g., PCI, PCI express), memory busses, or other types of interface busses. For example, the processor(s) **1402** may include a memory controller **1416** and a platform controller hub **1430**. The memory controller **1416** facilitates communication between a memory device and other components of the processing system **1400**, while the platform controller hub **1430** provides connections to I/O devices via a local I/O bus.

[0223] The memory device **1420** can be a dynamic random-access memory (DRAM) device, a static random-access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. The memory device **1420** can, for example, operate as system memory for the processing system **1400**, to store data **1422** and instructions **1421** for use when the processor(s) **1402** executes an application or process. The memory controller **1416** can optionally couple with an external graphics processor **1418**, which may communicate with the graphics processor(s) **1408** in the processor(s) **1402** to perform graphics and media operations. In some embodiments, graphics, media, and or compute operations may be assisted by an accelerator **1412** which is a coprocessor that can be configured to perform a specialized set of graphics, media, or compute operations. For example, the accelerator **1412** may be a matrix multiplication accelerator used to optimize machine learning or compute operations. The accelerator **1412** can be a ray-tracing accelerator that can be used to perform ray-tracing operations in concert with the graphics processor(s) **1408**. The accelerator **1412** may also be an AI accelerator or NPU to accelerate neural network training or inference operations. In one embodiment, an external accelerator **1419** may be used in place of or in concert with the accelerator **1412**. The accelerator **1412** and/or external accelerator **1419** may have functionality similar to that of the accelerator device(s) **130** of FIG. **1**.

[0224] A display device **1411** may be provided that can connect to the processor(s) **1402**. The display device **1411** can be one or more of an internal display device, as in a mobile electronic device or a laptop device or an external display device attached via a display interface (e.g., DisplayPort, etc.). The display device **1411** can be a head mounted display (HMD) such as a stereoscopic display device for use in VR or AR applications.

[0225] The platform controller hub **1430** may enable peripherals to connect to memory device **1420** and processor(s) **1402** via a high-speed I/O bus. The I/O peripherals include, but are not limited to, an audio controller **1446**, a network controller **1434**, a firmware interface **1428**, a wireless transceiver **1426**, touch sensors **1425**, a data storage device **1424** (e.g., non-volatile memory, volatile memory, hard disk drive, flash memory, NAND, 3D NAND, 3D XPoint/Optane, etc.). The data storage device **1424** can connect via a storage interface (e.g., SATA) or via a peripheral bus, such as a Peripheral Component Interconnect bus (e.g., PCI, PCI express). The touch sensors **1425** can include touch screen sensors, pressure sensors, or fingerprint sensors. The wireless transceiver **1426** can be a Wi-Fi transceiver, a Bluetooth transceiver, or a mobile network transceiver such as a 3G, 4G, 5G, or Long-Term Evolution (LTE) transceiver. The firmware interface **1428** enables communication with system firmware, and can be, for example, a unified extensible firmware interface (UEFI). The network controller **1434** can enable a network connection to a wired network. In some embodiments, a high-performance network controller (not shown) couples with the interface bus(es) **1410**. The audio controller **1446** may be a multi-channel high-definition audio controller. In some of these embodiments the processing system **1400** includes an optional legacy I/O controller **1440** for coupling legacy (e.g., Personal System 2 (PS/2)) devices to the system. The platform controller hub **1430** can also connect to one or more Universal Serial Bus (USB) controllers **1442** to connect to input devices, such as keyboard and mouse **1443** combinations, a camera **1444**, or other USB input devices.

[0226] It will be appreciated that the processing system **1400** shown is example and not limiting, as other types of data processing systems that are differently configured may also be used. For example, an instance of the memory controller **1416** and platform controller hub **1430** may be integrated into a discrete external graphics processor, such as the external graphics processor **1418**.

The platform controller hub **1430** and/or memory controller **1416** may be external to the processor(s) **1402**. For example, the memory controller **1416** and platform controller hub **1430** may be external to the processing system **1400** and configured as a memory controller hub and peripheral controller hub within a system chipset that is in communication with the processor(s) **1402**.

[0227] For example, circuit boards (“sleds”) can be used on which components such as CPUs, memory, and other components are placed and are designed for increased thermal performance. Processing components such as the processors may be located on a top side of a sled while near memory, such as DIMMs, are located on a bottom side of the sled. As a result of the enhanced airflow provided by this design, the components may operate at higher frequencies and power levels than in typical systems, thereby increasing performance. Furthermore, the sleds are configured to blindly mate with power and data communication cables in a rack, thereby enhancing their ability to be quickly removed, upgraded, reinstalled, and/or replaced. Similarly, individual components located on the sleds, such as processors, accelerators, memory, and data storage drives, are configured to be easily upgraded due to their increased spacing from each other. In the illustrative embodiment, the components additionally include hardware attestation features to prove their authenticity.

[0228] A data center can utilize a single network architecture (“fabric”) that supports multiple other network architectures including Ethernet and Omni-Path. The sleds can be coupled to switches via optical fibers, which provide higher bandwidth and lower latency than typical twisted pair cabling (e.g., Category 5, Category 5e, Category 6, Category 7, Category 8, etc.). Due to the high bandwidth, low latency interconnections and network architecture, the data center may, in use, pool resources, such as memory, accelerators (e.g., GPUs, graphics accelerators, FPGAs, ASICs, neural network and/or artificial intelligence accelerators, etc.), and data storage drives that are physically disaggregated, and provide them to compute resources (e.g., processors), enabling the compute resources to access the pooled resources as if they were local.

[0229] A power supply or source can provide voltage and/or current to the processing system **1400** or any component or system described herein. In one example, the power supply includes an AC to DC (alternating current to direct current) adapter to plug into a wall outlet. Such AC power can be renewable energy (e.g., solar power) power source. In one example, the power source includes a DC power source, such as an external AC to DC converter. A power source or power supply may also include wireless charging hardware to charge via proximity to a charging field. The power source can include an internal battery, alternating current supply, motion-based power supply, solar power supply, or fuel cell source.

[0230] FIG. **15**A-**15**C illustrate computing systems and graphics processors. The elements of FIG. **15**A-**15**C having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such.

[0231] FIG. **15**A is a block diagram of a processor **1500**, which may be a variant of one of the processor(s) **1402** and may be used in place of one of those. Therefore, the disclosure of any features in combination with the processor **1500** herein also discloses a corresponding combination with the processor(s) **1402** but is not limited to such. The processor **1500** may have one or more processor cores **1502**A-**1502**N, at least one memory controller **1514**, and a graphics processor **1508**. The graphics processor **1508** may be integrated within the processor **1500**, within a system chipset, or coupled via a system bus. The processor **1500** can include additional cores up to and including additional core **1502**N represented by the dashed lined boxes. Each of processor cores **1502**A-**1502**N includes one or more internal cache unit(s) **1504**A-**1504**N. In some embodiments each processor core **1502**A-**1502**N also has access to one or more shared cache unit(s) **1506**. The internal cache units **1504**A-**1504**N and shared cache unit(s) **1506** represent a cache memory

hierarchy within the processor **1500**. The cache memory hierarchy may include at least one level of instruction and data cache within each processor core and one or more levels of shared mid-level cache, such as a Level 2 (L2), Level 3 (L3), Level 4 (L4), or other levels of cache, where the highest level of cache before external memory is classified as the LLC. In some embodiments, cache coherency logic maintains coherency between the various cache units (e.g., shared cache unit(s) **1506** and internal cache unit(s) **1504**A-**1504**N).

[0232] The processor **1500** may also include one or more bus controller units **1516** and a system agent core **1510**. The one or more bus controller units **1516** manage a set of peripheral buses, such as one or more PCI or PCI express busses. The one or more bus controller units **1516** can also manage one or more memory buses to various external memory devices (not shown). The system agent core **1510** provides management functionality for the various processor components and can include the at least one memory controller **1514**.

[0233] For example, one or more of the processor cores **1502**A-**1502**N may include support for simultaneous multi-threading. The system agent core **1510** includes components for coordinating and operating cores **1502**A-**1502**N during multi-threaded processing. System agent core **1510** may additionally include a power control unit (PCU), which includes logic and components to regulate the power state of processor cores **1502**A-**1502**N and a graphics processor **1508**.

[0234] The processor **1500** may additionally include a graphics processor **1508** to execute graphics processing operations. In some of these embodiments, the graphics processor **1508** couples with the one or more shared cache unit(s) **1506**, and the system agent core **1510**, including the at least one memory controller **1514**. The system agent core **1510** may also include a display controller **1511** to drive graphics processor output to one or more coupled displays. The display controller **1511** may also be a separate module coupled with the graphics processor via at least one interconnect or may be integrated within the graphics processor **1508**.

[0235] A ring or mesh based interconnect **1512** may be used to couple the internal components of the processor **1500**. However, an alternative interconnect unit may be used, such as a point-to-point interconnect, a switched interconnect, or other techniques, including techniques well known in the art. In some of these embodiments with a ring or mesh based interconnect **1512**, the graphics processor **1508** couples with the ring or mesh based interconnect **1512** via an I/O link **1513**.

[0236] The example I/O link **1513** represents at least one of multiple varieties of I/O interconnects, including an on package I/O interconnect which facilitates communication between various processor components and a high-performance memory module **1518**, such as an embedded DRAM module (eDRAM) or a high-bandwidth memory (HBM) module. Optionally, each of the processor cores **1502**A-**1502**N and the graphics processor **1508** can use the high-performance memory module **1518** as unified memory and/or a shared Last Level Cache when a DRAM memory system is also present. Optionally, one or more accelerators **1515** may also be included within the processor **1500**, including, for example, an NPU to accelerate certain neural network operations. The NPU can enable lower power inference operations relative to use of the graphics processor **1508**, or can operate in concert with the graphics processor **1508** to enable higher inference performance relative to the graphics processor **1508** alone. In one embodiment, an NPU within the one or more accelerators **1515** can include matrix or tensor acceleration logic and can be used to implement at least some of the computational operations that are described herein as implementable via the graphics processor **1508**.

[0237] The processor cores **1502**A-**1502**N may, for example, be homogenous cores executing the same instruction set architecture. Alternatively, the processor cores **1502**A-**1502**N are heterogeneous in terms of instruction set architecture (ISA), where one or more of processor cores **1502**A-**1502**N execute a first instruction set, while at least one of the other cores executes a subset of the first instruction set or a different instruction set. The processor cores **1502**A-**1502**N may be heterogeneous in terms of microarchitecture, where one or more cores having a relatively higher power consumption couple with one or more power cores having a lower power consumption. As

another example, the processor cores **1502**A-**1502**N are heterogeneous in terms of computational capability. Additionally, the processor **1500** can be implemented on one or more chips or chiplets, or as an SoC integrated circuit having the illustrated components, in addition to other components. The SoC integrated circuit may be implemented using multiple chiplets.

[0238] FIG. **15**B is a block diagram of hardware logic of a graphics processor core block **1519**, according to some embodiments described herein. In some embodiments, elements of FIG. **15**B having the same reference numbers (or names) as the elements of any other figure herein may operate or function in a manner similar to that described elsewhere herein. In one embodiment, the graphics processor core block **1519** is example of one partition of a graphics processor. The graphics processor core block **1519** can be included within the graphics processor **1508** of FIG. **15**A or a discrete graphics processor, parallel processor, and/or compute accelerator. A graphics processor as described herein may include multiple graphics core blocks based on target power and performance envelopes. Each graphics processor core block **1519** can include a function block **1530** coupled with multiple graphics cores **1521**A-**1521**F that include modular blocks of fixed function logic and general-purpose programmable logic. The graphics processor core block **1519** also includes shared/cache memory **1536** that is accessible by all graphics cores **1521**A-**1521**F, rasterizer logic **1537**, and additional fixed function logic **1538**.

[0239] In some embodiments, the function block **1530** includes a geometry/fixed function pipeline **1531** that can be shared by all graphics cores in the graphics processor core block **1519**. In various embodiments, the geometry/fixed function pipeline **1531** includes a 3D geometry pipeline a video front-end unit, a thread spawner and global thread dispatcher, and a unified return buffer manager, which manages unified return buffers. In one embodiment the function block **1530** also includes a graphics SoC interface **1532**, a graphics microcontroller **1533**, and a media pipeline **1534**. The graphics SoC interface **1532** provides an interface between the graphics processor core block **1519** and other core blocks within a graphics processor or compute accelerator SoC. The graphics microcontroller **1533** is a programmable sub-processor that is configurable to manage various functions of the graphics processor core block **1519**, including thread dispatch, scheduling, and pre-emption. The media pipeline **1534** includes logic to facilitate the decoding, encoding, pre-processing, and/or post-processing of multimedia data, including image and video data. The media pipeline **1534** implement media operations via requests to compute or sampling logic within the graphics cores **1521**-**1521**F. One or more pixel backends **1535** can also be included within the function block **1530**. The one or more pixel backends **1535** include a cache memory to store pixel color values and can perform blend operations and lossless color compression of rendered pixel data.

[0240] In one embodiment the graphics SoC interface **1532** enables the graphics processor core block **1519** to communicate with general-purpose application processor cores (e.g., CPUs) and/or other components within an SoC or a system host CPU that is coupled with the SoC via a peripheral interface. The graphics SoC interface **1532** also enables communication with off-chip memory hierarchy elements such as a shared last level cache memory, system RAM, and/or embedded on-chip or on-package DRAM. The graphics SoC interface **1532** can also enable communication with fixed function devices within the SoC, such as camera imaging pipelines, and enables the use of and/or implements global memory atomics that may be shared between the graphics processor core block **1519** and CPUs within the SoC. The graphics SoC interface **1532** can also implement power management controls for the graphics processor core block **1519** and enable an interface between a clock domain of the graphics processor core block **1519** and other clock domains within the SoC. In one embodiment the graphics SoC interface **1532** enables receipt of command buffers from a command streamer and global thread dispatcher that are configured to provide commands and instructions to each of one or more graphics cores within a graphics processor. The commands and instructions can be dispatched to the media pipeline **1534** when media operations are to be performed, the geometry and fixed function pipeline **1531** when

graphics processing operations are to be performed. When compute operations are to be performed, compute dispatch logic can dispatch the commands to the graphics cores **1521**A-**1521**F, bypassing the geometry and media pipelines.

[0241] The graphics microcontroller **1533** can be configured to perform various scheduling and management tasks for the graphics processor core block **1519**. In one embodiment the graphics microcontroller **1533** can perform graphics and/or compute workload scheduling on the various vector engines **1522**A-**1522**F, **1524**A-**1524**F and matrix engines **1523**A-**1523**F, **1525**A-**1525**F within the graphics cores **1521**A-**1521**F. In this scheduling model, host software executing on a CPU core of an SoC including the graphics processor core block **1519** can submit workloads to one of multiple graphics processor doorbells, which invokes a scheduling operation on the appropriate graphics engine. Scheduling operations include determining which workload to run next, submitting a workload to a command streamer, pre-empting existing workloads running on an engine, monitoring progress of a workload, and notifying host software when a workload is complete. In one embodiment the graphics microcontroller **1533** can also facilitate low-power or idle states for the graphics processor core block **1519**, providing the graphics processor core block **1519** with the ability to save and restore registers within the graphics processor core block **1519** across low-power state transitions independently from the operating system and/or graphics driver software on the system.

[0242] The graphics processor core block **1519** may have greater than or fewer than the illustrated graphics cores **1521**A-**1521**F, up to N modular graphics cores. For each set of N graphics cores, the graphics processor core block **1519** can also include shared/cache memory **1536**, which can be configured as shared memory or cache memory, rasterizer logic **1537**, and additional fixed function logic **1538** to accelerate various graphics and compute processing operations.

[0243] Within each graphics cores **1521**A-**1521**F is set of execution resources that may be used to perform graphics, media, and compute operations in response to requests by graphics pipeline, media pipeline, or shader programs. The graphics cores **1521**A-**1521**F include multiple vector engines **1522**A-**1522**F, **1524**A-**1524**F, matrix acceleration units **1523**A-**1523**F, **1525**A-**1525**D, cache/shared local memory (SLM), a sampler **1526**A-**1526**F, and a ray tracing unit **1527**A-**1527**F.

[0244] The vector engines **1522**A-**1522**F, **1524**A-**1524**F are general-purpose graphics processing units capable of performing floating-point and integer/fixed-point logic operations in service of a graphics, media, or compute operation, including graphics, media, or compute/GPGPU programs. The vector engines **1522**A-**1522**F, **1524**A-**1524**F can operate at variable vector widths using SIMD, SIMT, or SIMT+SIMD execution modes. The matrix acceleration units **1523**A-**1523**F, **1525**A-**1525**D include matrix-matrix and matrix-vector acceleration logic that improves performance on matrix operations, particularly low and mixed precision (e.g., INT8, FP16, BF16, FP8, FP4) matrix operations used for machine learning. In one embodiment, the matrix acceleration units **1523**A-**1523**F, **1525**A-**1525**D have support for microscaling (MX) formats. In one embodiment, each of the matrix acceleration units **1523**A-**1523**F, **1525**A-**1525**D includes one or more systolic arrays of processing elements that can perform concurrent matrix multiply or dot product operations on matrix elements.

[0245] The sampler **1526**A-**1526**F can read media or texture data into memory and can sample data differently based on a configured sampler state and the texture/media format that is being read. Threads executing on the vector engines **1522**A-**1522**F, **1524**A-**1524**F or matrix acceleration units **1523**A-**1523**F, **1525**A-**1525**D can make use of the cache/SLM **1528**A-**1528**F within each of the graphics cores **1521**A-**1521**F. The cache/SLM **1528**A-**1528**F can be configured as cache memory or as a pool of shared memory that is local to each of the respective graphics cores **1521**A-**1521**F. The ray tracing units **1527**A-**1527**F within the graphics cores **1521**A-**1521**F include ray traversal/intersection circuitry for performing ray traversal using bounding volume hierarchies (BVHs) and identifying intersections between rays and primitives enclosed within the BVH volumes. In one embodiment the ray tracing units **1527**A-**1527**F include circuitry for performing

depth testing and culling (e.g., using a depth buffer or similar arrangement). In one implementation, the ray tracing units **1527**A-**1527**F perform traversal and intersection operations in concert with image denoising, at least a portion of which may be performed using an associated matrix acceleration unit **1523**A-**1523**F, **1525**A-**1525**D.

[0246] FIG. **15**C is a block diagram of general-purpose graphics processing unit (GPGPU **1570**) that can be configured as a graphics processor, e.g., the graphics processor **1508**, and/or compute accelerator, according to embodiments described herein. The GPGPU **1570** can interconnect with host processors (e.g., one or more CPU(s) **1546**) and memory **1571**, **1572** via one or more system and/or memory busses. Memory **1571** may be system memory that can be shared with the one or more CPU(s) **1546**, while memory **1572** is device memory that is dedicated to the GPGPU **1570**. For example, components within the GPGPU **1570** and memory **1572** may be mapped into memory addresses that are accessible to the one or more CPU(s) **1546**. Access to memory **1571** and **1572** may be facilitated via a memory controller **1568**. The memory controller **1568** may include an internal direct memory access (DMA) controller **1569** or can include logic to perform operations that would otherwise be performed by a DMA controller. In one embodiment, at least one of the one or more CPU(s) **1546** can include one or more accelerators **1545**, including but not limited to neural network accelerators.

[0247] The GPGPU **1570** includes multiple global cache memories, including an L2 cache **1553**, L1 cache **1554**, an instruction cache **1555**, and shared memory **1556**, at least a portion of which may also be partitioned as a cache memory. The GPGPU **1570** also includes multiple compute units **1560**A-**1560**N. Each compute unit **1560**A-**1560**N includes a set of vector registers **1561**, scalar registers **1562**, vector logic units **1563**, scalar logic units **1564**, and a scheduler **1584**. The compute units **1560**A-**1560**N can also include local shared memory **1565** and local cache memory **1566**. The compute units **1560**A-**1560**N can couple with a constant cache **1567**, which can be used to store constant data, which is data that will not change during the run of kernel or shader program that executes on the GPGPU **1570**. The constant cache **1567** may be a scalar data cache and cached data can be fetched directly into the scalar registers **1562**. In one embodiment, the compute unit **1560**A-**1560**N additionally include at least one matrix unit **1580** to accelerate matrix, tensor, or artificial intelligence operations and at least one ray tracing unit (RT unit **1582**) to accelerate ray tracing operations. The at least one matrix unit **1580** and RT unit **1582** can include similar functionality as other matrix/tensor accelerators and ray tracing accelerators described herein.

[0248] During operation, the one or more CPU(s) **1546** can write commands into registers or memory in the GPGPU **1570** that has been mapped into an accessible address space. The command processors **1557** can read the commands from registers or memory and determine how those commands will be processed within the GPGPU **1570**. A thread dispatcher **1558** can then be used to dispatch threads to the compute units **1560**A-**1560**N to perform those commands. Each compute unit **1560**A-**1560**N can execute threads independently of the other compute units. Additionally, each compute unit **1560**A-**1560**N can be independently configured for conditional computation and can conditionally output the results of computation to memory. The command processors **1557** can interrupt the one or more CPU(s) **1546** when the submitted commands are complete.

[0249] FIG. **16** is a block diagram of a graphics processor **1600**, which may be a discrete graphics processing unit, or may be a graphics processor integrated with a plurality of processing cores, or other semiconductor devices such as, but not limited to, memory devices or network interfaces. The elements of the graphics processor **1600** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. For example, the graphics processor **1600** may be a variant of the graphics processor **1508** and may be used in place of the graphics processor **1508**. The graphics processor may communicate via a memory mapped I/O interface to registers on the graphics processor and with commands placed into the processor memory. The

graphics processor **1600** may include a memory interface **1614** to access local memory, one or more internal caches, one or more shared external caches, and/or system memory.
[0250] The graphics processor **1600** can include a display controller **1602** to drive display output data to a display device **1618**. Display controller **1602** includes hardware for one or more overlay planes for the display and composition of multiple layers of video or user interface elements. The display device **1618** can be an internal or external display device. In one embodiment the display device **1618** is a head mounted display device, such as a VR or AR display device. Graphics processor **1600** may include a video codec engine **1606** to encode, decode, or transcode media to, from, or between one or more media encoding formats, including, but not limited to Moving Picture Experts Group (MPEG) formats such as MPEG-2, Advanced Video Coding (AVC) formats such as H.264/MPEG-4 AVC, H.265/HEVC, Alliance for Open Media (AOMedia) VP8, VP9, as well as the Society of Motion Picture & Television Engineers (SMPTE) 421M/VC-1, and Joint Photographic Experts Group (JPEG) formats such as JPEG, and Motion JPEG (MJPEG) formats.
[0251] The graphics processor **1600** may include a block image transfer (BLIT) engine **1603** to perform two-dimensional (2D) rasterizer operations including, for example, bit-boundary block transfers, or 2D graphics operations may be performed using one or more components of graphics processing engine (GPE **1610**). GPE **1610** may include a 3D pipeline **1612** for performing 3D operations, such as rendering three-dimensional images and scenes using processing functions that act upon 3D primitive shapes (e.g., rectangle, triangle, etc.). The 3D pipeline **1612** includes programmable and fixed function elements that perform various tasks within the element and/or spawn execution threads to a 3D/Media subsystem **1615**. While 3D pipeline **1612** can be used to perform media operations, the GPE **1610** may also include a media pipeline **1616** that is specifically used to perform media operations, such as image or video decode, encode, post-processing, and enhancement. The media pipeline **1616** may include fixed function or programmable logic units to perform one or more specialized media operations, such as video decode acceleration, video de-interlacing, and video encode acceleration in place of, or on behalf of video codec engine **1606**. Media pipeline **1616** may additionally include a thread spawning unit to spawn threads for execution on 3D/Media subsystem **1615**. The spawned threads perform computations for the media operations on one or more graphics execution units included in 3D/Media subsystem **1615**.
[0252] FIG. **17**A illustrates a graphics processor **1720**, being a variant of the graphics processor **1600** of FIG. **16** and may be used in place of the graphics processor **1600** and vice versa. Therefore, the disclosure of any features in combination with the graphics processor **1600** herein also discloses a corresponding combination with the graphics processor **1720** but is not limited to such. The graphics processor **1720** has a tiled architecture, according to embodiments described herein. The graphics processor **1720** may include a graphics processing engine cluster **1722** having multiple graphics processing engines within a plurality of graphics engine tiles. Each graphics engine tile **1710**A-**1710**D can be interconnected via a set of tile interconnects **1723**A-**1723**F. Each graphics engine tile **1710**A-**1710**D can also be connected to a memory module or memory devices **1726**A-**1726**D via memory interconnects **1725**A-**1725**D. The memory devices **1726**A-**1726**D can use any graphics memory technology. For example, the memory devices **1726**A-**1726**D may be graphics double data rate (GDDR) memory. The memory devices **1726**A-**1726**D may be high-bandwidth memory (HBM) modules that can be on-die with their respective graphics engine tile **1710**A-**1710**D. The memory devices **1726**A-**1726**D may be stacked memory devices that can be stacked on top of their respective graphics engine tile **1710**A-**1710**D. Each graphics engine tile **1710**A-**1710**D and associated memory devices **1726**A-**1726**D may reside on separate chiplets, which are bonded to a base die or base substrate, as described in further detail in FIG. **25**A-**25**B.
[0253] The graphics processor **1720** may be configured with a non-uniform memory access (NUMA) system in which memory devices **1726**A-**1726**D are coupled with associated graphics engine tiles **1710**A-**1710**D. A given memory device may be accessed by graphics engine tiles other

than the tile to which it is directly connected. However, access latency to the memory devices **1726**A-**1726**D may be lowest when accessing a local tile. In one embodiment, a cache coherent NUMA (ccNUMA) system is enabled that uses the tile interconnects **1723**A-**1723**F to enable communication between cache controllers within the graphics engine tiles **1710**A-**1710**D to keep a consistent memory image when more than one cache stores the same memory location.
[0254] The graphics processing engine cluster **1722** can connect with an interconnect fabric **1724**, which may interconnect with an on-chip or on-package fabric. In one embodiment the interconnect fabric **1724** includes a network processor, network on a chip (NoC), or another switching processor to enable the interconnect fabric **1724** to act as a packet switched interconnect fabric that switches data packets between components of the graphics processor **1720**. The interconnect fabric **1724** can enable communication between graphics engine tiles **1710**A-**1710**D and components such as the video codec engine **1706** and one or more copy engines **1704**. The one or more copy engines **1704** can be used to move data out of, into, and between the memory devices **1726**A-**1726**D and memory that is external to the graphics processor **1720** (e.g., system memory). The interconnect fabric **1724** can also be used to interconnect the graphics engine tiles **1710**A-**1710**D. The graphics processor **1720** may optionally include a display controller **1702** to enable a connection with a display device **1718**. The graphics processor may also be configured as a graphics or compute accelerator. In the accelerator configuration, the display controller **1702** and display device **1718** may be omitted.
[0255] The graphics processor **1720** can connect to a host system via a host interface **1728**. The host interface **1728** can enable communication between the graphics processor **1720**, system memory, and/or other system components. The host interface **1728** can be, for example, a PCI express bus or another type of host system interface. For example, the host interface **1728** may be an NVLink or NVSwitch interface. The host interface **1728** and interconnect fabric **1724** can cooperate to enable multiple instances of the graphics processor **1720** to act as single logical device. Cooperation between the host interface **1728** and interconnect fabric **1724** can also enable the individual graphics engine tiles **1710**A-**1710**D to be presented to the host system as distinct logical graphics devices.
[0256] FIG. **17**B illustrates a compute accelerator **1730**, according to embodiments described herein. The compute accelerator **1730** can include architectural similarities with the graphics processor **1720** of FIG. **17**B and is optimized for compute acceleration. A compute engine cluster **1732** can include a set of compute engine tiles **1740**A-**1740**D that include execution logic that is optimized for parallel or vector-based general-purpose compute operations. In one embodiment, the compute accelerator **1730** may be configured as an AI accelerator or NPU. In such embodiment, the execution logic of the compute engine tiles **1740**A-**1740**D may be directed primarily towards matrix or tensor operations and include tensor cores or matrix engines described herein. The compute engine tiles **1740**A-**1740**D may not include fixed function graphics processing logic, although in some embodiments one or more of the compute engine tiles **1740**A-**1740**D can include logic to perform media acceleration. The compute engine tiles **1740**A-**1740**D can connect to the memory devices **1726**A-**1726**D via memory interconnects **1725**A-**1725**D. The memory devices **1726**A-**1726**D and memory interconnects **1725**A-**1725**D may be similar technology as in graphics processor **1720** or can be different. The compute engine tiles **1740**A-**1740**D can also be interconnected via a set of tile interconnects **1723**A-**1723**F and may be connected with and/or interconnected by the interconnect fabric **1724**. In one embodiment the compute accelerator **1730** includes a large L3 cache **1736** that can be configured as a device-wide cache. The compute accelerator **1730** can also connect to a host processor and memory via a host interface **1728** in a similar manner as the graphics processor **1720** of FIG. **17**B.
[0257] The compute accelerator **1730** can also include an integrated network interface **1742**. In one embodiment the integrated network interface **1742** includes a network processor and controller logic that enables the compute engine cluster **1732** to communicate over a physical layer interconnect **1744** without having data traverse memory of a host system. In one embodiment, one

of the compute engine tiles **1740**A-**1740**D is replaced by network processor logic and data to be transmitted or received via the physical layer interconnect **1744** may be transmitted directly to or from the memory devices **1726**A-**1726**D. Multiple instances of the compute accelerator **1730** may be joined via the physical layer interconnect **1744** into a single logical device. Alternatively, the various compute engine tiles **1740**A-**1740**D may be presented as distinct network accessible compute accelerator devices.

Graphics Processing Resources

[0258] FIG. **18**A-**18**C illustrate execution logic including an array of processing elements employed in a graphics processor, according to embodiments described herein. FIG. **18**A illustrates graphics core cluster, according to an embodiment. FIG. **18**B illustrates a vector engine of a graphics core, according to an embodiment. FIG. **18**C illustrates a matrix engine of a graphics core, according to an embodiment. Elements of FIG. **18**A-**18**C having the same reference numbers as the elements of any other figure herein may operate or function in any manner similar to that described elsewhere herein but are not limited as such. For example, the elements of FIG. **18**A-**18**C can be considered in the context of the graphics processor core block **1519** of FIG. **15**B. In one embodiment, the elements of FIG. **18**A-**18**C have similar functionality to equivalent components of the graphics processor **1508** of FIG. **15**A or the GPGPU **1570** of FIG. **15**C.

[0259] As shown in FIG. **18**A, in one embodiment the graphics core cluster **1800** includes a graphics processor core block **1519**, which can include any number of graphics cores (e.g., graphics core **1815**A, graphics core **1815**B, through graphics core **1815**N). Multiple instances of the graphics processor core block **1519** may be included. In one embodiment the elements of the graphics cores **1815**A-**1815**N have similar or equivalent functionality as the elements of the graphics cores **1521**A-**1521**F of FIG. **15**B. In such embodiment, the graphics cores **1815**A-**1815**N each include circuitry including but not limited to vector engines **1802**A-**1802**N, matrix engines **1803**A-**1803**N, memory load/store units **1804**A-**1804**N, instruction caches **1805**A-**1805**N, data caches/shared local memory **1806**A-**1806**N, ray tracing units **1808**A-**1808**N, samplers **1810**A-**1810**N. The circuitry of the graphics cores **1815**A-**1815**N can additionally include fixed function logic **1812**A-**1812**N. The number of vector engines **1802**A-**1802**N and matrix engines **1803**A-**1803**N within the graphics cores **1815**A-**1815**N of a design can vary based on the workload, performance, and power targets for the design.

[0260] With reference to graphics core **1815**A, the vector engine **1802**A and matrix engine **1803**A are configurable to perform parallel compute operations on data in a variety of integer and floating-point data formats based on instructions associated with shader programs. Each vector engine **1802**A and matrix engine **1803**A can act as a programmable general-purpose computational unit that is capable of executing multiple simultaneous hardware threads while processing multiple data elements in parallel for each thread. The vector engine **1802**A and matrix engine **1803**A support the processing of variable width vectors at various SIMD widths, including but not limited to SIMD8, SIMD16, and SIMD32. Input data elements can be stored as a packed data type in a register and the vector engine **1802**A and matrix engine **1803**A can process the various elements based on the data size of the elements. For example, when operating on a 256-bit wide vector, the bits of the vector are stored in a register and the vector is processed as four separate 64-bit packed data elements (Quad-Word (QW) size data elements), eight separate 32-bit packed data elements (Double Word (DW) size data elements), sixteen separate 16-bit packed data elements (Word (W) size data elements), or thirty-two separate 8-bit data elements (byte (B) size data elements). However, different vector widths and register sizes are possible. In one embodiment, the vector engine **1802**A and matrix engine **1803**A are also configurable for SIMT operation on warps or thread groups of various sizes (e.g., 8, 16, or 32 threads).

[0261] Continuing with graphics core **1815**A, the memory load/store unit **1804**A services memory access requests that are issued by the vector engine **1802**A, matrix engine **1803**A, and/or other components of the graphics core **1815**A that have access to memory. The memory access request

can be processed by the memory load/store unit **1804**A to load or store the requested data to or from cache or memory into a register file associated with the vector engine **1802**A and/or matrix engine **1803**A. The memory load/store unit **1804**A can also perform prefetching operations. With additional reference to FIG. **19**, in one embodiment, the memory load/store unit **1804**A is configured to provide SIMT scatter/gather prefetching or block prefetching for data stored in memory **1910**, from memory that is local to other tiles via the tile interconnect **1908**, or from system memory. Prefetching can be performed to a specific L1 cache (e.g., data cache/shared local memory **1806**A), the L2 cache **1904** or the L3 cache **1906**. In one embodiment, a prefetch to the L3 cache **1906** automatically results in the data being stored in the L2 cache **1904**.

[0262] The instruction cache **1805**A stores instructions to be executed by the graphics core **1815**A. In one embodiment, the graphics core **1815**A also includes instruction fetch and prefetch circuitry that fetches or prefetches instructions into the instruction cache **1805**A. The graphics core **1815**A also includes instruction decode logic to decode instructions within the instruction cache **1805**A. The data cache/shared local memory **1806**A can be configured as a data cache that is managed by a cache controller that implements a cache replacement policy and/or configured as explicitly managed shared memory. The ray tracing unit **1808**A includes circuitry to accelerate ray tracing operations. The sampler **1810**A provides texture sampling for 3D operations and media sampling for media operations. The fixed function logic **1812**A includes fixed function circuitry that is shared between the various instances of the vector engine **1802**A and matrix engine **1803**A. Graphics cores **1815**B-**1815**N can operate in a similar manner as graphics core **1815**A.

[0263] Functionality of the instruction caches **1805**A-**1805**N, data caches/shared local memory **1806**A-**1806**N, ray tracing units **1808**A-**1808**N, samplers **1810**A-**1810**N, and fixed function logic **1812**A-**1812**N corresponds with equivalent functionality in the graphics processor architectures described herein. For example, the instruction caches **1805**A-**1805**N can operate in a similar manner as instruction cache **1555** of FIG. **15**C. The data caches/shared local memory **1806**A-**1806**N, ray tracing units **1808**A-**1808**N, and samplers **1810**A-**1810**N can operate in a similar manner as the cache/SLM **1528**A-**1528**F, ray tracing units **1527**A-**1527**F, and samplers **1526**A-**1526**F of FIG. **15**B. The fixed function logic **1812**A-**1812**N can include elements of the geometry/fixed function pipeline **1531** and/or additional fixed function logic **1538** of FIG. **15**B. In one embodiment, the ray tracing units **1808**A-**1808**N include circuitry to perform ray tracing acceleration operations performed by the ray tracing cores **372** of FIG. **3**.

[0264] As shown in FIG. **18**B, in one embodiment the vector engine **1802** includes an instruction fetch unit **1837**, a general register file array (GRF **1824**), an architectural register file array (ARF **1826**), a thread arbiter **1822**, a send unit **1830**, a branch unit **1832**, SIMD FPUs **1834**, and in one embodiment, SIMD ALUs **1835**. The GRF **1824** and ARF **1826** includes the set of general register files and architecture register files associated with each hardware thread that may be active in the vector engine **1802**. In one embodiment, per thread architectural state is maintained in the ARF **1826**, while data used during thread execution is stored in the GRF **1824**. The execution state of each thread, including the instruction pointers for each thread, can be held in thread-specific registers in the ARF **1826**. Register renaming may be used to dynamically allocate registers to hardware threads.

[0265] In one embodiment the vector engine **1802** has an architecture that is a combination of Simultaneous Multi-Threading (SMT) and fine-grained Interleaved Multi-Threading (IMT). The architecture has a modular configuration that can be fine-tuned at design time based on a target number of simultaneous threads and number of registers per graphics core, where graphics core resources are divided across logic used to execute multiple simultaneous threads. The number of logical threads that may be executed by the vector engine **1802** is not limited to the number of hardware threads, and multiple logical threads can be assigned to each hardware thread.

[0266] In one embodiment, the vector engine **1802** can co-issue multiple instructions, which may each be different instructions. The thread arbiter **1822** can dispatch the instructions to one of the

send unit **1830**, branch unit **1832**, or SIMD FPUs **1834** for execution. Each execution thread can access **128** general-purpose registers within the GRF **1824**, where each register can store 32 bytes, accessible as a variable width vector of 32-bit data elements. In one embodiment, each thread has access to 4 Kbytes within the GRF **1824**, although embodiments are not so limited, and greater or fewer register resources may be provided in other embodiments. In one embodiment the vector engine **1802** is partitioned into seven hardware threads that can independently perform computational operations, although the number of threads per vector engine **1802** can also vary according to embodiments. For example, in one embodiment up to 16 hardware threads are supported. In an embodiment in which seven threads may access 4 Kbytes, the GRF **1824** can store a total of 28 Kbytes. Where 16 threads may access 4 Kbytes, the GRF **1824** can store a total of 64 Kbytes. Flexible addressing modes can permit registers to be addressed together to build effectively wider registers or to represent strided rectangular block data structures.

[0267] In one embodiment, memory operations, sampler operations, and other longer-latency system communications are dispatched via "send" instructions that are executed by the message passing send unit **1830**. In one embodiment, branch instructions are dispatched to the branch unit **1832** to facilitate SIMD divergence and eventual convergence.

[0268] In one embodiment the SIMD FPUs **1834** of the vector engine **1802** perform floating-point operations. In one embodiment, the SIMD FPUs **1834** also support integer computation. In one embodiment the SIMD FPUs **1834** can execute up to M number of 32-bit floating-point (or integer) operations, or execute up to 2M 16-bit integer or 16-bit floating-point operations. In one embodiment, at least one of the FPUs provides extended math capability to support high-throughput transcendental math functions and double precision 64-bit floating-point. In some embodiments, SIMD ALUs **1835** configured to perform 8-bit integer operations are also present and may be specifically optimized to perform operations associated with machine learning computations. In one embodiment, the SIMD ALUs **1835** are replaced by SIMD FPUs **1834** that are configurable to perform integer and floating-point operations. In one embodiment, the SIMD FPUs **1834** and SIMD ALUs **1835** are configurable to execute SIMT programs. In one embodiment, combined SIMD+SIMT operation is supported.

[0269] In one embodiment, arrays of multiple instances of the vector engine **1802** can be instantiated in a graphics core. For scalability, product architects can choose the exact number of vector engines per graphics core grouping. In one embodiment the vector engine **1802** can execute instructions across a plurality of execution channels. In a further embodiment, each thread executed on the vector engine **1802** is executed on a different channel.

[0270] As shown in FIG. **18**C, in one embodiment the matrix engine **1803** includes an array of processing elements that are configured to perform tensor operations including vector/matrix and matrix/matrix operations, such as but not limited to matrix multiply and/or dot product operations. The matrix engine **1803** is configured with M rows and N columns of processing elements **1852**AA-**1852**MN that include multiplier and adder circuits organized in a pipelined fashion. In one embodiment, the processing elements **1852**AA-**1852**MN make up the physical pipeline stages of an N wide and M deep systolic array that can be used to perform vector/matrix or matrix/matrix operations in a data-parallel manner, including matrix multiply, fused multiply-add, dot product or other general matrix-matrix multiplication (GEMM) operations. In one embodiment the matrix engine **1803** supports 16-bit and 8-bit floating-point operations, as well as 8-bit, 4-bit, 2-bit, and binary integer operations. The matrix engine **1803** can also be configured to accelerate specific machine learning operations. In such embodiments, the matrix engine **1803** can be configured with support for the bfloat (brain floating-point) 16-bit floating-point format or a tensor float 32-bit floating-point format (TF32) that have different numbers of mantissa and exponent bits relative to Institute of Electrical and Electronics Engineers (IEEE) 754 formats.

[0271] In one embodiment, during each cycle, each stage can add the result of operations performed at that stage to the output of the previous stage. In other embodiments, the pattern of

data movement between the processing elements **1852**AA-**1852**MN after a set of computational cycles can vary based on the instruction or macro-operation being performed. For example, in one embodiment partial sum loopback is enabled and the processing elements may instead add the output of a current cycle with output generated in the previous cycle. In one embodiment, the final stage of the systolic array can be configured with a loopback to the initial stage of the systolic array. In such embodiment, the number of physical pipeline stages may be decoupled from the number of logical pipeline stages that are supported by the matrix engine **1803**. For example, where the processing elements **1852**AA-**1852**MN are configured as a systolic array of M physical stages, a loopback from stage M to the initial pipeline stage can enable the processing elements **1852**AA-**1852**MN to operate as a systolic array of, for example, 2M, 3M, 4M, etc., logical pipeline stages.
[0272] In one embodiment, the matrix engine **1803** includes memory **1841**A-**1841**N, **1842**A-**1842**M to store input data in the form of row and column data for input matrices. Memory **1842**A-**1842**M is configurable to store row elements (A0-Am) of a first input matrix and memory **1841**A-**1841**N is configurable to store column elements (B0-Bn) of a second input matrix. The row and column elements are provided as input to the processing elements **1852**AA-**1852**MN for processing. In one embodiment, row and column elements of the input matrices can be stored in a systolic register file **1840** within the matrix engine **1803** before those elements are provided to the memory **1841**A-**1841**N, **1842**A-**1842**M. In one embodiment, the systolic register file **1840** is excluded and the memory **1841**A-**1841**N, **1842**A-**1842**M is loaded from registers in an associated vector engine (e.g., GRF **1824** of vector engine **1802** of FIG. **18**B) or other memory of the graphics core that includes the matrix engine **1803** (e.g., data cache/shared local memory **1806**A for matrix engine **1803**A of FIG. **18**A). Results generated by the processing elements **1852**AA-**1852**MN are then output to an output buffer and/or written to a register file (e.g., systolic register file **1840**, GRF **1824**, data cache/shared local memory **1806**A-**1806**N) for further processing by other functional units of the graphics processor or for output to memory.
[0273] In some embodiments, the matrix engine **1803** is configured with support for input sparsity, where multiplication operations for sparse regions of input data can be bypassed by skipping multiply operations that have a zero-value operand. In one embodiment, the processing elements **1852**AA-**1852**MN are configured to skip the performance of certain operations that have zero value input. In one embodiment, sparsity within input matrices can be detected and operations having known zero output values can be bypassed before being submitted to the processing elements **1852**AA-**1852**MN. The loading of zero value operands into the processing elements can be bypassed and the processing elements **1852**AA-**1852**MN can be configured to perform multiplications on the non-zero value input elements. The matrix engine **1803** can also be configured with support for output sparsity, such that operations with results that are pre-determined to be zero are bypassed. For input sparsity and/or output sparsity, in one embodiment, metadata is provided to the processing elements **1852**AA-**1852**MN to indicate, for a processing cycle, which processing elements and/or data channels are to be active during that cycle.
[0274] In one embodiment, the matrix engine **1803** includes hardware to enable operations on sparse data having a compressed representation of a sparse matrix that stores non-zero values and metadata that identifies the positions of the non-zero values within the matrix. Example compressed representations include but are not limited to compressed tensor representations such as compressed sparse row (CSR), compressed sparse column (CSC), compressed sparse fiber (CSF) representations. Support for compressed representations enable operations to be performed on input in a compressed tensor format without the compressed representation having to be decompressed or decoded. In such embodiment, operations can be performed only on non-zero input values and the resulting non-zero output values can be mapped into an output matrix. In some embodiments, hardware support is also provided for machine-specific lossless data compression formats that are used when transmitting data within hardware or across system busses. Such data may be retained in a compressed format for sparse input data and the matrix engine **1803** can use

the compression metadata for the compressed data to enable operations to be performed on only non-zero values, or to enable blocks of zero data input to be bypassed for multiply operations.
[0275] In various embodiments, input data can be provided by a programmer in a compressed tensor representation, or a codec can compress input data into the compressed tensor representation or another sparse data encoding. In addition to support for compressed tensor representations, streaming compression of sparse input data can be performed before the data is provided to the processing elements **1852**AA-**1852**MN. In one embodiment, compression is performed on data written to a cache memory associated with the graphics core cluster **1800**, with the compression being performed with an encoding that is supported by the matrix engine **1803**. In one embodiment, the matrix engine **1803** includes support for input having structured sparsity in which a pre-determined level or pattern of sparsity is imposed on input data. This data may be compressed to a known compression ratio, with the compressed data being processed by the processing elements **1852**AA-**1852**MN according to metadata associated with the compressed data.
[0276] FIG. **19** illustrates a tile **1900** of a multi-tile processor, according to an embodiment. In one embodiment, the tile **1900** is representative of one of the graphics engine tiles **1710**A-**1710**D of FIG. **17**A or compute engine tiles **1740**A-**1740**D of FIG. **17**B. The tile **1900** of the multi-tile graphics processor includes an array of graphics core clusters (e.g., graphics core cluster **1800**A, graphics core cluster **1800**B, through graphics core cluster **1800**N), with each graphics core cluster having an array of graphics cores **515**A-**515**N. The tile **1900** also includes a global dispatcher **1902** to dispatch threads to processing resources of the tile **1900**.
[0277] The tile **1900** can include or couple with an L3 cache **1906** and memory **1910**. In various embodiments, the L3 cache **1906** may be excluded or the tile **1900** can include additional levels of cache, such as an L4 cache. In one embodiment, each instance of the tile **1900** in the multi-tile graphics processor is associated with memory **1910**, such as in FIG. **17**A and FIG. **17**B. In one embodiment, a multi-tile processor can be configured as a multi-chip module in which the L3 cache **1906** and/or memory **1910** reside on separate chiplets than the graphics core clusters **1800**A-**1800**N. In this context, a chiplet is an at least partially packaged integrated circuit that includes distinct units of logic that can be assembled with other chiplets into a larger package. For example, the L3 cache **1906** can be included in a dedicated cache chiplet or can reside on the same chiplet as the graphics core clusters **1800**A-**1800**N. In one embodiment, the L3 cache **1906** can be included in an active base die or active interposer.
[0278] A memory fabric **1903** enables communication among the graphics core clusters **1800**A-**1800**N, L3 cache **1906**, and memory **1910**. An L2 cache **1904** couples with the memory fabric **1903** and is configurable to cache transactions performed via the memory fabric **1903**. A tile interconnect **1908** enables communication with other tiles on the graphics processors and may be one of tile interconnects **1723**A-**1723**F of FIGS. **17**A and **17**B. In embodiments in which the L3 cache **1906** is excluded from the tile **1900**, the L2 cache **1904** may be configured as a combined L2/L3 cache. The memory fabric **1903** is configurable to route data to the L3 cache **1906** or memory controllers associated with the memory **1910** based on the presence or absence of the L3 cache **1906** in a specific implementation. The L3 cache **1906** can be configured as a per-tile cache that is dedicated to processing resources of the tile **1900** or may be a partition of a GPU-wide L3 cache.
[0279] FIG. **20** is a block diagram illustrating graphics processor instruction formats **2000**. The graphics processor execution units support an instruction set having instructions in multiple formats. The solid lined boxes illustrate the components that are generally included in an execution unit instruction, while the dashed lines include components that are optional or that are only included in a sub-set of the instructions. In some embodiments the graphics processor instruction formats **2000** described and illustrated are macro-instructions, in that they are instructions supplied to the execution unit, as opposed to micro-operations resulting from instruction decode once the instruction is processed. Thus, a single instruction may cause hardware to perform multiple micro-operations.

[0280] The graphics processor execution units as described herein may natively support instructions in a 128-bit instruction format **2010**. A compacted 64-bit instruction format **2030** is available for some instructions based on the selected instruction, instruction options, and number of operands. The native 128-bit instruction format **2010** provides access to all instruction options, while some options and operations are restricted in the 64-bit instruction format **2030**. The native instructions available in the 64-bit instruction format **2030** vary by embodiment. The instruction is compacted in part using a set of index values in an index field **2013**. The execution unit hardware references a set of compaction tables based on the index values and uses the compaction table outputs to reconstruct a native instruction in the 128-bit instruction format **2010**. Other sizes and formats of instruction can be used.

[0281] For each format, instruction opcode **2012** defines the operation that the execution unit is to perform. The execution units execute each instruction in parallel across the multiple data elements of each operand. For example, in response to an add instruction the execution unit performs a simultaneous add operation across each color channel representing a texture element or picture element. By default, the execution unit performs each instruction across all data channels of the operands. Instruction control field **2014** may enable control over certain execution options, such as channels selection (e.g., predication) and data channel order (e.g., swizzle). For instructions in the 128-bit instruction format **2010** an exec-size field **2016** limits the number of data channels that will be executed in parallel. An exec-size field **2016** may not be available for use in the compacted 64-bit instruction format **2030**.

[0282] Some execution unit instructions have up to three operands including two source operands, src0 **2020**, src1 **2022**, and one destination operand (dest **2018**). Other instructions, such as, for example, data manipulation instructions, dot product instructions, multiply-add instructions, or multiply-accumulate instructions, can have a third source operand (e.g., SRC2 **2024**). The instruction opcode **2012** determines the number of source operands. An instruction's last source operand can be an immediate (e.g., hard-coded) value passed with the instruction. The execution units may also support multiple destination instructions, where one or more of the destinations is implied or implicit based on the instruction and/or the specified destination.

[0283] The 128-bit instruction format **2010** may include an access/address mode field **2026** specifying, for example, whether direct register addressing mode or indirect register addressing mode is used. When direct register addressing mode is used, the register address of one or more operands is directly provided by bits in the instruction.

[0284] The 128-bit instruction format **2010** may also include an access/address mode field **2026**, which specifies an address mode and/or an access mode for the instruction. The access mode may be used to define a data access alignment for the instruction. Access modes including a 16-byte aligned access mode and a 1-byte aligned access mode may be supported, where the byte alignment of the access mode determines the access alignment of the instruction operands. For example, when in a first mode, the instruction may use byte-aligned addressing for source and destination operands and when in a second mode, the instruction may use 16-byte-aligned addressing for all source and destination operands.

[0285] The address mode portion of the access/address mode field **2026** may determine whether the instruction is to use direct or indirect addressing. When direct register addressing mode is used bits in the instruction directly provide the register address of one or more operands. When indirect register addressing mode is used, the register address of one or more operands may be computed based on an address register value and an address immediate field in the instruction.

[0286] Instructions may be grouped based on instruction opcode **2012** bit-fields to simplify Opcode decode **2040**. For an 8-bit opcode, bits 4, 5, and 6 allow the execution unit to determine the type of opcode. The precise opcode grouping shown is merely an example. A move and logic opcode group **2042** may include data movement and logic instructions (e.g., move (mov), compare (cmp)). The move and logic opcode group **2042** may share the five least significant bits (LSB), where move

(mov) instructions are in the form of 0000xxxxb and logic instructions are in the form of 0001xxxxb. A flow control instruction group **2044** (e.g., call, jump (jmp)) includes instructions in the form of 0010xxxxb (e.g., 0x20). A miscellaneous instruction group **2046** includes a mix of instructions, including synchronization instructions (e.g., wait, send) in the form of 0011xxxxb (e.g., 0x30). A parallel math instruction group **2048** includes component-wise arithmetic instructions (e.g., add, multiply (mul)) in the form of 0100xxxxb (e.g., 0x40). The parallel math instruction group **2048** performs the arithmetic operations in parallel across data channels. The vector math group **2050** includes arithmetic instructions (e.g., dp4) in the form of 0101xxxxb (e.g., 0x50). The vector math group performs arithmetic such as dot product calculations on vector operands. The opcode decode **2040**, in one embodiment, can be used to determine which portion of an execution unit will be used to execute a decoded instruction. For example, some instructions may be designated as systolic instructions that will be performed by a systolic array. Other instructions, such as ray-tracing instructions (not shown) can be routed to a ray-tracing core or ray-tracing logic within a slice or partition of execution logic.

Graphics Pipeline

[0287] FIG. **21** is a block diagram of a graphics processor **2100**, according to another embodiment. The elements of FIG. **21** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such.

[0288] The graphics processor **2100** may include different types of graphics processing pipelines, such as a geometry pipeline **2120**, a media pipeline **2130**, a display engine **2140**, thread execution logic **2150**, and a render output pipeline **2170**. The graphics processor **2100** may be a graphics processor within a multi-core processing system that includes one or more general-purpose processing cores. The graphics processor may be controlled by register writes to one or more control registers (not shown) or via commands issued to the graphics processor **2100** via a ring or mesh interconnect **2102**. The ring or mesh interconnect **2102** may couple the graphics processor **2100** to other processing components, such as other graphics processors or general-purpose processors. Commands from ring or mesh interconnect **2102** are interpreted by a command streamer **2103**, which supplies instructions to individual components of the geometry pipeline **2120** or the media pipeline **2130**.

[0289] Command streamer **2103** may direct the operation of a vertex fetcher **2105** that reads vertex data from memory and executes vertex-processing commands provided by command streamer **2103**. The vertex fetcher **2105** may provide vertex data to a vertex shader **2107**, which performs coordinate space transformation and lighting operations to each vertex. Vertex fetcher **2105** and vertex shader **2107** may execute vertex-processing instructions by dispatching execution threads to graphics cores **2152**A-**2152**B via a thread dispatcher **2131**.

[0290] The graphics cores **2152**A-**2152**B may be an array of vector processors having an instruction set for performing graphics and media operations. The graphics cores **2152**A-**2152**B may have an attached L1 cache **2151** that is specific for each array or shared between the arrays. The cache can be configured as a data cache, an instruction cache, or a single cache that is partitioned to contain data and instructions in different partitions.

[0291] A geometry pipeline **2120** may include tessellation components to perform hardware-accelerated tessellation of 3D objects. A programmable hull shader **2111** may configure the tessellation operations. A programmable domain shader **2117** may provide back-end evaluation of tessellation output. A tessellator **2113** may operate at the direction of a programmable hull shader **2111** and contain special purpose logic to generate a set of detailed geometric objects based on a coarse geometric model that is provided as input to geometry pipeline **2120**. In addition, if tessellation is not used, tessellation components (e.g., programmable hull shader **2111**, tessellator **2113**, and programmable domain shader **2117**) can be bypassed. The tessellation components can

operate based on data received from the vertex shader **2107**.
[0292] Complete geometric objects may be processed by a geometry shader **2119** via one or more threads dispatched to graphics cores **2152**A-**2152**B, or can proceed directly to the clipper **2129**. The geometry shader may operate on entire geometric objects, rather than vertices or patches of vertices as in previous stages of the graphics pipeline. If the tessellation is disabled, the geometry shader **2119** receives input from the vertex shader **2107**. The geometry shader **2119** may be programmable by a geometry shader program to perform geometry tessellation if the tessellation units are disabled.
[0293] Before rasterization, a clipper **2129** processes vertex data. The clipper **2129** may be a fixed function clipper or a programmable clipper having clipping and geometry shader functions. A rasterizer and depth test component **2173** in the render output pipeline **2170** may dispatch pixel shaders to convert the geometric objects into per pixel representations. The pixel shader logic may be included in thread execution logic **2150**. Optionally, an application can bypass the rasterizer and depth test component **2173** and access un-rasterized vertex data via a stream out unit **2123**.
[0294] The graphics processor **2100** has an interconnect bus, interconnect fabric, or some other interconnect mechanism that allows data and message passing amongst the major components of the processor. In some embodiments, graphics cores **2152**A-**2152**B and associated logic units (e.g., L1 cache **2151**, sampler **2154**, texture cache **2158**, etc.) interconnect via a data port **2156** to perform memory access and communicate with render output pipeline components of the processor. A sampler **2154**, L1 cache **2151**, texture cache **2158** and graphics cores **2152**A-**2152**B each may have separate memory access paths. Optionally, the texture cache **2158** can also be configured as a sampler cache.
[0295] The render output pipeline **2170** may contain a rasterizer and depth test component **2173** that converts vertex-based objects into an associated pixel-based representation. The rasterizer logic may include a windower/masker unit to perform fixed function triangle and line rasterization. An associated render cache **2178** and depth cache **2179** are also available in some embodiments. A pixel operations component **2177** performs pixel-based operations on the data, though in some instances, pixel operations associated with 2D operations (e.g., bit block image transfers with blending) are performed by the 2D engine **2141** or substituted at display time by the display controller **2143** using overlay display planes. A shared L3 cache **2175** may be available to all graphics components, allowing the sharing of data without the use of main system memory.
[0296] The media pipeline **2130** may include a media engine **2137** and a video front-end **2134**. Video front-end **2134** may receive pipeline commands from the command streamer **2103**. The media pipeline **2130** may include a separate command streamer. Video front-end **2134** may process media commands before sending the command to the media engine **2137**. Media engine **2137** may include thread spawning functionality to spawn threads for dispatch to thread execution logic **2150** via thread dispatcher **2131**.
[0297] The graphics processor **2100** may include a display engine **2140**. This display engine **2140** may be external to the graphics processor **2100** and may couple with the graphics processor via the ring or mesh interconnect **2102**, or some other interconnect bus or fabric. Display engine **2140** may include a 2D engine **2141** and a display controller **2143**. Display engine **2140** may contain special purpose logic capable of operating independently of the 3D pipeline. Display controller **2143** may couple with a display device (not shown), which may be a system integrated display device, as in a laptop computer, or an external display device attached via a display device connector.
[0298] The geometry pipeline **2120** and media pipeline **2130** maybe configurable to perform operations based on multiple graphics and media programming interfaces and are not specific to any one application programming interface (API). A driver software for the graphics processor may translate API calls that are specific to a particular graphics or media library into commands that can be processed by the graphics processor. Support may be provided for the Open Graphics Library (OpenGL), Open Computing Language (OpenCL), and/or Vulkan graphics and compute API, all

from the Khronos Group. Support may also be provided for the Direct3D library from the Microsoft Corporation. A combination of these libraries may be supported. Support may also be provided for the Open Source Computer Vision Library (OpenCV). A future API with a compatible 3D pipeline would also be supported if a mapping can be made from the pipeline of the future API to the pipeline of the graphics processor.

Graphics Pipeline Programming

[0299] FIG. **22**A is a block diagram illustrating a graphics processor command format **2200** used for programming graphics processing pipelines, such as, for example, the pipelines described herein in conjunction with FIG. **16** and FIG. **21**. FIG. **22**B is a block diagram illustrating a graphics processor command sequence **2210** according to an embodiment. The solid lined boxes in FIG. **22**A illustrate the components that are generally included in a graphics command while the dashed lines include components that are optional or that are only included in a sub-set of the graphics commands. The graphics processor command format **2200** of FIG. **22**A includes fields to identify a client **2202**, a command operation code (opcode **2204**), and a data field **2206** for the command. A sub-opcode **2205** and a command size **2208** are also included in some commands.

[0300] Client **2202** may specify the client unit of the graphics device that processes the command data. A graphics processor command parser may examine the client field of each command to condition the further processing of the command and route the command data to the appropriate client unit. The graphics processor client units may include a memory interface unit, a render unit, a 2D unit, a 3D unit, and a media unit. Each client unit may have a corresponding processing pipeline that processes the commands. Once the command is received by the client unit, the client unit reads the opcode **2204** and, if present, sub-opcode **2205** to determine the operation to perform. The client unit performs the command using information in data field **2206**. For some commands a command size **2208** is expected to explicitly specify the size of the command. The command parser may automatically determine the size of at least some of the commands based on the command opcode. Commands may be aligned via multiples of a double word. Other command formats can also be used.

[0301] The flow diagram in FIG. **22**B illustrates a graphics processor command sequence **2210**. Software or firmware of a data processing system that features an example graphics processor may use a version of the command sequence shown to set up, execute, and terminate a set of graphics operations. A sample command sequence is shown and described for purposes of example only and is not limited to these specific commands or to this command sequence. Moreover, the commands may be issued as batch of commands in a command sequence, such that the graphics processor will process the sequence of commands in at least partially concurrence.

[0302] The graphics processor command sequence **2210** may begin with a pipeline flush command **2212** to cause any active graphics pipeline to complete the currently pending commands for the pipeline. Optionally, the 3D pipeline **2222** and the media pipeline **2224** may not operate concurrently. The pipeline flush is performed to cause the active graphics pipeline to complete any pending commands. In response to a pipeline flush, the command parser for the graphics processor will pause command processing until the active drawing engines complete pending operations and the relevant read caches are invalidated. Optionally, any data in the render cache that is marked ‘dirty’ can be flushed to memory. Pipeline flush command **2212** can be used for pipeline synchronization or before placing the graphics processor into a low power state.

[0303] A pipeline select command **2213** may be used when a command sequence utilizes the graphics processor to explicitly switch between pipelines. A pipeline select command **2213** may be utilized only once within an execution context before issuing pipeline commands unless the context is to issue commands for both pipelines. A pipeline flush command **2212** may be utilized immediately before a pipeline switch via the pipeline select command **2213**.

[0304] A pipeline control command **2214** may configure a graphics pipeline for operation and may be used to program the 3D pipeline **2222** and the media pipeline **2224**. The pipeline control

command **2214** may configure the pipeline state for the active pipeline. The pipeline control command **2214** may be used for pipeline synchronization and to clear data from one or more cache memories within the active pipeline before processing a batch of commands.
[0305] Commands related to the return buffer state **2216** may be used to configure a set of return buffers for the respective pipelines to write data. Some pipeline operations cause the allocation, selection, or configuration of one or more return buffers into which the operations write intermediate data during processing. The graphics processor may also use one or more return buffers to store output data and to perform cross thread communication. The return buffer state **2216** may include selecting the size and number of return buffers to use for a set of pipeline operations.
[0306] The remaining commands in the command sequence differ based on the active pipeline for operations. Based on a pipeline determination **2220**, the command sequence is tailored to the 3D pipeline **2222** beginning with the 3D pipeline state **2230** or the media pipeline **2224** beginning at the media pipeline state **2240**.
[0307] The commands to configure the 3D pipeline state **2230** include 3D state setting commands for vertex buffer state, vertex element state, constant color state, depth buffer state, and other state variables that are to be configured before 3D primitive commands are processed. The values of these commands are determined at least in part based on the particular 3D API in use. The 3D pipeline state **2230** commands may also be able to selectively disable or bypass certain pipeline elements if those elements will not be used.
[0308] A 3D primitive **2232** command may be used to submit 3D primitives to be processed by the 3D pipeline. Commands and associated parameters that are passed to the graphics processor via the 3D primitive **2232** command are forwarded to the vertex fetch function in the graphics pipeline. The vertex fetch function uses the 3D primitive **2232** command data to generate vertex data structures. The vertex data structures are stored in one or more return buffers. The 3D primitive **2232** command may be used to perform vertex operations on 3D primitives via vertex shaders. To process vertex shaders, 3D pipeline **2222** dispatches shader execution threads to graphics processor execution units.
[0309] The 3D pipeline **2222** may be triggered via an execute **2234** command or event. A register may write trigger command executions. An execution may be triggered via a ‘go’ or ‘kick’ command in the command sequence. Command execution may be triggered using a pipeline synchronization command to flush the command sequence through the graphics pipeline. The 3D pipeline will perform geometry processing for the 3D primitives. Once operations are complete, the resulting geometric objects are rasterized and the pixel engine colors the resulting pixels. Additional commands to control pixel shading and pixel back-end operations may also be included for those operations.
[0310] The graphics processor command sequence **2210** may follow the media pipeline **2224** path when performing media operations. In general, the specific use and manner of programming for the media pipeline **2224** depends on the media or compute operations to be performed. Specific media decode operations may be offloaded to the media pipeline during media decode. The media pipeline can also be bypassed and media decode can be performed in whole or in part using resources provided by one or more general-purpose processing cores. The media pipeline may also include elements for general-purpose graphics processor unit (GPGPU) operations, where the graphics processor is used to perform SIMD vector operations using computational shader programs that are not explicitly related to the rendering of graphics primitives.
[0311] Media pipeline **2224** may be configured in a similar manner as the 3D pipeline **2222**. A set of commands to configure the media pipeline state **2240** are dispatched or placed into a command queue before the media object commands **2242**. Commands for the media pipeline state **2240** may include data to configure the media pipeline elements that will be used to process the media objects. This includes data to configure the video decode and video encode logic within the media

pipeline, such as encode or decode format. Commands for the media pipeline state **2240** may also support the use of one or more pointers to “indirect” state elements that contain a batch of state settings.

[0312] Media object commands **2242** may supply pointers to media objects for processing by the media pipeline. The media objects include memory buffers containing video data to be processed. Optionally, all media pipeline states should be valid before issuing a media object command **2242**. Once the pipeline state is configured and media object commands **2242** are queued, the media pipeline **2224** is triggered via an execute command **2244** or an equivalent execute event (e.g., register write). Output from media pipeline **2224** may then be post processed by operations provided by the 3D pipeline **2222** or the media pipeline **2224**. GPGPU operations may be configured and executed in a similar manner as media operations.

Graphics Software Architecture

[0313] FIG. **23** illustrates an example graphics software architecture for a data processing system **2300**. Such a software architecture may include a 3D graphics application **2310**, an operating system **2320**, and a processor **2330**. The processor **2330** may include a graphics processor **2332** and one or more general-purpose processor core(s) **2334**. The processor **2330** may be a variant of one of the processor(s) **1402** or any other of the processors described herein, and may be used in place thereof. Therefore, the disclosure of any features in combination with the processor(s) **1402** or any other of the processors described herein also discloses a corresponding combination with the graphics processor **2332** but is not limited to such. Moreover, the elements of FIG. **23** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. The 3D graphics application **2310** and operating system **2320** are each executed in the system memory **2350** of the data processing system.

[0314] 3D graphics application **2310** may contain one or more shader programs including shader instructions **2312**. The shader language instructions may be in a high-level shader language, such as the High-Level Shader Language (HLSL) of Direct3D, the OpenGL Shader Language (GLSL), and so forth. The application may also include executable instructions **2314** in a machine language suitable for execution by the general-purpose processor core(s) **2334**. The application may also include graphics objects **2316** defined by vertex data.

[0315] The operating system **2320** may be a Microsoft® Windows® operating system from the Microsoft Corporation, a proprietary UNIX-like operating system, or an open source UNIX-like operating system using a variant of the Linux kernel. The operating system **2320** can support a graphics API **2322** such as the Direct3D API, the OpenGL API, or the Vulkan API. When the Direct3D API is in use, the operating system **2320** uses a front-end shader compiler **2324** to compile any shader instructions **2312** in HLSL into a lower-level shader language. The compilation may be a just-in-time (JIT) compilation or the application can perform shader pre-compilation. High-level shaders may be compiled into low-level shaders during the compilation of the 3D graphics application **2310**. The shader instructions **2312** may be provided in an intermediate form, such as a version of the Standard Portable Intermediate Representation (SPIR) used by the Vulkan API.

[0316] User mode graphics driver **2326** may contain a back-end shader compiler **2327** to convert the shader instructions **2312** into a hardware specific representation. When the OpenGL API is in use, shader instructions **2312** in the GLSL high-level language are passed to a user mode graphics driver **2326** for compilation. The user mode graphics driver **2326** may use operating system kernel mode functions **2328** to communicate with a kernel mode graphics driver **2329**. The kernel mode graphics driver **2329** may communicate with graphics processor **2332** to dispatch commands and instructions.

IP Core Implementations

[0317] One or more aspects may be implemented by representative code stored on a machine-readable medium which represents and/or defines logic within an integrated circuit such as a processor. For example, the machine-readable medium may include instructions which represent various logic within the processor. When read by a machine, the instructions may cause the machine to fabricate the logic to perform the techniques described herein. Such representations, known as "IP cores," are reusable units of logic for an integrated circuit that may be stored on a tangible, machine-readable medium as a hardware model that describes the structure of the integrated circuit. The hardware model may be supplied to various customers or manufacturing facilities, which load the hardware model on fabrication machines that manufacture the integrated circuit. The integrated circuit may be fabricated such that the circuit performs operations described in association with any of the embodiments described herein.
[0318] FIG. **24** is a block diagram illustrating an IP core development system **2400** that may be used to manufacture an integrated circuit to perform operations according to an embodiment. The IP core development system **2400** may be used to generate modular, re-usable designs that can be incorporated into a larger design or used to construct an entire integrated circuit (e.g., an SOC integrated circuit). A design facility **2430** can generate a software simulation **2410** of an IP core design in a high-level programming language (e.g., C/C++). The software simulation **2410** can be used to design, test, and verify the behavior of the IP core using a simulation model **2412**. The simulation model **2412** may include functional, behavioral, and/or timing simulations. A register transfer level design (RTL design **2415**) can then be created or synthesized from the simulation model **2412**. The RTL design **2415** is an abstraction of the behavior of the integrated circuit that models the flow of digital signals between hardware registers, including the associated logic performed using the modeled digital signals. In addition to an RTL design **2415**, lower-level designs at the logic level or transistor level may also be created, designed, or synthesized. Thus, the particular details of the initial design and simulation may vary.
[0319] The RTL design **2415** or equivalent may be further synthesized by the design facility into a hardware model **2420**, which may be in a hardware description language (HDL), or some other representation of physical design data. The HDL may be further simulated or tested to verify the IP core design. The IP core design can be stored for delivery to a fabrication facility **2465** using non-volatile memory **2440** (e.g., hard disk, flash memory, or any non-volatile storage medium). The fabrication facility **2465** may be a 3.sup.rd party fabrication facility. Alternatively, the IP core design may be transmitted (e.g., via the Internet) over a wired connection **2450** or wireless connection **2460**. The fabrication facility **2465** may then fabricate an integrated circuit that is based at least in part on the IP core design. The fabricated integrated circuit can be configured to perform operations in accordance with at least one embodiment described herein.
[0320] FIG. **25**A illustrates cross-section side view of a package assembly **2590** for an integrated circuit that includes multiple units of hardware logic chiplets connected to a substrate **2580** (e.g., base die). A graphics processing unit, parallel processor, and/or compute accelerator as described herein can be composed from diverse silicon chiplets that are separately manufactured. In this context, a chiplet is an at least partially packaged integrated circuit that includes distinct units of logic that can be assembled with other chiplets into a larger package. A diverse set of chiplets with different IP core logic can be assembled into a single device. Additionally, the chiplets can be integrated into a base die or base chiplet using active interposer technology. The concepts described herein enable the interconnection and communication between the different forms of IP within the GPU. IP cores can be manufactured using different process technologies and composed during manufacturing, which avoids the complexity of converging multiple IPs, especially on a large SoC with several flavors IPs, to the same manufacturing process. Enabling the use of multiple process technologies improves the time to market and provides a cost-effective way to create multiple product SKUs. Additionally, the disaggregated IPs are more amenable to being power gated independently, components that are not in use on a given workload can be powered off, reducing

overall power consumption.
[0321] In various embodiments a package assembly **2590** can include fewer or greater number of components and chiplets that are interconnected by an interconnect fabric **2585** or a bridge structure **2587**. The bridge structure **2587** may be used to facilitate a point-to-point interconnect between, for example, a logic or I/O chiplet **2574** and memory chiplets **2575**. In some implementations, the bridge structure **2587** may also be embedded within the substrate **2580**. The chiplets within the package assembly **2590** may have a 2.5D arrangement using Chip-on-Wafer-on-Substrate (CoWoS) stacking in which multiple dies are stacked side-by-side on a silicon interposer that includes through-silicon vias (TSVs) to couple the chiplets with the substrate **2580**, which includes electrical connections to the package interconnect **2583**.
[0322] In one embodiment, silicon interposer is an active interposer **2589** that includes embedded logic in addition to TSVs. In such embodiment, the chiplets within the package assembly **2590** are arranged using 3D face to face die stacking on top of the active interposer **2589**. The active interposer **2589** can include I/O hardware logic **2591**, cache memory **2592**, and other hardware logic **2593**, in addition to the interconnect fabric **2585** and the bridge structure **2587**. The interconnect fabric **2585** enables communication between the various logic chiplets within the active interposer **2589**. The interconnect fabric **2585** may be an NoC interconnect or another form of packet switched fabric that switches data packets between components of the package assembly. For complex assemblies, the interconnect fabric **2585** may be a dedicated chiplet enables communication between the various hardware logic of the package assembly **2590**.
[0323] The hardware logic chiplets can include special purpose hardware logic chiplets **2572**, a logic or I/O chiplet **2574**, and/or memory chiplets **2575**. The special purpose hardware logic chiplets **2572** and logic or I/O chiplet **2574** may be implemented at least partly in configurable logic or fixed-functionality logic hardware and can include one or more portions of any of the processor core(s), graphics processor(s), parallel processors, or other accelerator devices described herein. The memory chiplets **2575** can be DRAM (e.g., GDDR, HBM) memory or cache (SRAM) memory. Cache memory **2592** within the active interposer **2589** (or substrate **2580**) can act as a global cache for the package assembly **2590**, part of a distributed global cache, or as a dedicated cache for the interconnect fabric **2585**.
[0324] Each chiplet can be fabricated as separate semiconductor die and coupled with a base die that is embedded within or coupled with the substrate **2580**. The coupling with the substrate **2580** can be performed via an interconnect structure **2573**. The interconnect structure **2573** may be configured to route electrical signals between the various chiplets and logic within the substrate **2580**. The interconnect structure **2573** can include interconnects such as, but not limited to bumps or pillars. In some embodiments, the interconnect structure **2573** may be configured to route electrical signals such as, for example, input/output (I/O) signals and/or power or ground signals associated with the operation of the logic, I/O and memory chiplets. In one embodiment, an additional interconnect structure couples the active interposer **2589** with the substrate **2580**.
[0325] The substrate **2580** may be an epoxy-based laminate substrate and/or may also include other suitable types of substrates. The package assembly **2590** can be connected to other electrical devices via a package interconnect **2583**. The package interconnect **2583** may be coupled to a surface of the substrate **2580** to route electrical signals to other electrical devices, such as a motherboard, other chipset, or multi-chip module.
[0326] A logic or I/O chiplet **2574** and a memory chiplet **2575** may be electrically coupled via a bridge structure **2587** that is configured to route electrical signals between the logic or I/O chiplet **2574** and a memory chiplet **2575**. The bridge structure **2587** may be a dense interconnect structure that provides a route for electrical signals. The bridge structure **2587** may include a bridge substrate composed of glass or a suitable semiconductor material. Electrical routing features can be formed on the bridge substrate to provide a chip-to-chip connection between the logic or I/O chiplet **2574** and a memory chiplet **2575**. The bridge structure **2587** may also be referred to as a silicon bridge or

an interconnect bridge. For example, the bridge structure **2587** is an Embedded Multi-die Interconnect Bridge (EMIB). Alternatively, the bridge structure **2587** may simply be a direct connection from one chiplet to another chiplet.

[0327] FIG. **25**B illustrates a package assembly **2594** including interchangeable chiplets **2595**, according to an embodiment. The interchangeable chiplets **2595** can be assembled into standardized chiplet slots or chiplet sockets on base chiplets **2596**, **2598**. The base chiplets **2596**, **2598** can be coupled via a bridge interconnect **2597**, which can be similar to the other bridge interconnects described herein and may be, for example, an EMIB. Memory chiplets can also be connected to logic or I/O chiplets via a bridge interconnect. I/O and logic chiplets can communicate via an interconnect fabric. The base chiplets can each support one or more slots in a standardized format for one of logic or I/O or memory/cache.

[0328] SRAM and power delivery circuits may be fabricated into one or more of the base chiplets **2596**, **2598**, which can be fabricated using a different process technology relative to the interchangeable chiplets **2595** that are stacked on top of the base chiplets. For example, the base chiplets **2596**, **2598** can be fabricated using a larger process technology, while the interchangeable chiplets can be manufactured using a smaller process technology. One or more of the interchangeable chiplets **2595** may be memory (e.g., DRAM) chiplets. Different memory densities can be selected for the package assembly **2594** based on the power, and/or performance targeted for the product that uses the package assembly **2594**. Additionally, logic chiplets with a different number of type of functional units can be selected at time of assembly based on the power, and/or performance targeted for the product. Additionally, chiplets containing IP logic cores of differing types can be inserted into the interchangeable chiplet slots, enabling hybrid processor designs that can mix and match different technology IP blocks.

Example System on a Chip Integrated Circuit

[0329] FIG. **26** illustrates an example integrated circuit that may be fabricated using one or more IP cores. In addition to what is illustrated, other logic and circuits may be included, including additional graphics processors/cores, peripheral interface controllers, or general-purpose processor cores. The elements of FIG. **26** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such.

[0330] The system on a chip integrated circuit **2600** includes one or more application processor(s) **2605** (e.g., CPUs), a graphics processor **2610**, which may be a variant of the graphics processor(s) **1408**, or of any graphics processor described herein and may be used in place of any graphics processor described. Therefore, the disclosure of any features in combination with a graphics processor herein also discloses a corresponding combination with the graphics processor **2610** but is not limited to such. The system on a chip integrated circuit **2600** may additionally include an image processor **2615** and/or a video processor **2620**, any of which may be a modular IP core from the same or multiple different design facilities. The system on a chip integrated circuit **2600** may include peripheral or bus logic including a USB controller **2625**, UART controller **2630**, an SPI/SDIO controller **2635**, and an I.sup.2S/I.sup.2C controller **2640**. Additionally, the integrated circuit can include a display device **2645** coupled to one or more of a high-definition multimedia interface (HDMI) controller **2650** and a reliability, availability, and serviceability engine (RAS engine **2655**). The RAS engine **2655** is used to identify potential faults that may occur during device runtime to minimize the downtime that would result if those potential faults were to occur. Storage may be provided by a flash memory subsystem **2660** including flash memory and a flash memory controller. Memory interface may be provided via a memory controller **2665** for access to SDRAM or SRAM memory devices. Some integrated circuits additionally include an embedded security engine **2670**, which may contain a source of randomness, such as Intel DRNG, secure time, secure storage, Physically Unclonable Functions (PUF), one-time programmable (OTP)

fuses, or OTP memory.

Capability-Based Memory Access Control for Graphics Processors and Accelerators

[0331] Parallel computing is a type of computation in which many calculations or the execution of processes are carried out simultaneously. Parallel computing may come in a variety of forms, including, but not limited to, SIMD or SIMT. SIMD describes computers with multiple processing elements that perform the same operation on multiple data points simultaneously. In one example, the figures discussed above refer to SIMD and its implementation in a general processor in terms of EUs, FPUs, and ALUs. In a common SIMD machine, data is packaged into registers, each containing an array of channels. Instructions operate on the data found in channel n of a register with the data found in the same channel of another register. SIMD machines are advantageous in areas where a single sequence of instructions can be simultaneously applied to high amounts of data. For example, in one embodiment, a graphics processor (e.g., GPGPU, GPU, etc.) can be used to perform SIMD vector operations using computational shader programs.

[0332] Various embodiments can also apply to use execution by use of Single Instruction Multiple Thread (SIMT) as an alternate to use of SIMD or in addition to use of SIMD. Reference to a SIMD core or operation can apply also to SIMT or apply to SIMD in combination with SIMT. The following description is discussed in terms of SIMD machines. However, embodiments herein are not solely limited to application in the SIMD context and may apply in other parallel computing paradigms, such as SIMT, for example. For ease of discussion and explanation, the following description generally focuses on a SIMD implementation. However, embodiments can similarly apply to SIMT machines with no modifications to the described techniques and methodologies. With respect to SIMT machines, similar patterns as discussed below can be followed to provide instructions to the systolic array and execute the instructions on the SIMT machine. Other types of parallel computing machines may also utilize embodiments herein as well.

[0333] As parallel rendering graphics architectures become more complex and as increasing amounts of workloads are being shifted to parallel rendering graphics architectures, increasing emphasis and attention is being placed on security vulnerabilities in these environments. In certain examples, memory corruption (e.g., by an attacker, such as another tenant in a multi-tenant environment) can be caused by buffer overflow, use after free (UAF) (e.g., a pointer that references a block of memory (e.g., a buffer) that has been de-allocated), cross-domain attacks, and/or out-of-bound access (e.g., memory access using the base address of a block of memory and an offset that exceeds the allocated size of the block), to name a few examples.

[0334] Workloads based on a parallel rendering graphics architecture should be hardened against such memory safety vulnerabilities and attacks. As parallel rendering graphics architectures continue to grow in size, supporting multi-tenant isolation in the parallel rendering graphics architecture allows for the environments to be securely divided to host multiple, isolated workloads.

[0335] In some current approaches to provide memory safety on CPUs, memory accesses can be made via capability-based access control. Capability-based access control relies on use of capabilities, which are a communicable (e.g., unforgeable) token of authority through which programs can access all memory and services within an address space indicated by the capability. The capability is a hardware type that can be held in registers or in memory. The capability is a value that references an object along with an associated set of one or more access rights. In some cases, capabilities can be specified as pointers (e.g., fat pointers, such as 128-bit pointers) that embed bounds, permissions, and so on, in addition to the pointer itself. Examples of metadata that can be embedded in capabilities include single- or double-ended bounds, tag or version, permission bits, a compartment identifier (ID), privilege level, accessed and/or dirty bits, identifier for code authorized to access the data such as a hash value, key, KeyID, tweak value or IV/counter value used by the processor circuitry to encrypt/decrypt data and/or other metadata, an aggregate cryptographic MAC value, Integrity-Check Value (ICV), or ECC code for the data allocation,

element size, e.g., to allow generating an error if an attempt is made to access an allocation at an offset that is not an even multiple of the element size, and data object size, e.g., to permit generating an exception when accessing invalid locations outside of the data object, even if the space reserved for the allocation is larger than the size utilized for the data object.

[0336] However, capability-based access control is not yet implemented in parallel rendering graphics architectures. GPUs and other accelerators differ from CPUs in ways that affect the design of a capability feature. As such, implementations of the disclosure address the above-noted technical problems by providing for capability-based memory access control for graphics processors and accelerators. In one implementation, embodiments provide for capability-based access control in GPUs and other types of accelerators.

[0337] In implementations herein, the parallel rendering graphics architecture, such as a GPU or other accelerator, implements a capability-based addressing scheme by enabling creation of capabilities in the parallel rendering graphics architecture to control which processes may access which objects in memory. Implementations further extend data storage (e.g., registers and/or memory) in the parallel rendering graphics architecture with an additional bit (referred to herein as a validity bit) that indicates that a particular location is a capability.

[0338] In implementations herein, the parallel rendering graphics architecture provides specific characteristics that the capability-based access control model can be adapted for. For example, the specific characteristics include performing coarse-grained large accesses with predictable data patterns known in advance, having multiple levels of memory in separate address spaces from main memory, data structures that do not typically contain pointers, and dedicated register space for pointers and non-pointers. Implementations herein adapt the capability-based access control model to take advantage of these specific characteristics by providing a capability instruction that checks bulk accesses in advance, generating different formats of capabilities dependent on the memory being accessed, and limiting storage of capabilities to specific memory areas/types and/or to specific registers.

[0339] For example, a ChkTag (check tag) instruction executed by the GPU may consult a tag table in memory to determine that the address of a memory granule being accessed currently matches a memory tag value stored in a memory tag for that memory granule location. The tag table may be stored in shared virtual memory (SVM) shared between the host CPU and GPU. In this way, the host CPU may allocate a buffer of memory (from example, from a shared heap), setting a tag value for each tag in the tag table for each memory granule corresponding to the granule virtual address in the allocated buffer. The CPU may then pass a pointer (i.e. virtual memory address) to that buffer's virtual memory location to the GPU including the matching tag value in the pointer corresponding to the tag values set in the tag table. The GPU, on executing a ChkTag instruction, may then use shared virtual memory to lookup the same tag table as set by the CPU during memory allocation, and verify the tag value in the pointer matches the tag value set in the tag table for that granule's memory address. If the pointer tag and memory tag table tag value do not match, the GPU may issue a memory fault or exception due to a potential memory safety violation (e.g. a buffer overflow or use-after-free condition).

[0340] A technical advantage of implementations of the disclosure includes improved memory safety in the parallel rendering graphics architecture, including improved multi-tenant sharing of GPUs and other accelerators. FIGS. **27**-**34** provide further details on the approach to implement capability-based memory access control for graphics processors and accelerators.

[0341] FIG. **27** is a block diagram illustrating an example computing system **2700** for providing capability-based memory access control for graphics processors and accelerators, according to implementations herein. The elements of FIG. **27** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. Therefore, the discussion

of any features in combination with a graphics processor herein also discloses a corresponding combination with the example computing system **2700**, but is not limited to such.
[0342] In one implementation, the computing system **2700** can include one more CPU(s) **2705** and a GPU **2710** communicably coupled to a main memory **2770**. In one implementation the GPUs **2710** may be the same as GPGPUs **806**A-**806**D described with respect to FIG. **8**, and/or GPGPU **1570** described with respect to FIG. **15**C, for example. Moreover, more than one GPU **2710** may be included in computing system **2700**.
[0343] In some implementations, CPU(s) **2705** and GPU **2710** includes a coupling (e.g., connection) to memory **2770** and/or shared memory **2775**. Memory **2770** may refer to a memory local to the CPU(s) **2705** and/or GPU **2710** (e.g., system memory). In certain examples, memory **2770** is a memory separate from the CPU(s) **2705** and/or GPU **2710**, for example, memory of a server.
[0344] In some implementations, the GPU **2710** can interconnect with host processors (e.g., one or more CPU(s) **2705**) and memory **2770**, **2775** via one or more system and/or memory busses. Shared memory **2775** may be system memory that can be shared with the one or more CPU(s) **2705**, while memory **2770** may be device memory that is dedicated to the GPU **2710**. For example, components within the GPU **2710** and memory **2775** may be mapped into memory addresses that are accessible to the one or more CPU(s) **2705**. Access to memory **2770** and **2775** may be facilitated via a memory controller (not shown). The memory controller may include an internal direct memory access (DMA) controller or can include logic to perform operations that would otherwise be performed by a DMA controller.
[0345] Note that the figures herein may not depict all data communication connections. One of ordinary skill in the art will appreciate that this is to not obscure certain details in the figures. Note that a double headed arrow in the figures may not utilize two-way communication, for example, it may indicate one-way communication (e.g., to or from that component or device). Any or all combinations of communications paths may be utilized in certain examples herein.
[0346] In one implementation, the one or more CPU(s) **2705** can write commands (e.g., instructions) into registers or memory in the GPGPU **2710** that has been mapped into an accessible address space. The command processors **2712** can read the commands from registers or memory and determine how those commands will be processed within the GPU **2710**. A thread dispatcher **2714** can then be used to dispatch threads to the compute units **2720** to perform those commands. Although a single compute unit **2720** is shown in FIG. **27**, more than one compute unit **2720** may be implemented in GPU **2710** in implementations herein. Each compute unit **2720** can execute threads independently of the other compute units **2720**. Additionally, each compute unit **2720** can be independently configured for conditional computation and can conditionally output the results of computation to memory. The command processors **2712** can interrupt the one or more CPU(s) **2705** when the submitted commands are complete.
[0347] The GPU **2710** can include multiple global cache memories, shown as cache hierarchy memory **2760**, which can include an L2 cache, L1 cache, and an instruction cache, as well as shared memory **2750**, at least a portion of which may also be partitioned as a cache memory.
[0348] Each compute unit **2720** of GPU **2710** can include a scheduler **2722** and execution circuit(s) **2730**. In some implementations, the execution circuits **2730** can include a set of execution processing circuitry **2734**, which may include vector logic units and scalar logic units. The execution processing circuitry **2734** may also include at least one matrix unit to accelerate matrix, tensor, or artificial intelligence operations and at least one ray tracing unit (not shown) to accelerate ray tracing operations. The at least one matrix unit and RT unit can include similar functionality as other matrix/tensor accelerators and ray tracing accelerators described herein.
[0349] The compute units **2720** can also include local shared memory **2724** and local cache **2726**, as well as a set of register files **2740**. The register files **2740** may include vector registers and scalar registers, for example.

[0350] In one implementation, the commands received from CPU(s) **2705** may include an instruction that is to request access to a block (or blocks) of memory storing a capability (e.g., or a pointer) and/or an instruction that is to request access to a block(s) of memory through a capability (e.g., or a pointer) to the block(s) of the memory.
[0351] In one implementation, GPU **2710** includes a capability management circuitry **2732**. Although the capability management circuitry **2732** is depicted within the execution circuits **2730**, it should be understood that the capability management circuitry **2732** can be located elsewhere, for example, in another component of GPU **2710** or separate from the depicted components of GPU **2710**.
[0352] In certain examples, capability management circuitry **2732** is to, in response to receiving an instruction that is requested for fetch, decode, and/or execution, check if the instruction is a capability instruction or a non-capability instruction (e.g., a capability-unaware instruction), for example.
[0353] If the instruction is a capability instruction, the capability management circuitry **2732** is to allow access to memory (e.g., register files **2740**, local shared memory **2724**, local cache **2726**, shared memory **2750**, cache hierarchy memory **2760**, and/or main memory **2770**) storing (**1**) a capability and/or (**2**) data and/or instructions (e.g., an object) protected by a capability.
[0354] If the instruction is a non-capability instruction, the capability management circuitry **2732** is not to allow access to memory (e.g., register files **2740**, local shared memory **2724**, local cache **2726**, shared memory **2750**, cache hierarchy memory **2760**, and/or main memory **2770**) storing (**1**) a capability and/or (**2**) data and/or instructions (e.g., an object) protected by a capability.
[0355] In some implementations, capability management circuitry **2732** is to check if an instruction is a capability instruction or a non-capability instruction by checking (i) a field (e.g., opcode) of the instruction (e.g., checking a corresponding bit or bits of the field that indicate if that instruction is a capability instruction or a non-capability instruction) and/or (ii) if a particular register is a "capability" type of register (e.g., instead of a general-purpose data register) (e.g., implying that certain register(s) are not to be used to store a capability or capabilities).
[0356] In some implementations, capability management circuitry **2732** is to manage the capabilities, e.g., the capability management circuitry **2732** is to set and/or clear validity tags **2725**, **2727**, **2755**, **2765**. In certain examples, capability management circuitry **2732** is to clear the validity tag **2725**, **2727**, **2755**, **2765** of a capability in a register, such as capability register **2744** in response to that register being written to by a non-capability instruction.
[0357] In certain examples, capability management circuitry **2732** is to enforce security properties on changes to capability data (e.g., metadata) by enforcing: (i) provenance validity that ensures that valid capabilities can only be constructed by instructions that do so explicitly (e.g., not by byte manipulation) from other valid capabilities (e.g., with this property applying to capabilities in registers and in memory), (ii) capability monotonicity that ensures, when any instruction constructs a new capability (e.g., except in sealed capability manipulation and exception raising), it cannot exceed the permissions and bounds of the capability from which it was derived, and/or (iii) reachable capability monotonicity that ensures, in any execution of arbitrary code, until execution is yielded to another domain, the set of reachable capabilities (e.g., those accessible to the current program state via registers, memory, sealing, unsealing, and/or constructing sub-capabilities) cannot increase.
[0358] In some implementations, the capability is stored in a single line of data. In some implementations, the capability is stored in multiple lines of data. In certain examples, capabilities (e.g., one or more fields thereof) themselves can be stored in registers, such as in registers of register files **2740**, and/or can also be stored in memory. For example, the capabilities (with corresponding validity tags) can be stored in one or more of local shared memory **2724** as capability/tags **2725**, local cache **2726** as capability/tags **2727**, shared memory **2750** as capability/tags **2755**, cache hierarchy memory **2760** as capability/tags **2765**, and so on. The

capability/tags **2725**, **2727**, **2755**, **2765** may be stored in data structures (e.g., table) specific for capabilities in memory. In certain examples, a validity tag is stored in the data structure for a capability stored in memory. In some implementations, validity tags are not accessible by non-capability (e.g., load and/or store) instructions. In certain examples, a validity tag is stored along with the capability stored in memory (e.g., in one contiguous block).

[0359] As previously noted, depicted GPU **2710** can also include register files **2740**. Register files **2740** may include one or more registers, for example, general purpose (e.g., data) register(s) **2742** and/or (optional) (e.g., dedicated only for capabilities) capability registers **2744**.

[0360] In one implementation, an indication (e.g., name) of a destination register for a capability in register(s) **2744** is an operand of an (e.g., supervisor level) instruction (e.g., having a mnemonic of LoadCap) that is to load the capability from memory into register(s) **2744**. In certain examples, an indication (e.g., name) of the source register for capability in register(s) **2744** is an operand of an (e.g., supervisor level) instruction (e.g., having a mnemonic of StoreCap) that is to store the capability from the register(s) **2744** into memory (e.g., memory **2724**, **2726**, **2750**, **2760**, **2770**).

[0361] In one implementation, the source storage location (e.g., virtual address) for a capability in memory (e.g., memory **2724**, **2726**, **2750**, **2760**, **2770**) is an operand of an (e.g., supervisor level or user level) instruction (e.g., having a mnemonic of LoadCap) that is to load the capability from the memory into register(s) **2744**. In certain examples, the destination storage location (e.g., virtual address) for capability in memory is an operand of an (e.g., supervisor level) instruction (e.g., having a mnemonic of StoreCap) that is to store the capability from the register(s) **2744** into memory. In certain examples, the instruction is requested for execution by executing some privileged process authorized to do so and/or by executing user code.

[0362] As previously noted, conventional capability-based access control has not been implemented in parallel rendering graphics architectures, such as GPUs and other accelerators. Such GPUs and accelerators differ from CPUs in ways that affect the design of a capability feature. As such, implementations of the disclosure provide for capability-based memory access control for graphics processors and accelerators. As shown in FIG. **27**, GPU **2710** can implement a capability-based addressing scheme by enabling support for capabilities in the GPU **2710**, where the capabilities control which processes may access which objects in memory of the GPU **2710**. Implementations further extend data storage (e.g., registers and/or memory) in the GPU **2710** with an additional bit (e.g., validity bit) that indicates that a particular location is a capability.

[0363] In some implementations, as previously discussed, a ChkTag (check tag) instruction executed by the GPU **2710** may consult a tag table **2780** in memory **2775** to determine that the address of a memory granule being accessed currently matches a memory tag value stored in a memory tag for that memory granule location. The tag table **2780** may be stored in shared virtual memory (SVM) of shared memory **2775** that is shared between the host CPU **2705** and GPU **2710**. In this way, the host CPU **2705** may allocate a buffer of memory (from example, from a shared heap), setting a tag value for each tag in the tag table **2780** for each memory granule corresponding to the granule virtual address in the allocated buffer.

[0364] The CPU **2705** may then pass a pointer (i.e. virtual memory address) to that buffer's virtual memory location to the GPU **2710** including the matching tag value in the pointer corresponding to the tag values set in the tag table **2780**. The GPU **2710**, on executing a ChkTag instruction, may then use the shared virtual memory to lookup the same tag table as set by the CPU **2705** during memory allocation, and verify the tag value in the pointer matches the tag value set in the tag table **2780** for that granule's memory address. If the pointer tag and memory tag table tag value do not match, the GPU **2710** may issue a memory fault or exception due to a potential memory safety violation (e.g. a buffer overflow or use-after-free condition).

[0365] In implementations herein, GPU **2710** (and other accelerator architectures) provides specific characteristics that the capability-based access control model can be adapted for. For example, the specific characteristics include performing coarse-grained large accesses with predictable data

patterns known in advance, having multiple levels of memory in separate address spaces from main memory, data structures that do not typically contain pointers, and dedicated register space for pointers and non-pointers. Implementations herein adapt the capability-based access control model to take advantage of these specific characteristics by providing a capability instruction that checks bulk accesses in advance, generating different formats of capabilities dependent on the memory being accessed, and limiting storage of capabilities to specific memory areas/types and/or to specific registers.

[0366] Further details of enabling a capability-based access control model in accordance with the GPU/accelerator-specific characteristics are further discussed below with respect to FIG. **28**.

[0367] FIG. **28** is a block diagram illustrating an example capability management circuit **2800** for providing capability-based memory access control for graphics processors and accelerators, according to implementations herein. In one implementation, capability management circuit **2800** may be the same as capability management circuitry **2732** described with respect to FIG. **27**. As such, the elements of FIG. **28** having the same or similar names as the elements of FIG. **27** herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. Therefore, the discussion of any features in combination with a graphics processor herein also discloses a corresponding combination with the example capability management circuit **2800**, but is not limited to such.

[0368] In one implementation, the capability management circuit **2800** can include a bulk access capability checker **2810**, a memory-specific capability generator **2820**, a capability location designator **2830**, a capability decryptor **2850**, and/or a default capability generator **2860**. More or less components than those shown in capability management circuit **2800** may be included in accordance with implementations herein.

[0369] With respect to bulk access capability checker **2810**, GPUs, such as GPU **2710** of FIG. **27** hosting capability management circuit **2800**, can perform coarse-grained, large accesses to memory (e.g., memory **2724**, **2726**, **2750**, **2760**, **2770**) with predictable access patterns that are determined far in advance with no branches that can cut accesses short. The implication of this characteristic is that there is an opportunity to check bulk accesses in advance. For example, implementations herein can provide a new bulk access capability instruction for the bulk access checker. The bulk access capability instruction can accept a base address and a size for a large access (as operands) and checks the entire access, generating an exception if the bounds check fails. Accordingly, subsequent data accesses do not have to be checked individually against the capability. Further details of the bulk access capability check are described below with respect to FIG. **30**.

[0370] With respect to memory-specific capability generator **2820**, GPUs, such as GPU **2710** of FIG. **27** hosting capability management circuit **2800**, have multiple levels of local, fast memories that are in separate address spaces from main memory. For example, memories **2724**, **2726**, **2750**, **2760** of GPU are examples of the local, fast memories of GPU **2710** in separate address spaces from main memory **2770**. The characteristic of GPU **2710** provides an opportunity to use smaller capabilities (in terms of capability format) when accessing these small, local memories.

[0371] In some implementations, capability management circuit **2800** can use capability-based access control for enforcing memory safety and low-overhead compartmentalization. In some implementations, capabilities utilize a larger data width than a pointer, for example, using (e.g., at least 128-bit or in some cases at least 129-bit) registers that are wider than the registers (e.g., 64-bit) used to store pointers to allow for storage of a capability (e.g., a pointer along with its bounds, permissions, and/or other metadata). This can introduce overhead within the design for expanded register files. Some example capabilities allow for (e.g., 128-bit or 129-bit) capabilities to span a plurality of (e.g., 64-bit) registers without expanding the register file, for example, without expanding the (e.g., logical) width of general-purpose registers to the capability width and/or without defining an additional, separate register file to hold the width of the capability because

expanding registers and/or introducing new registers increases processor (e.g., silicon) area overhead and timing latency.
[0372] In some implementations herein, a capability may have different formats and/or fields. In some examples, a capability is more than twice the width of a native (e.g., integer) pointer type of the baseline architecture, for example, 129-bit capabilities on 64-bit platforms, 65-bit capabilities on 32-bit platforms, and so on. In certain examples, each capability includes an (e.g., integer) address of the natural size for the architecture (e.g., 32 or 64 bit) and also additional metadata (e.g., that is compressed in order to fit) in the remaining (e.g., 32 or 64) bits of the capability. In certain examples, each capability includes (or is associated with) a (e.g., 1-bit) validity “tag” whose value is maintained in registers and memory by the architecture (e.g., by capability management circuit **108**). In certain examples, each element of the capability contributes to the protection model and is enforced by hardware (e.g., capability management circuit **108**). Further discussion of specialized formats for capabilities as implemented in GPU **2710** is discussed below with respect to FIG. **28**.
[0373] In certain examples, when stored in memory, valid capabilities are to be naturally aligned (e.g., at 64-bit or 128-bit boundaries) depending on capability size where that is the granularity at which in-memory tags are maintained. In certain examples, partial or complete overwrites with data, rather than a complete overwrite with a valid capability, lead to the in-memory tag being cleared, preventing corrupted capabilities from later being dereferenced. In certain examples, capability compression reduces the memory footprint of capabilities, e.g., such that the full capability, including address, permissions, and bounds fits within a certain width (e.g., 128 bits plus a 1-bit out-of-band tag). In certain examples, capability compression takes advantage of redundancy between the address and the bounds, which occurs where a pointer typically falls within (or close to) its associated allocation. In certain examples, the compression scheme uses a floating-point representation, allowing high-precision bounds for small objects, but uses stronger alignment and padding for larger allocations.
[0374] As noted above, GPU memories **2724**, **2726**, **2750**, **2760** can be local, fast memories that exist in separate address spaces from main memory **2770**. This characteristic of GPU **2710** memories provides an opportunity to use smaller capabilities (in terms of capability format) when accessing these small, local memories. For example, in one implementation, instead of having to start with a 64-bit pointer as a capability that is usable for addressing the full main memory **2770** and further expanding that to store bounds and other capability info, a much smaller pointer can be used by memory-specific capability generator **2820** of capability management circuit **2800**.
[0375] For example, if the local memory (e.g., local shared memory **2724**, local cache **2726**, etc.) is only 1MiB (mebibyte), the memory-specific capability generator **2820** may utilize capabilities for this local memory that are smaller (e.g., 20 bits) for the capability (e.g., pointer) itself. Similarly, bounds within the capability can also be made smaller. This both reduces storage overheads as well as power and performance overheads for checking bounds. Further details of the memory-specific capabilities of GPU are further discussed below with respect to FIG. **31**.
[0376] With respect to capability location designator **2830**, GPUs, such as GPU **2710** of FIG. **27** hosting capability management circuit **2800**, often have data structures that do not typically contain pointers. Moreover, little of the register space of GPUs is usable for storing pointers and is instead primarily utilized for non-pointer data. In implementations here, this characteristic of GPUs (and other accelerator devices), such as GPU **2710** of FIG. **27**, avoids having to utilize the approach of allowing capabilities to be stored anywhere in memory.
[0377] Instead, in implementations herein, the capability location designator **2830** of capability management circuit **2800** can designate certain regions of memory (e.g., one or more of memories **2724**, **2726**, **2750**, **2760** of GPU **2710** of FIG. **27**) as storing capabilities. Once designated, it is those regions of memory that incur the additional storage overheads of per-word validity bits and cache area overheads of propagating validity bits along with memory words.
[0378] In implementations herein, the capability location designator **2830** can dynamically

configure the various pages or regions of memory designated for capabilities. In some implementations, the capability-enabled designation could be determined based on the location of the memory in the memory hierarchy. For example, small local memories could support capabilities even if main memory does not. In some implementations, the capability location designator **2830** is responsible for initializing capabilities in the designated local memories in a trustworthy manner.

[0379] Furthermore, the capability location designator **2830** includes a capability-specific register designator **2840** that can also designate particular registers, such as capability registers **2744** of register files **2740** of GPU **2710** in FIG. **27**, as registers capable of storing capabilities. As a result, relatively fewer registers are used to store capabilities and there is less demand to expand registers for capabilities as compared to traditional CPU implementations for capability-based access control models.

[0380] Further details of the capability location designation of GPUs are further discussed below with respect to FIG. **32**.

[0381] With respect to capability decryptor **2850**, GPUs, such as GPU **2710** of FIG. **27** hosting capability management circuit **2800**, may receive capabilities that are specified by a CPU (or other host device) and passed through main memory. In some implementations, for data security purposes, the memory containing the capabilities is encrypted on the CPU prior to being passed to the GPU.

[0382] In implementations herein, the capability decryptor **2850** of capability management circuit **2800** can be used to orchestrate the process to decrypt the block containing the capabilities in local memory of the GPU, where it cannot be tampered with by other devices in the system. The host device, such as a CPU, may generate capabilities on the CPU referring to linear addresses in main memory shared via a shared virtual memory with the GPU. The CPU then can position the capabilities within the memory image at locations expected by the GPU. The CPU encrypts the memory image and includes a header specifying the locations of all capabilities in the memory image and transfers that encrypted memory image to the GPU where it is decrypted.

[0383] In one implementation, the capability decryptor **2850** at the GPU identifies an indicator in the decrypted memory image, such as a table within the transferred data (memory image), that indicates where the capabilities are within the memory image. The capability decryptor **2850** may then identify the capabilities in the memory image and set up the capabilities (e.g., set their corresponding validity bits) in the GPU's memory in order to protect the capabilities from that point on.

[0384] This approach provides a way for the capabilities and validity bits to be transferred from the CPU to the GPU while making sure that there is no tampering with the transmission. Further details of the capability decryption of GPUs are further discussed below with respect to FIG. **33**.

[0385] With respect to default capability generator **2860**, GPUs, such as GPU **2710** of FIG. **27** hosting capability management circuit **2800**, may be configured to run workloads for multiple tenants in a multi-tenancy configuration. In the case of coarse-grained multi-tenancy implementations (in the absence of fine-grained memory safety), certain capabilities rarely have to be updated to point to different regions of memory, as a tenant mostly operates on its own, large chunk of data drawn from main memory and then processed further in local memories. This characteristic creates opportunities to define certain registers (or memory) as containing capabilities that are consulted by default in the absence of a more specific capability being supplied for an access. This can overlay isolation boundaries on the address space defined by the IOMMU configuration.

[0386] In implementations herein, the default capability generator **2860** can establish and configure such default capabilities. In one example, FIG. **29** is a block diagram illustrating a computing system **2900** supporting default capability registers for multi-tenancy, in accordance with implementations herein. In one implementation, computing system **2900** may be the same as

computing system **2700** described with respect to FIG. **27**. As shown in FIG. **29**, computing system **2900** may include a GPU **2905** supporting multiple tenants including GPU-Tenant 1 **2910-1** and GPU-Tenant 2 **2910-2**. In one implementation, GPU **2905** is the same as GPU **2710** described with respect to FIG. **27**. Each GPU-Tenant **2910-1**, **2910-2** may access defined portions of capability-accessed memory **2920** using capabilities. Capability-accessed memory **2920** may have a memory region 1 **2920-1** that is accessible by GPU-Tenant 1 **2910-1**, and a memory region 2 **2920-2** that is accessible by GPU-Tenant 2 **2910-2**.

[0387] In implementations herein, each memory region **2920-1**, **2920-2** may be associated with a default capability register **2915-1**, **2915-2**. The default capability registers **2915-1**, **2915-2** can store a default capability for the corresponding memory region **2920-1**, **2920-2**. The capabilities stored in the default capability registers **2915-1**, **2915-2** can be checked by default when the GPU **2905** does not supply any specific object-granular capability. In some implementations, the workload can be defined with more specific object-granular capabilities for memory within the memory regions **2920-1**, **2920-2** as well.

[0388] Further details of the default capability generation of GPUs are further discussed below with respect to FIG. **34**.

[0389] FIG. **30** is a flow diagram illustrating an embodiment of a method **3000** for providing bulk capability checking in a capability-based memory access control for graphics processors and accelerators. Method **3000** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **3000** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity, clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-29** may not be repeated or discussed hereafter. In one implementation, a processor implementing a capability management circuit, such as capability management circuitry **2732** of FIG. **27** and/or capability management circuit **2800** of FIG. **28**, may perform method **3000**.

[0390] Method **3000** begins at processing block **3010** where the processor may determine that a set of data accesses triggers a bulk access capability check. Then, at block **3020**, the processor may generate bulk access capability check to combine with the set of data accesses.

[0391] Subsequently, at block **3030**, the processor may, prior to performing a first data access of the set of data accesses, perform a bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability. Lastly, at block **3040**, the processor may, responsive to the bulk access capability check passing, allow the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.

[0392] FIG. **31** is a flow diagram illustrating an embodiment of a method **3100** for providing memory-specific capability generation for capability-based memory access control for graphics processors and accelerators. Method **3100** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **3100** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity, clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-30** may not be repeated or discussed hereafter. In one implementation, capability management circuit, such as capability management circuitry **2732** of FIG. **27** and/or capability management circuit **2800** of FIG. **28**, may perform method **3100**.

[0393] Method **3100** begins at processing block **3110** where the processor may receive request to generate a capability in a graphics processor. Then, at block **3120**, the processor may identify type of memory that the capability is to reference.

[0394] Subsequently, at block **3130**, the processor may determine format of the capability based on the type of memory, where the format of the capability can vary based on the type of memory. Lastly, at block **3140**, The processor may generate the capability in accordance with the determined format.

[0395] FIG. **32** is a flow diagram illustrating an embodiment of a method **3200** for providing capability location designation for capability-based memory access control for graphics processors and accelerators. Method **3200** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **3200** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity, clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-31** may not be repeated or discussed hereafter. In one implementation, capability management circuit, such as capability management circuitry **2732** of FIG. **27** and/or capability management circuit **2800** of FIG. **28**, may perform method **3200**.

[0396] Method **3200** begins at processing block **3210** where the processor may initiate a capability configuration process in a graphics processing system. Then, at block **3220**, the processor may identify regions of memory of the graphics processing system to designate as capable of storing capabilities.

[0397] Subsequently, at block **3230**, the processor may designate one or more of the identified regions as capability storage regions. At block **3240**, the processor may receive request to generate a capability. Lastly, at block **3250**, the processor may cause the capability to be generated in one of the capability storage regions.

[0398] FIG. **33** is a flow diagram illustrating an embodiment of a method **3300** for providing capability decryption and configuration for capability-based memory access control for graphics processors and accelerators. Method **3300** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **3300** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity, clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-32** may not be repeated or discussed hereafter. In one implementation, capability management circuit, such as capability management circuitry **2732** of FIG. **27** and/or capability management circuit **2800** of FIG. **28**, may perform method **3300**.

[0399] Method **3300** begins at processing block **3310** where the processor may receive, at a graphics processing system, an encrypted memory image from a host device, where the encrypted memory image includes a header specifying locations of capabilities within the memory image. At block **3320**, the processor may decrypt the memory image into local memory for the GPU.

[0400] Subsequently, at block **3330**, the processor may read the header of the memory image specifying the locations of the capabilities. Then, at block **3340**, the processor may utilize a capability instruction to set the validity bits for each of the locations of the capabilities. Lastly, at block **3350**, the processor may execute workload containing one or more of the capabilities on the GPU.

[0401] FIG. **34** is a flow diagram illustrating an embodiment of a method **3400** for providing default capability generation for capability-based memory access control for graphics processors and accelerators. Method **3400** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **3400** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity,

clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-33** may not be repeated or discussed hereafter. In one implementation, capability management circuit, such as capability management circuitry **2732** of FIG. **27** and/or capability management circuit **2800** of FIG. **28**, may perform method **3400**.

[0402] Method **3400** begins at processing block **3410** where the processor may identify memory region assigned to a tenant hosted by a GPU. Then, at block **3420**, the processor may configure a register of the GPU as a default capability register for storing a capability for the entire memory region of the tenant.

[0403] Subsequently, at block **3430**, the processor may determine that an access is requested to the memory region of the tenant. Lastly, at block **3440**, the processor may perform capability check using capability for the memory region in the default capability register.

[0404] The following examples pertain to further embodiments. Example 1 is an apparatus to facilitate capability-based memory access control for graphics processors and accelerators. The apparatus of Example 1 includes one or more processing cores to: determine that a set of data accesses triggers a bulk access capability check; generate the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, perform the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allow the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.

[0405] In Example 2, the subject matter of Example 1 can optionally include wherein the capability comprises a data structure that references an object and comprises a set of one or more access rights, and wherein the set of one or more access rights comprises at least one of bounds or permissions. In Example 3, the subject matter of any one of Examples 1-2 can optionally include wherein the one or more processing cores further to: responsive to the bulk access capability check failing, generating an exception for the set of data accesses.

[0406] In Example 4, the subject matter of any one of Examples 1-3 can optionally include wherein the one or more processing cores are further to: receive a request to generate the capability; identify a type of memory that the capability is to reference; determine a format of the capability based on the type of the memory, wherein the format of the capability can vary based on the type of the memory; and generate the capability in accordance with the format.

[0407] In Example 5, the subject matter of any one of Examples 1-4 can optionally include wherein the one or more processing cores are further to: initiate a capability configuration process; identify regions of memory to designate as capable of storing capabilities; designate one or more of the regions as capability storage regions; receive a request to generate the capability; and cause the capability to be generated in one of the capability storage regions.

[0408] In Example 6, the subject matter of any one of Examples 1-5 can optionally include wherein the one or more processing cores are further to: receive an encrypted memory image from a host device, wherein the encrypted memory image comprises a header specifying locations of capabilities within the memory image; decrypt the memory image into local memory; read the header of the memory image specifying the locations of the capabilities; utilize a capability instruction to set validity bits for each of the locations of the capabilities; and execute a workload containing one or more of the capabilities.

[0409] In Example 7, the subject matter of any one of Examples 1-6 can optionally include wherein the one or more processing cores are further to: identify a memory region assigned to a tenant hosted using the one or more processing cores; configure a register of the one or more processing cores as a default capability register for storing a default capability for the memory region of the tenant; determine that an access is requested to the memory region of the tenant; and perform a default capability check using the default capability for the memory region in the default capability register.

[0410] In Example 8, the subject matter of any one of Examples 1-7 can optionally include wherein the one or more processing cores comprises a graphics processing unit (GPU). In Example 9, the subject matter of any one of Examples 1-8 can optionally include wherein the one or more processing cores are at least one of a single instruction multiple data (SIMD) machine or a single instruction multiple thread (SIMT) machine.
[0411] Example 10 is a method for facilitating capability-based memory access control for graphics processors and accelerators. The method of Example 10 can include determining, by one or more processing cores, that a set of data accesses triggers a bulk access capability check; generating the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, performing the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allowing the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.
[0412] In Example 11, the subject matter of Example 10 can optionally include wherein the capability comprises a data structure that references an object and comprises a set of one or more access rights, and wherein the set of one or more access rights comprises at least one of bounds or permissions. In Example 12, the subject matter of Examples 10-11 can optionally include further comprising: receiving a request to generate the capability; identifying a type of memory that the capability is to reference; determining a format of the capability based on the type of the memory, wherein the format of the capability can vary based on the type of the memory; and generating the capability in accordance with the format.
[0413] In Example 13, the subject matter of Examples 10-12 can optionally include further comprising: initiating a capability configuration process; identifying regions of memory to designate as capable of storing capabilities; designating one or more of the regions as capability storage regions; receiving a request to generate the capability; and causing the capability to be generated in one of the capability storage regions.
[0414] In Example 14, the subject matter of Examples 10-13 can optionally include further comprising: receiving an encrypted memory image from a host device, wherein the encrypted memory image comprises a header specifying locations of capabilities within the memory image; decrypting the memory image into local memory; reading the header of the memory image specifying the locations of the capabilities; utilizing a capability instruction to set validity bits for each of the locations of the capabilities; and executing a workload containing one or more of the capabilities.
[0415] In Example 15, the subject matter of Examples 10-14 can optionally include further comprising: identifying a memory region assigned to a tenant hosted using the one or more processing cores; configuring a register of the one or more processing cores as a default capability register for storing a default capability for the memory region of the tenant; determining that an access is requested to the memory region of the tenant; and performing a default capability check using the default capability for the memory region in the default capability register.
[0416] Example 16 is a non-transitory computer-readable storage medium for facilitating capability-based memory access control for graphics processors and accelerators. The non-transitory computer-readable storage medium of Example 16 having instructions stored thereon, which when executed by one or more processors, cause the one or more processors to perform operations comprising: determining, by one or more processing cores of the one or more processors, that a set of data accesses triggers a bulk access capability check; generating the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, performing the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allowing the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.

[0417] In Example 17, the subject matter of Example 16 can optionally include wherein the operations further comprise: receiving a request to generate the capability; identifying a type of memory that the capability is to reference; determining a format of the capability based on the type of the memory, wherein the format of the capability can vary based on the type of the memory; and generating the capability in accordance with the format.
[0418] In Example 18, the subject matter of Examples 16-17 can optionally include wherein the operations further comprise: initiating a capability configuration process; identifying regions of memory to designate as capable of storing capabilities; designating one or more of the regions as capability storage regions; receiving a request to generate the capability; and causing the capability to be generated in one of the capability storage regions. In Example 19, the subject matter of Examples 16-18 can optionally include wherein the operations further comprise: receiving an encrypted memory image from a host device, wherein the encrypted memory image comprises a header specifying locations of capabilities within the memory image; decrypting the memory image into local memory; reading the header of the memory image specifying the locations of the capabilities; utilizing a capability instruction to set validity bits for each of the locations of the capabilities; and executing a workload containing one or more of the capabilities.
[0419] In Example 20, the subject matter of Examples 16-19 can optionally include wherein the operations further comprise: identifying a memory region assigned to a tenant hosted using the one or more processing cores; configuring a register of the one or more processing cores as a default capability register for storing a default capability for the memory region of the tenant; determining that an access is requested to the memory region of the tenant; and performing a default capability check using the default capability for the memory region in the default capability register.
[0420] Example 21 is a system for facilitating capability-based memory access control for graphics processors and accelerators. The system of Example 21 can optionally include a memory; and a processor communicably coupled to the memory and comprising one or more processing cores to: determine that a set of data accesses triggers a bulk access capability check; generate the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, perform the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allow the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.
[0421] In Example 22, the subject matter of Example 21 can optionally include wherein the capability comprises a data structure that references an object and comprises a set of one or more access rights, and wherein the set of one or more access rights comprises at least one of bounds or permissions. In Example 23, the subject matter of any one of Examples 21-22 can optionally include wherein the one or more processing cores further to: responsive to the bulk access capability check failing, generating an exception for the set of data accesses.
[0422] In Example 24, the subject matter of any one of Examples 21-23 can optionally include wherein the one or more processing cores are further to: receive a request to generate the capability; identify a type of memory that the capability is to reference; determine a format of the capability based on the type of the memory, wherein the format of the capability can vary based on the type of the memory; and generate the capability in accordance with the format.
[0423] In Example 25, the subject matter of any one of Examples 21-24 can optionally include wherein the one or more processing cores are further to: initiate a capability configuration process; identify regions of memory to designate as capable of storing capabilities; designate one or more of the regions as capability storage regions; receive a request to generate the capability; and cause the capability to be generated in one of the capability storage regions.
[0424] In Example 26, the subject matter of any one of Examples 21-25 can optionally include wherein the one or more processing cores are further to: receive an encrypted memory image from a host device, wherein the encrypted memory image comprises a header specifying locations of

capabilities within the memory image; decrypt the memory image into local memory; read the header of the memory image specifying the locations of the capabilities; utilize a capability instruction to set validity bits for each of the locations of the capabilities; and execute a workload containing one or more of the capabilities.
[0425] In Example 27, the subject matter of any one of Examples 21-26 can optionally include wherein the one or more processing cores are further to: identify a memory region assigned to a tenant hosted using the one or more processing cores; configure a register of the one or more processing cores as a default capability register for storing a default capability for the memory region of the tenant; determine that an access is requested to the memory region of the tenant; and perform a default capability check using the default capability for the memory region in the default capability register.
[0426] In Example 28, the subject matter of any one of Examples 21-27 can optionally include wherein the one or more processing cores comprises a graphics processing unit (GPU). In Example 29, the subject matter of any one of Examples 21-28 can optionally include wherein the one or more processing cores are at least one of a single instruction multiple data (SIMD) machine or a single instruction multiple thread (SIMT) machine.
[0427] Example 30 is an apparatus for facilitating capability-based memory access control for graphics processors and accelerators, comprising means for determining that a set of data accesses triggers a bulk access capability check; means for generating the bulk access capability check to combine with the set of data accesses; means for performing, prior to performing a first data access of the set of data accesses, the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and means for allowing, responsive to the bulk access capability check passing, the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses. In Example 31, the subject matter of Example 30 can optionally include the apparatus further configured to perform the method of any one of the Examples 11 to 15.
[0428] Example 32 is at least one machine readable medium comprising a plurality of instructions that in response to being executed on a computing device, cause the computing device to carry out a method according to any one of Examples 10-15. Example 33 is an apparatus for facilitating capability-based memory access control for graphics processors and accelerators, configured to perform the method of any one of Examples 10 to 15. Example 34 is an apparatus for facilitating capability-based memory access control for graphics processors and accelerators, comprising means for performing the method of any one of Examples 10 to 15. Specifics in the Examples may be used anywhere in one or more embodiments.
The foregoing description and drawings are to be regarded in an illustrative rather than a restrictive sense. Persons skilled in the art will understand that various modifications and changes may be made to the embodiments described herein without departing from the broader spirit and scope of the features set forth in the appended claims.

Claims

1. An apparatus comprising: one or more processing cores to: determine that a set of data accesses triggers a bulk access capability check; generate the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, perform the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allow the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.
2. The apparatus of claim 1, wherein the capability comprises a data structure that references an object and comprises a set of one or more access rights, and wherein the set of one or more access

rights comprises at least one of bounds or permissions.
3. The apparatus of claim 1, wherein the one or more processing cores further to: responsive to the bulk access capability check failing, generating an exception for the set of data accesses.
4. The apparatus of claim 1, wherein the one or more processing cores are further to: receive a request to generate the capability; identify a type of memory that the capability is to reference; determine a format of the capability based on the type of the memory, wherein the format of the capability can vary based on the type of the memory; and generate the capability in accordance with the format.
5. The apparatus of claim 1, wherein the one or more processing cores are further to: initiate a capability configuration process; identify regions of memory to designate as capable of storing capabilities; designate one or more of the regions as capability storage regions; receive a request to generate the capability; and cause the capability to be generated in one of the capability storage regions.
6. The apparatus of claim 1, wherein the one or more processing cores are further to: receive an encrypted memory image from a host device, wherein the encrypted memory image comprises a header specifying locations of capabilities within the memory image; decrypt the memory image into local memory; read the header of the memory image specifying the locations of the capabilities; utilize a capability instruction to set validity bits for each of the locations of the capabilities; and execute a workload containing one or more of the capabilities.
7. The apparatus of claim 1, wherein the one or more processing cores are further to: identify a memory region assigned to a tenant hosted using the one or more processing cores; configure a register of the one or more processing cores as a default capability register for storing a default capability for the memory region of the tenant; determine that an access is requested to the memory region of the tenant; and perform a default capability check using the default capability for the memory region in the default capability register.
8. The apparatus of claim 1, wherein the one or more processing cores are comprised in a graphics processing unit (GPU).
9. The apparatus of claim 1, wherein the one or more processing cores are at least one of a single instruction multiple data (SIMD) machine or a single instruction multiple thread (SIMT) machine.
10. A method comprising: determining, by one or more processing cores, that a set of data accesses triggers a bulk access capability check; generating the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, performing the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allowing the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.
11. The method of claim 10, wherein the capability comprises a data structure that references an object and comprises a set of one or more access rights, and wherein the set of one or more access rights comprises at least one of bounds or permissions.
12. The method of claim 10, further comprising: receiving a request to generate the capability; identifying a type of memory that the capability is to reference; determining a format of the capability based on the type of the memory, wherein the format of the capability can vary based on the type of the memory; and generating the capability in accordance with the format.
13. The method of claim 10, further comprising: initiating a capability configuration process; identifying regions of memory to designate as capable of storing capabilities; designating one or more of the regions as capability storage regions; receiving a request to generate the capability; and causing the capability to be generated in one of the capability storage regions.
14. The method of claim 10, further comprising: receiving an encrypted memory image from a host device, wherein the encrypted memory image comprises a header specifying locations of capabilities within the memory image; decrypting the memory image into local memory; reading

the header of the memory image specifying the locations of the capabilities; utilizing a capability instruction to set validity bits for each of the locations of the capabilities; and executing a workload containing one or more of the capabilities.

15. The method of claim 10, further comprising: identifying a memory region assigned to a tenant hosted using the one or more processing cores; configuring a register of the one or more processing cores as a default capability register for storing a default capability for the memory region of the tenant; determining that an access is requested to the memory region of the tenant; and performing a default capability check using the default capability for the memory region in the default capability register.

16. A non-transitory computer-readable medium having instructions stored thereon, which when executed by one or more processors, cause the one or more processors to perform operations comprising: determining, by one or more processing cores of the one or more processors, that a set of data accesses triggers a bulk access capability check; generating the bulk access capability check to combine with the set of data accesses; prior to performing a first data access of the set of data accesses, performing the bulk access capability check to check that the set of data accesses is allowed to be accessed in accordance with a capability; and responsive to the bulk access capability check passing, allowing the set of data accesses to proceed without performing a capability check for each individual data access of the set of data accesses.

17. The non-transitory computer-readable medium of claim 16, wherein the operations further comprise: receiving a request to generate the capability; identifying a type of memory that the capability is to reference; determining a format of the capability based on the type of the memory, wherein the format of the capability can vary based on the type of the memory; and generating the capability in accordance with the format.

18. The non-transitory computer-readable medium of claim 16, wherein the operations further comprise: initiating a capability configuration process; identifying regions of memory to designate as capable of storing capabilities; designating one or more of the regions as capability storage regions; receiving a request to generate the capability; and causing the capability to be generated in one of the capability storage regions.

19. The non-transitory computer-readable medium of claim 16, wherein the operations further comprise: receiving an encrypted memory image from a host device, wherein the encrypted memory image comprises a header specifying locations of capabilities within the memory image; decrypting the memory image into local memory; reading the header of the memory image specifying the locations of the capabilities; utilizing a capability instruction to set validity bits for each of the locations of the capabilities; and executing a workload containing one or more of the capabilities.

20. The non-transitory computer-readable medium of claim 16, wherein the operations further comprise: identifying a memory region assigned to a tenant hosted using the one or more processing cores; configuring a register of the one or more processing cores as a default capability register for storing a default capability for the memory region of the tenant; determining that an access is requested to the memory region of the tenant; and performing a default capability check using the default capability for the memory region in the default capability register.